CSES Analysis Notebook

100 newest substantial analyses · tasks 3406 down to 1711

Mechanical filter: at least 900 source characters.
Short introductory analyses are omitted.

Contents

3406 · Triangle Number Sums
3403 · Longest Common Subsequence
3399 · Raab Game I
3398 · Permutation Rounds
3397 · Permutation Order
3359 · Minimal Grid Path
3356 · Distinct Values Queries II
3355 · Sum of Four Squares
3314 · Mountain Range
3304 · Visible Buildings Queries
3226 · Subarray Sum Queries II
3217 · Knight Moves Grid
3163 · Range Interval Queries
3154 · System of Linear Equations
2428 · Distinct Values Subarrays II
2420 · Palindrome Queries
2417 · Counting Coprime Pairs
2416 · Increasing Array Queries
2413 · Counting Towers
2229 · Permutation Inversions
2220 · Counting Numbers
2217 · Collecting Numbers II
2214 · Inverse Inversions
2210 · Counting Grids
2209 · Counting Necklaces
2207 · Grundy's Game
2206 · Pizzeria Queries
2205 · Gray Code
2187 · Bracket Sequences II
2186 · Special Substrings
2185 · Prime Multiples
2184 · Missing Coin Sum Queries
2183 · Missing Coin Sum
2182 · Divisor Analysis
2181 · Counting Tilings
2177 · Strongly Connected Edges
2169 · Nested Ranges Count
2168 · Nested Ranges Check
2166 · Prefix Sum Queries
2165 · Tower of Hanoi
2164 · Josephus Queries
2163 · Josephus Problem II
2143 · Reachability Queries
2138 · Reachable Nodes
2137 · Corner Subgrid Count
2136 · Hamming Distance
2134 · Path Queries II
2133 · Dynamic Connectivity
2110 · Substring Distribution
2109 · Substring Order II
2108 · Substring Order I
2107 · String Functions
2106 · Repeating Substring
2105 · Distinct Substrings
2104 · Pattern Positions
2103 · Counting Patterns
2102 · Finding Patterns
2101 · New Roads Queries
2088 · Knuth Division
2087 · Houses and Schools
2086 · Subarray Squares
2085 · Monster Game II
2084 · Monster Game I
2081 · Fixed-Length Paths II
2080 · Fixed-Length Paths I
2079 · Finding a Centroid
2077 · Necessary Cities
2076 · Necessary Roads
2072 · Cut and Paste
2064 · Bracket Sequences I
1755 · Palindrome Reorder
1753 · String Matching
1751 · Planets Cycles
1750 · Planets Queries I
1749 · List Removals
1748 · Increasing Subsequence II
1746 · Array Description
1745 · Money Sums
1744 · Rectangle Cutting
1743 · String Reorder
1740 · Intersection Points
1739 · Forest Queries II
1737 · Range Queries and Copies
1736 · Polynomial Queries
1735 · Range Updates and Sums
1734 · Distinct Values Queries
1733 · Finding Periods
1732 · Finding Borders
1731 · Word Combinations
1728 · Inversion Probability
1726 · Moving Robots
1725 · Dice Probability
1724 · Graph Paths II
1723 · Graph Paths I
1722 · Fibonacci Numbers
1717 · Christmas Party
1716 · Distributing Apples
1715 · Creating Strings II
1713 · Counting Divisors
1711 · Distinct Routes

3406 · Triangle Number Sums

Task · source analysis length 1,262 characters

Problem statement

A triangle number is a positive integer of the form $1 + 2 + \cdots + k$. The first triangle numbers are 1, 3, 6, 10 and 15.

Every positive integer can be represented as a sum of triangle numbers. For example, $42 = 21 + 21$ and $1337 = 1326 + 10 + 1$.

Given a positive integer n, determine the smallest number of triangle numbers that sum to n.

Input

The first line has an integer t: the number of tests.

After that, each line has a positive integer n.

Output

For each test, print the smallest number of triangle numbers.

Constraints

- $1 \le t \le 100$
- $1 \le n \le 10^{12}$

It can be shown that any positive integer can be represented as a sum of at most three triangle numbers. Thus, the answer is always 1, 2 or 3.

First, we precalculate a list of all triangle numbers between $1 \ldots 10^{12}$. The list contains about $1.4 \cdot 10^6$ numbers.

The answer is 1 if the list contains n. Otherwise, the answer is 2 if the list contains two numbers a and b that sum to n, which can be checked using a two-pointer approach. Otherwise, we know that the answer is 3.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

int main() {
    int t;
    cin >> t;

    vector<ll> triangles;
    ll triangle = 0;
    for (int i = 1;; i++) {
        triangle += i;
        if (triangle > 1e12) break;
        triangles.push_back(triangle);
    }

    while (t--) {
        ll n;
        cin >> n;

        if (find(triangles.begin(), triangles.end(), n) != triangles.end()) {
            cout << "1\n";
```

```
            continue;
        }

        bool found = false;

        int right = triangles.size() - 1;
        for (int left = 0; left <= right; left++) {
            while (right > 0 && triangles[left] + triangles[right] > n) {
                right--;
            }
            if (triangles[left] + triangles[right] == n) {
                cout << "2\n";
                found = true;
                break;
            }
        }

        if (!found) {
            cout << "3\n";
        }
    }
}
```

References

- Fermat polygonal number theorem

3403 · Longest Common Subsequence

Task · source analysis length 1,584 characters

Problem statement

Given two arrays of integers, find their longest common subsequence.

A subsequence is a sequence of array elements from left to right that can contain gaps. A common subsequence is a subsequence that appears in both arrays.

Input

The first line has two integers n and m: the sizes of the arrays.

The second line has n integers $a_1, a_2, \ldots, a_n$: the contents of the first array.

The third line has m integers $b_1, b_2, \ldots, b_m$: the contents of the second array.

Output

First print the length of the longest common subsequence.

After that, print an example of such a sequence. If there are several solutions, you can print any of them.

Constraints

- $1 \le n, m \le 1000$
- $1 \le a_i, b_i \le 10^9$

We use dynamic programming to calculate the longest common subsequence length for all prefix pairs of the given arrays.

Let $\mathrm{lcs}[i][j]$ denote the longest common subsequence length for the sequences $[a_1, a_2, \ldots, a_i]$ and $[b_1, b_2, \ldots, b_j]$. If $a_i = b_j$, then $\mathrm{lcs}[i][j] = \mathrm{lcs}[i-1][j-1] + 1$ because the last elements of the sequences can be matched. Otherwise, $\mathrm{lcs}[i][j] = \max(\mathrm{lcs}[i-1][j], \mathrm{lcs}[i][j-1])$, which corresponds to removing the last element either from the first or the second sequence.

Finally, $\mathrm{lcs}[n][m]$ gives the longest common subsequence length and we can construct an example of such a subsequence by tracking the dynamic programming calculation path.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<int> a(n + 1), b(m + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int i = 1; i <= m; i++) {
        cin >> b[i];
    }

    vector<vector<int>> lcs(n + 1, vector<int>(m + 1));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
```

```
            if (a[i] == b[j]) {
                lcs[i][j] = lcs[i - 1][j - 1] + 1;
            } else {
                lcs[i][j] = max(lcs[i - 1][j], lcs[i][j - 1]);
            }
        }
    }

    cout << lcs[n][m] << "\n";

    vector<int> result;
    while (lcs[n][m] > 0) {
        if (a[n] == b[m]) {
            result.push_back(a[n]);
            n--;
            m--;
        } else if (lcs[n - 1][m] == lcs[n][m]) {
            n--;
        } else {
            m--;
        }
    }

    reverse(result.begin(), result.end());
    for (auto x : result) {
        cout << x << " ";
    }
    cout << "\n";
}
```

References

- Longest common subsequence *Wikipedia*

3399 · Raab Game I

Task · source analysis length 1,139 characters

Problem statement

Consider a two player game where each player has n cards numbered $1, 2, \ldots, n$. On each turn both players place one of their cards on the table. The player who placed the higher card gets one point. If the cards are equal, neither player gets a point. The game continues until all cards have been played.

You are given the number of cards n and the players' scores at the end of the game, a and b. Your task is to give an example of how the game could have played out.

Input

The first line contains one integer t: the number of tests.

Then there are t lines, each with three integers n, a and b.

Output

For each test case print YES if there is a game with the given outcome and NO otherwise.

If the answer is YES, print an example of one possible game. Print two lines representing the order in which the players place their cards. You can give any valid example.

Constraints

- $1 \le t \le 1000$
- $1 \le n \le 100$
- $0 \le a, b \le n$

If $a + b > n$, there is no solution because the total number of points cannot exceed the number of turns.

If exactly one player has 0 points, there is no solution because it is not possible for the other player to win some rounds while all other rounds result in ties.

Otherwise, we can always construct a solution as follows:

- The first player always plays $1, 2, \ldots, n$
- The second player first plays $a + 1, a + 2, \ldots, a + b$, then $1, 2, \ldots, a$ and finally $a + b + 1, a + b + 2, \ldots, n$.

In this construction, the first player first loses b times and then wins a times. After these rounds, there are $n - a - b$ ties.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int t;
    cin >> t;

    while (t--) {
        int n, a, b;
        cin >> n >> a >> b;

        if (a + b > n) {
            cout << "NO\n";
```

```
            continue;
        }

        if ((a == 0 || b == 0) && a + b != 0) {
            cout << "NO\n";
            continue;
        }

        cout << "YES\n";
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }
        cout << "\n";
        for (int i = 1; i <= b; i++) {
            cout << a + i << " ";
        }
        for (int i = 1; i <= a; i++) {
            cout << i << " ";
        }
        for (int i = a + b + 1; i <= n; i++) {
            cout << i << " ";
        }
        cout << "\n";
    }
}
```

3398 · Permutation Rounds

Task · source analysis length 1,560 characters

Problem statement

There is a sorted array $[1, 2, \ldots, n]$ and a permutation $p_1, p_2, \ldots, p_n$. On each round, all elements move according to the permutation: the element at position i moves to position p_i.

After how many rounds is the array sorted again for the first time?

Input

The first line has an integer n.

The next line contains n integers $p_1, p_2, \ldots, p_n$.

Output

Print the number of rounds modulo $10^9 + 7$.

Constraints

- $1 \le n \le 2 \cdot 10^5$

We can present the permutation as a set of cycles. For example, the permutation $[5, 3, 2, 6, 4, 1, 8, 7]$ consists of the following cycles:

- $1 \to 5 \to 4 \to 6 \to 1$ (length 4)
- $2 \to 3 \to 2$ (length 2)
- $7 \to 8 \to 7$ (length 2)

The answer to the problem is the least common multiple lcm of all cycle lengths. In the above permutation, the answer is $\operatorname{lcm}(4, 2, 2$=4).

The following solution determines all cycle lengths and finds the prime factorization of each cycle length. We calculate the lcm by going through all primes that appear in the cycle lengths. For each prime x, we multiply the result by x^k where x^k is the greatest power that divides any of the cycle lengths.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

int main() {
    int n;
    cin >> n;

    vector<int> order(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> order[i];
    }

    vector<int> max_count(n + 1);

    for (int i = 1; i <= n; i++) {
        if (order[i] == 0) {
            continue;
        }

        int pos = i;
```

```
        int length = 0;
        do {
            length++;
            int new_pos = order[pos];
            order[pos] = 0;
            pos = new_pos;
        } while (pos != i);

        for (int x = 2; x * x <= length; x++) {
            int count = 0;
            while (length % x == 0) {
                length /= x;
                count++;
            }
            max_count[x] = max(max_count[x], count);
        }
        if (!max_count[length]) {
            max_count[length] = 1;
        }
    }

    ll result = 1;
    for (int x = 2; x <= n; x++) {
        for (int i = 0; i < max_count[x]; i++) {
            result *= x;
            result %= M;
        }
    }
    cout << result << "\n";
}
```

3397 · Permutation Order

Task · source analysis length 1,763 characters

Problem statement

Let $p(n, k)$ denote the kth permutation $in lexicographical order$ of $1 \ldots n$. For example, $p(4, 1$=

$$1, 2, 3, 4$$

) and $p(4, 2$=

$$1, 2, 4, 3$$

).

Your task is to process two types of tests:

1. Given n and k, find $p(n, k)$
2. Given n and $p(n, k)$, find k

Input

The first line has an integer t: the number of tests.

Each test is either "1 n k" or "2 n $p(n, k)$".

Output

For each test, print the answer according to the example.

Constraints

- $1 \leq t \leq 1000$
- $1 \leq n \leq 20$
- $1 \leq k \leq n!$

First task

We are given n and k, and our task is to find $p(n, k)$, i.e., the kth permutation $in lexicographical order$ of $1 \ldots n$.

If k is between 1 and $(n-1!)$, the first number is 1. If k is between $(n-1!+1)$ and $2(n-1!)$, the first number is 2. In general, the first number is

$$\left\lfloor \frac{k-1}{(n-1)!} \right\rfloor + 1.$$

We can find the numbers one by one using the same idea. After every step, we remove the chosen number from the list of possible numbers.

Second task

We are given n and $p(n, k)$ and our task is to find k.

Initially $k = 1$. If the first number is 1, we don't change k. If the first number is 2, we increase k by $(n - 1!)$. If the first number is 3, we increase k by $2(n - 1!)$. In general, if the first number is x, we increase k by

$$(x - 1)(n - 1)!.$$

We can process the numbers one by one and increase the value of k using the same idea. After every step, we remove x from the list of numbers.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;
using ll = long long;
const int N = 20;

int main() {
    int t;
    cin >> t;

    vector<ll> fact(N + 1);
    fact[0] = 1;
    for (int i = 1; i <= N; i++) {
        fact[i] = fact[i - 1] * i;
    }

    while (t--) {
        int op;
        cin >> op;

        int n;
        cin >> n;

        ll f = fact[n];
        vector<int> numbers(n);
        iota(numbers.begin(), numbers.end(), 1);

        if (op == 1) {
            ll k;
            cin >> k;
            k--;
            for (int i = 0; i < n; i++) {
                f /= n - i;
                cout << numbers[k / f] << " ";
                numbers.erase(numbers.begin() + (k / f));
                k %= f;
            }
            cout << "\n";
        }

        if (op == 2) {
            ll k = 1;
            for (int i = 0; i < n; i++) {
                int x;
                cin >> x;
                f /= n - i;
                auto it = find(numbers.begin(), numbers.end(), x);
                k += (it - numbers.begin()) * f;
                numbers.erase(it);
            }
            cout << k << "\n";
        }
    }
}
```

3359 · Minimal Grid Path

Task · source analysis length 1,549 characters

Problem statement

You are given an $n \times n$ grid whose each square contains a letter.

You should move from the upper-left square to the lower-right square. You can only move right or down.

What is the lexicographically minimal string you can construct?

Input

The first line has an integer n: the size of the grid.

After this, there are n lines that describe the grid. Each line has n letters between A and Z.

Output

Print the lexicographically minimal string.

Constraints

- $1 \le n \le 3000$

The algorithm consists of $2n - 2$ rounds that construct the minimal string corresponding to a path from the top-left square to the bottom-right square. It maintains a prefix of this minimal string throughout the process.

In each round, we examine all possible ending positions of paths that correspond to the current prefix. The prefix is extended by adding a new letter. From each ending position, we consider moving one step to the right and one step down to determine the next letter to add to the path.

It is important to ensure that the list of ending positions contains only distinct positions after each round. Otherwise, the algorithm could process the same ending positions multiple times, making it inefficient.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<string> grid(n);
    for (int i = 0; i < n; i++) {
        cin >> grid[i];
    }

    cout << grid[0][0];

    vector<pair<int, int>> level;
    level.emplace_back(0, 0);

    for (int i = 0; i < 2 * n - 2; i++) {
        vector<pair<int, int>> next_level;
        char min_letter = 'Z' + 1;

        auto update_next_level = [&](int y, int x) {
            if (y < n && x < n) {
                char letter = grid[y][x];
                if (letter < min_letter) {
```

```
                    next_level.clear();
                    min_letter = letter;
                }
                if (letter == min_letter) {
                    next_level.emplace_back(y, x);
                }
            }
        };

        for (auto [y, x] : level) {
            update_next_level(y + 1, x);
            update_next_level(y, x + 1);
        }

        cout << min_letter;

        level = next_level;
        level.erase(unique(level.begin(), level.end()), level.end());
    }
    cout << "\n";
}
```

3356 · Distinct Values Queries II

Task · source analysis length 2,415 characters

Problem statement

Given an array of n integers, your task is to process q queries of the following types:

1. update the value at position k to u
2. check if every value in range $[a, b]$ is distinct

Input

The first line has two integers n and q: the number of values and queries.

The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has three integers: either "$1\ k\ u$" or "$2\ a\ b$".

Output

For each query of type 2, print YES if every value in the range is distinct and NO otherwise.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le x_i, u \le 10^9$
- $1 \le k \le n$
- $1 \le a \le b \le n$

We use a segment tree that supports range minimum queries. For each array index, we store the next array index that contains the same value, or ∞ if there is no such index. Each array value in range $[a, b]$ is distinct exactly when the minimum value in the segment tree for the range $[a, b]$ is greater than b.

To update the tree, we use an additional set structure that contains pairs of the form (x, i), where i is the array index and x is the value at that index. This set allows us to efficiently find both the next and previous array indices that have the same value.

The time complexity of the solution is $O((n + q$ \log n)).

```
#include <iostream>
#include <set>
#include <vector>
using namespace std;
const int INF = 999999999;
const int N = 1 << 18;

int tree[2 * N];

void set_val(int pos, int value) {
    pos += N;
    tree[pos] = value;
    for (pos /= 2; pos >= 1; pos /= 2) {
        tree[pos] = min(tree[2 * pos], tree[2 * pos + 1]);
    }
}

int get_min(int a, int b) {
    a += N;
    b += N;
    int result = tree[a];
```

```
    while (a <= b) {
        if (a % 2 == 1) {
            result = min(result, tree[a++]);
        }
        if (b % 2 == 0) {
            result = min(result, tree[b--]);
        }
        a /= 2;
        b /= 2;
    }
    return result;
}

set<pair<int, int>> pairs;

int find_next(int pos, int value) {
    auto it = pairs.upper_bound({value, pos});
    if (it == pairs.end() || it->first != value) {
        return INF;
    }
    return it->second;
}

int find_prev(int pos, int value) {
    auto it = pairs.lower_bound({value, pos});
    if (it == pairs.begin()) {
        return 0;
    }
    it--;
    if (it->first != value) {
        return 0;
    }
    return it->second;
}

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> values(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> values[i];
        pairs.insert({values[i], i});
    }

    for (int i = 1; i < 2 * N; i++) {
        tree[i] = INF;
    }

    for (int i = 1; i <= n; i++) {
        int next_pos = find_next(i, values[i]);
        set_val(i, next_pos);
    }

    while (q--) {
        int op;
        cin >> op;

        if (op == 1) {
            int k, u;
            cin >> k >> u;

            int prev_pos, next_pos;

            prev_pos = find_prev(k, values[k]);
            next_pos = find_next(k, values[k]);
            set_val(prev_pos, next_pos);
            pairs.erase({values[k], k});

            values[k] = u;

            prev_pos = find_prev(k, values[k]);
            set_val(prev_pos, k);
            next_pos = find_next(k, values[k]);
            set_val(k, next_pos);
            pairs.insert({values[k], k});
        }

        if (op == 2) {
            int a, b;
            cin >> a >> b;
            int pos = get_min(a, b);
            if (pos <= b) {
                cout << "NO\n";
            } else {
                cout << "YES\n";
```

```
            }
        }
    }
}
```

3355 · Sum of Four Squares

Task · source analysis length 1,057 characters

Problem statement

A well known result in number theory is that every non-negative integer can be represented as the sum of four squares of non-negative integers.

You are given a non-negative integer n. Your task is to find four non-negative integers a, b, c and d such that $n = a^2 + b^2 + c^2 + d^2$.

Input

The first line has an integer t: the number of test cases.

Each of the next t lines has an integer n.

Output

For each test case, print four non-negative integers a, b, c and d that satisfy $n = a^2 + b^2 + c^2 + d^2$.

Constraints

- $1 \le t \le 1000$
- $0 \le n \le 10^7$
- the sum of all n is at most 10^7

Let $N = 10^7$ denote the maximum value of n.

We first find all integers between 0 and N that can be expressed as a sum of two squares. This can be done in $O(N)$ time. For each such integer $x = a^2 + b^2$, we store the first number a and the second number b.

After that, we can find a solution for any n in $O(n)$ time by finding a number x such that both x and $n - x$ can be expressed as a sum of two squares. It can be shown that there is always a solution.

```
#include <iostream>
#include <vector>
using namespace std;
const int N = 1e7;

int main() {
    int t;
    cin >> t;

    vector<bool> can(N + 1);
    vector<int> first(N + 1), second(N + 1);
    for (int a = 0; a * a <= N; a++) {
        for (int b = 0; a * a + b * b <= N; b++) {
            int x = a * a + b * b;
            can[x] = true;
            first[x] = a;
            second[x] = b;
        }
    }

    while (t--) {
        int n;
        cin >> n;
        for (int x = 0; x <= n; x++) {
            if (can[x] && can[n - x]) {
                cout << first[x] << " " << second[x] << " ";
```

```
                cout << first[n - x] << " " << second[n - x] << "\n";
                break;
            }
        }
    }
}
```

References

- Lagrange's four-square theorem *Wikipedia*

3314 · Mountain Range

Task · source analysis length 3,364 characters

Problem statement

There are n mountains in a row, each with a specific height. You begin your hang gliding route from some mountain.

You can glide from mountain a to mountain b if mountain a is taller than mountain b and all mountains between a and b.

What is the maximum number of mountains you can visit on your route?

Input

The first line has an integer n: the number of mountains.

The next line has n integers $h_1, h_2, \ldots, h_n$: the heights of the mountains.

Output:

Print one integer: the maximum number of mountains.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq h_i \leq 10^9$

Solution 1 $O(nlogn)$

We process the mountains in order of decreasing height and calculate the maximum length of a route that ends at each mountain.

For each mountain, we find the nearest higher mountain to the left and to the right. Since the mountains are processed in order of decreasing height, the maximum route length has already been calculated for both of these mountains. The longest route to the current mountain can be created by choosing the longest route to the left or right mountain and adding the current mountain to it.

There may be several mountains with the same height which makes the situation more complex. For this reason, we maintain an additional list of updates and apply the updates whenever the current height differs from the previous height.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<pair<int, int>> heights;
    for (int i = 1; i <= n; i++) {
        int h;
```

```
        cin >> h;
        heights.emplace_back(h, i);
    }
    sort(heights.rbegin(), heights.rend());

    map<int, int> counts;
    counts[0] = 0;
    counts[n + 1] = 0;

    int prev_height = 0;
    vector<pair<int, int>> updates;
    int result = 0;

    for (auto [height, pos] : heights) {
        if (height != prev_height) {
            for (auto [pos, count] : updates) {
                counts[pos] = count;
            }
            updates.clear();
            prev_height = height;
        }

        auto it = counts.upper_bound(pos);
        int next_pos = it->first;
        it--;
        int prev_pos = it->first;

        int new_count = max(counts[prev_pos], counts[next_pos]) + 1;
        updates.emplace_back(pos, new_count);
        result = max(result, new_count);
    }

    cout << result << "\n";
}
```

Solution 2 $O(n)$

It is also possible to solve the problem in linear time by using a different method for processing the mountains.

First, we find the nearest higher mountain to the left and to the right for each mountain, which can be done in linear time using stacks. We then create a graph that determines the order in which the mountains should be processed.

Next, we process the graph using topological sorting and calculate the maximum route length for each mountain, applying the same idea as in the previous solution.

```
#include <iostream>
#include <vector>
using namespace std;
const int INF = 1000000001;

int main() {
    int n;
    cin >> n;

    vector<int> heights(n + 2);
    heights[0] = INF;
    heights[n + 1] = INF;
    for (int i = 1; i <= n; i++) {
        cin >> heights[i];
    }

    vector<vector<int>> after(n + 2);

    vector<int> left_stack;
    left_stack.push_back(0);
    for (int i = 1; i <= n; i++) {
        while (heights[i] >= heights[left_stack.back()]) {
            left_stack.pop_back();
        }
        after[left_stack.back()].push_back(i);
        left_stack.push_back(i);
    }

    vector<int> right_stack;
    right_stack.push_back(n + 1);
    for (int i = n; i >= 1; i--) {
```

```
        while (heights[i] >= heights[right_stack.back()]) {
            right_stack.pop_back();
        }
        after[right_stack.back()].push_back(i);
        right_stack.push_back(i);
    }

    vector<int> count(n + 2);
    vector<int> result(n + 2);

    vector<int> queue;
    queue.push_back(0);
    queue.push_back(n + 1);

    int answer = 0;
    for (int i = 0; i < queue.size(); i++) {
        int pos = queue[i];
        for (auto next_pos : after[pos]) {
            result[next_pos] = max(result[next_pos], result[pos] + 1);
            answer = max(answer, result[next_pos]);
            count[next_pos]++;
            if (count[next_pos] == 2) {
                queue.push_back(next_pos);
            }
        }
    }
    cout << answer << "\n";
}
```

3304 · Visible Buildings Queries

Task · source analysis length 1,324 characters

Problem statement

There are n buildings in a row numbered $1, 2, \ldots, n$ from left to right. You are standing to the left of the first building. You can see a building if it is taller than all buildings to its left.

Your task is to process q queries: If only buildings in range $[a, b]$ existed, how many buildings would you see?

Input

The first line has two integers n and q: the number of buildings and queries.

The second line has n integers $h_1, h_2, \ldots, h_n$: the heights of the buildings.

Finally, there are q lines describing the queries. Each line has two integers a and b.

Output

For each query, print one integer: the number of visible buildings.

Constraints

- $1 \le n \le 10^5$
- $1 \le q \le 2 \cdot 10^5$
- $1 \le h_i \le 10^9$
- $1 \le a \le b \le n$

We first find the index of the next higher building for each building (or ∞ if there is no such building). This is done in linear time using a stack.

After that, we create a jump array that, for each building, indicates the next higher building to the right after x steps where x is a power of two. Since there are $O(\log n)$ possible powers of two, creating the array takes $O(n \log n)$ time.

Finally, we can answer any query in $O(\log n)$ time using the jump array.

```
#include <iostream>
#include <vector>
using namespace std;
const int N = 17;
const int INF = 1000000001;

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> heights(n + 2);
    for (int i = 1; i <= n; i++) {
        cin >> heights[i];
    }
    heights[n + 1] = INF;

    vector<vector<int>> jump(N, vector<int>(n + 2));
    jump[0][n + 1] = n + 1;

    vector<int> stack;
    stack.push_back(n + 1);
```

```
    for (int i = n; i >= 1; i--) {
        while (heights[stack.back()] <= heights[i]) {
            stack.pop_back();
        }
        jump[0][i] = stack.back();
        stack.push_back(i);
    }

    for (int k = 1; k < N; k++) {
        for (int i = 1; i <= n + 1; i++) {
            jump[k][i] = jump[k - 1][jump[k - 1][i]];
        }
    }

    while (q--) {
        int a, b;
        cin >> a >> b;
        int pos = a;
        int answer = 1;
        for (int k = N - 1; k >= 0; k--) {
            if (jump[k][pos] <= b) {
                answer += 1 << k;
                pos = jump[k][pos];
            }
        }
        cout << answer << "\n";
    }
}
```

3226 · Subarray Sum Queries II

Task · source analysis length 1,713 characters

Problem statement

You are given an array of n integers and q queries. In each query, your task is to calculate the maximum subarray sum in the range $[a, b]$.

Empty subarrays (with sum 0) are allowed.

Input

The first line contains two integers n and q: the number of elements and the number of queries.

Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Finally there are q lines that describe the queries. Each line has two integers a and b.

Output

Print the answer for each query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $-10^9 \le x_i \le 10^9$
- $1 \le a \le b \le n$

We create a segment tree where each node stores four values:

- `total_sum`: the total sum of values in the range
- `max_left`: the maximum sum starting from the left end of the range
- `max_right`: the maximum sum ending at the right end of the range
- `max_sum`: the maximum subarray sum in the range

Using these four values, we can efficiently calculate the maximum subarray sum for any given range.

The solution works in $O((n + q$ \log n)) time. While the problem does not require it, this approach also supports update queries.

```
#include <iostream>
using namespace std;
using ll = long long;
const int N = 1 << 18;

struct Node {
    ll total_sum, max_left, max_right, max_sum;
};

Node tree[2 * N];

Node combine(Node left, Node right) {
    Node result;
    result.total_sum = left.total_sum + right.total_sum;
    result.max_left = max(left.max_left, left.total_sum + right.max_left);
    result.max_right = max(right.max_right, left.max_right + right.total_sum);
    result.max_sum =
        max(max(left.max_sum, right.max_sum), left.max_right + right.max_left);
    return result;
}
```

```
void set_val(int pos, int value) {
    pos += N;
    tree[pos] = {value, value, value, value};

    for (pos /= 2; pos >= 1; pos /= 2) {
        tree[pos] = combine(tree[pos * 2], tree[pos * 2 + 1]);
    }
}

ll get_sum(int a, int b) {
    a += N;
    b += N;

    Node left = {}, right = {};
    while (a <= b) {
        if (a % 2 == 1) {
            left = combine(left, tree[a++]);
        }
        if (b % 2 == 0) {
            right = combine(tree[b--], right);
        }
        a /= 2;
        b /= 2;
    }

    return combine(left, right).max_sum;
}

int main() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        set_val(i, x);
    }

    for (int i = 1; i <= q; i++) {
        int a, b;
        cin >> a >> b;
        cout << get_sum(a, b) << "\n";
    }
}
```

3217 · Knight Moves Grid

Task · source analysis length 1,149 characters

Problem statement

There is a knight on an $n \times n$ chessboard. For each square, print the minimum number of moves the knight needs to do to reach the top-left corner.

Input

The only line has an integer n.

Output

Print the number of moves for each square.

Constraints

- $4 \le n \le 1000$

We first place the number 0 in the top-left square. Subsequently, for $x = 1, 2, 3, \ldots$, we place the number x in all unvisited squares that can be reached in one move from a square containing the number $x - 1$. This process continues until all squares have been visited.

This approach can be efficiently implemented using a queue that stores squares in order of increasing distance. The algorithm is similar to the breadth-first search algorithm.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<vector<int>> grid(n, vector<int>(n, -1));
    vector<pair<int, int>> queue;

    grid[0][0] = 0;
    queue.emplace_back(0, 0);

    int dy[8] = {1, 1, -1, -1, 2, 2, -2, -2};
    int dx[8] = {2, -2, 2, -2, 1, -1, 1, -1};

    for (int i = 0; i < queue.size(); i++) {
        auto [y, x] = queue[i];
        for (int d = 0; d < 8; d++) {
            int ny = y + dy[d];
            int nx = x + dx[d];
            if (ny < 0 || ny >= n || nx < 0 || nx >= n || grid[ny][nx] != -1) {
                continue;
            }
            grid[ny][nx] = grid[y][x] + 1;
            queue.emplace_back(ny, nx);
        }
    }

    for (int y = 0; y < n; y++) {
        for (int x = 0; x < n; x++) {
            cout << grid[y][x] << " ";
        }
        cout << "\n";
```

```
    }
}
```

3163 · Range Interval Queries

Task · source analysis length 3,434 characters

Problem statement

Given an array x of n integers, your task is to process q queries of the form: how many integers i satisfy $a \leq i \leq b$ and $c \leq x_i \leq d$?

Input

The first line has two integers n and q: the number of values and queries.

The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has four integers a, b, c and d: how many integers i satisfy $a \leq i \leq b$ and $c \leq x_i \leq d$?

Output

Print the result of each query.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$
- $1 \leq a \leq b \leq n$
- $1 \leq c \leq d \leq 10^9$

Solution 1 ($O(n \log n + q \log^2 n)$)

We create a segment tree where each node contains a sorted list of values in the corresponding subtree. We can create the tree in $O(n \log n)$ time using a merge sort like technique.

After that, we can process each query in $O(\log^2 n)$ time by using binary search to calculate the number of values in each segment tree node.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
const int N = 1 << 18;

vector<int> tree[2 * N];

int query(int a, int b, int c, int d) {
    a += N;
    b += N;

    int count = 0;

    while (a <= b) {
        if (a % 2 == 1) {
            count += upper_bound(tree[a].begin(), tree[a].end(), d) -
                     lower_bound(tree[a].begin(), tree[a].end(), c);
            a++;
        }
```

```
        if (b % 2 == 0) {
            count += upper_bound(tree[b].begin(), tree[b].end(), d) -
                     lower_bound(tree[b].begin(), tree[b].end(), c);
            b--;
        }
        a /= 2;
        b /= 2;
    }

    return count;
}

int main() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        tree[N + i] = {x};
    }

    for (int i = N - 1; i >= 1; i--) {
        merge(tree[2 * i].begin(), tree[2 * i].end(), tree[2 * i + 1].begin(),
              tree[2 * i + 1].end(), back_inserter(tree[i]));
    }

    while (q--) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;

        cout << query(a, b, c, d) << "\n";
    }
}
```

Solution 2 ($O((n + q \log n))$)

We create a list of events that includes one event for each array element and two events for each query. We also maintain a segment tree to calculate sums.

The events are created as follows:

- Event 1: Begin a query with value range $[c, d]$ (processing time c)
- Event 2: Process array value x (processing time x)
- Event 3: End a query with value range $[c, d]$ (processing time d)

After creating the events, we sort them and go through them in increasing order of processing times. If two events have the same processing time, we process them in the order of their event type.

Consider a query where the array range is $[a, b]$ and we want to calculate the number of values between $[c, d]$. We first calculate the number of values between $[1, c-1]$ using the segment tree (event type 1). Then we process all array values between $[c, d]$ and add them to the segment tree (event type 2). Finally we calculate the number of values between $[1, d]$ using the segment tree (event type 3).

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
const int N = 1 << 18;

int tree[2 * N];

void set_val(int pos) {
    pos += N;
    tree[pos] = 1;

    for (pos /= 2; pos >= 1; pos /= 2) {
        tree[pos] = tree[2 * pos] + tree[2 * pos + 1];
    }
}

int get_sum(int a, int b) {
    a += N;
    b += N;

    int sum = 0;
```

```
    while (a <= b) {
        if (a % 2 == 1) {
            sum += tree[a++];
        }
        if (b % 2 == 0) {
            sum += tree[b--];
        }
        a /= 2;
        b /= 2;
    }
    return sum;
}

int main() {
    int n, q;
    cin >> n >> q;

    vector<vector<int>> events;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        events.push_back({x, 2, i});
    }

    for (int i = 1; i <= q; i++) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        events.push_back({c, 1, a, b, i});
        events.push_back({d, 3, a, b, i});
    }

    sort(events.begin(), events.end());

    vector<int> result(q + 1);

    for (auto event : events) {
        if (event[1] == 1) {
            int sum = get_sum(event[2], event[3]);
            result[event[4]] -= sum;
        }

        if (event[1] == 2) {
            set_val(event[2]);
        }

        if (event[1] == 3) {
            int sum = get_sum(event[2], event[3]);
            result[event[4]] += sum;
        }
    }

    for (int i = 1; i <= q; i++) {
        cout << result[i] << "\n";
    }
}
```

3154 · System of Linear Equations

Task · source analysis length 1,580 characters

Problem statement

You are given $n \cdot (m+1)$ coefficients $a_{i,j}$ and b_i which form the following n linear equations:

- $a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,m}x_m = b_1 \pmod{10^9+7}$
- $a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,m}x_m = b_2 \pmod{10^9+7}$
- $\cdots$
- $a_{n,1}x_1 + a_{n,2}x_2 + \cdots + a_{n,m}x_m = b_n \pmod{10^9+7}$

Your task is to find any m integers $x_1, x_2, \ldots, x_m$ that satisfy the given equations.

Input

The first line has two integers n and m: the number of equations and variables.

The next n lines each have $m+1$ integers $a_{i,1}, a_{i,2}, \ldots, a_{i,m}, b_i$: the coefficients of the i-th equation.

Output

Print m integers $x_1, x_2, \ldots, x_m$: the values of the variables that satisfy the equations. The values must also satisfy $0 \le x_i < 10^9+7$. You can print any valid solution. If no solution exists print only -1.

Constraints

- $1 \le n, m \le 500$
- $0 \le a_{i,j}, b_i < 10^9+7$

The following code uses Gaussian elimination to solve the system of linear equations. The algorithm works in $O(n^2 m)$ time.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int M = 1000000007;

ll inverse(ll x) {
    ll power = M - 2;
    ll result = 1;
    while (power > 0) {
        if (power & 1) result = result * x % M;
        x = x * x % M;
        power /= 2;
    }
    return result;
}

vector<vector<ll>> matrix;

void multiply(int row, int m, ll x) {
    for (int i = 0; i <= m; i++) {
        matrix[row][i] = matrix[row][i] * x % M;
    }
}

void subtract(int row1, int row2, int m, ll x) {
    for (int i = 0; i <= m; i++) {
```

```
        matrix[row2][i] -= matrix[row1][i] * x;
        matrix[row2][i] = (matrix[row2][i] % M + M) % M;
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    matrix.resize(n);
    for (int i = 0; i < n; i++) {
        matrix[i].resize(m + 1);
        for (int j = 0; j < m; j++) {
            cin >> matrix[i][j];
        }
        cin >> matrix[i][m];
    }

    vector<int> chosen(n);
    for (int i = 0; i < n; i++) {
        bool found = false;
        for (int j = 0; j < m; j++) {
            if (matrix[i][j]) {
                chosen[i] = j;
                multiply(i, m, inverse(matrix[i][j]));
                found = true;
                break;
            }
        }

        if (!found && matrix[i][m]) {
            cout << "-1\n";
            return 0;
        }

        for (int j = 0; j < n; j++) {
            if (i == j || !matrix[j][chosen[i]]) continue;
            subtract(i, j, m, matrix[j][chosen[i]]);
        }
    }

    for (int i = 0; i < m; i++) {
        bool found = false;
        for (int j = 0; j < n; j++) {
            if (chosen[j] == i) {
                cout << matrix[j][m] << " ";
                found = true;
                break;
            }
        }
        if (!found) cout << 0 << " ";
    }
    cout << "\n";
}
```

References

- Gaussian elimination *Wikipedia*

2428 · Distinct Values Subarrays II

Task · source analysis length 1,230 characters

Problem statement

Given an array of n integers, your task is to calculate the number of subarrays that have at most k distinct values.

Input

The first input line has two integers n and k.

The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output

Print one integer: the number of subarrays.

Constraints

- $1 \le k \le n \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$

We go through the array from left to right and calculate at each position i the number of valid subarrays that end at position i.

We maintain a data structure `rank` that contains pairs of the form (p, x): we have last seen element x at position p. We also use a data structure `prev` that stores the previous position of each element. We make sure that `rank` has at most $k+1$ pairs and if it has more pairs, we remove the pair whose p is smallest.

If `rank` has at most k pairs, then all subarrays that end at position i are valid. Otherwise, `rank` has $k+1$ pairs and the first pair gives us the last position that does not belong to a valid subarray.

We can update both data structures in $O(\log n)$ time, so the algorithm works in $O(n \log n)$ time.

```
#include <iostream>
#include <map>
#include <set>
using namespace std;
using ll = long long;

int main() {
    int n, k;
    cin >> n >> k;

    map<int, int> prev;
    set<pair<int, int>> rank;
    ll count = 0;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;

        rank.erase({prev[x], x});
        rank.insert({i, x});
        prev[x] = i;

        if (rank.size() > k + 1) {
            rank.erase(rank.begin());
        }
```

```
        if (rank.size() <= k) {
            count += i;
        } else {
            count += i - rank.begin()->first;
        }
    }

    cout << count << "\n";
}
```

2420 · Palindrome Queries

Task · source analysis length 2,016 characters

Problem statement

You are given a string that consists of n characters between a–z. The positions of the string are indexed $1, 2, \ldots, n$.

Your task is to process m operations of the following types:

1. Change the character at position k to x
2. Check if the substring from position a to position b is a palindrome

Input

The first input line has two integers n and m: the length of the string and the number of operations.

The next line has a string that consists of n characters.

Finally, there are m lines that describe the operations. Each line is of the form "1 k x" or "2 a b".

Output

For each operation 2, print YES if the substring is a palindrome and NO otherwise.

Constraints

- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le k \le n$
- $1 \le a \le b \le n$

We use a segment tree that maintains a string and supports two operations:

- Set the character at position k to x
- Calculate the hash value of a substring from position l to position r

We create two such segment trees: one for the actual string and one for the reverse string. Then we can efficiently calculate the hash value of a substring and the hash value of its reverse. The substring is a palindrome exactly when the hash values are the same.

The solution works in $O(n + m \log n)$ time.

```
#include <algorithm>
#include <chrono>
#include <iostream>
#include <random>
#include <vector>
using namespace std;
using ll = long long;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
const ll M = 991831889;
const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9 * M)(rng);

struct HashTree {
    struct Hash {
        ll hash = 0;
        ll mul = 1;

        Hash operator+(Hash other) {
            return {(hash * other.mul + other.hash) % M, mul * other.mul % M};
```

```
        }
    };

    int n;
    vector<Hash> tree;

    HashTree(string s) {
        n = 1;
        while (n < s.size()) n *= 2;
        tree.resize(2 * n);
        for (int i = 0; i < s.size(); i++) {
            tree[n + i] = {s[i], C};
        }
        for (int i = n - 1; i >= 1; i--) {
            tree[i] = tree[2 * i] + tree[2 * i + 1];
        }
    }

    void set(int k, char x) {
        k += n;
        tree[k] = {x, C};
        for (k /= 2; k >= 1; k /= 2) {
            tree[k] = tree[2 * k] + tree[2 * k + 1];
        }
    }

    ll hash(int l, int r) {
        l += n;
        r += n;
        Hash left, right;
        while (l <= r) {
            if (l % 2 == 1) left = left + tree[l++];
            if (r % 2 == 0) right = tree[r--] + right;
            l /= 2;
            r /= 2;
        }
        return (left + right).hash;
    }
};

int main() {
    int n, m;
    cin >> n >> m;
    string s;
    cin >> s;

    HashTree inc(s);
    reverse(s.begin(), s.end());
    HashTree dec(s);

    for (int mi = 1; mi <= m; mi++) {
        int t;
        cin >> t;
        if (t == 1) {
            int k;
            char x;
            cin >> k >> x;
            inc.set(k - 1, x);
            dec.set(n - k, x);
        } else {
            int a, b;
            cin >> a >> b;
            auto hash1 = inc.hash(a - 1, b - 1);
            auto hash2 = dec.hash(n - b, n - a);
            cout << (hash1 == hash2 ? "YES" : "NO") << "\n";
        }
    }
}
```

2417 · Counting Coprime Pairs

Task · source analysis length 3,802 characters

Problem statement

Given a list of n positive integers, your task is to count the number of pairs of integers that are coprime $i.e., their greatest common divisor is one$.

Input

The first input line has an integer n: the number of elements.

The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list.

Output

Print one integer: the answer for the task.

Constraints

- $1 \le n \le 10^5$
- $1 \le x_i \le 10^6$

Solution 1

Instead of calculating the number of coprime pairs, we calculate the number of pairs that are *not* coprime, i.e., they have a common divisor.

We go through the numbers and create a vector `factors` for each number. This vector consists of all distinct prime factors of the number. After that we go through all subsets of the vector, which correspond to all distinct numbers that can be created using the factors.

We use the inclusion-exclusion technique to calculate the answer. At position i, we calculate the number of pairs where $j < i$ and $\gcd(x[i], x[j] > 1)$. For this, we create a vector `total` that contains the number of times each factor combination has appeared in previous numbers.

Finally, we subtract the answer from $n(n-1/2)$ which is the number of all pairs that can be created using the numbers.

The solution works in $O(n(\sqrt{N} + 2^K K))$ time where N is the upper bound for input numbers and K is the upper bound for the number of distinct prime factors. Here $N = 10^6$ and $K = 7$ because $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 15 = 450450$.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int N = 1000000;

int main() {
    vector<int> total(N + 1);

    int n;
    cin >> n;
    ll answer = 0;
```

```
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;

        vector<int> factors;
        for (int f = 2; f * f <= x; f++) {
            if (x % f == 0) {
                factors.push_back(f);
                while (x % f == 0) x /= f;
            }
        }
        if (x != 1) factors.push_back(x);

        int k = factors.size();
        for (int b = 1; b < (1 << k); b++) {
            int prod = 1;
            for (int i = 0; i < k; i++) {
                if (b & (1 << i)) prod *= factors[i];
            }

            int add = total[prod];
            if (__builtin_parity(b) == 0) add = -add;
            answer += add;
            total[prod]++;
        }
    }

    cout << (n * ll(n - 1) / 2) - answer << "\n";
}
```

Solution 2

Another way to solve the problem is to use the Möbius inversion technique. We want to calculate the sum

$$\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(x_i,x_j)=1$$

]

where the indicator notation $[...]$ means that the value of the expression is 1 if the condition holds and 0 otherwise. This sum corresponds to

$$\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{d|\gcd(x_i,x_j)}\mu(d)$$

where $\mu(d)$ denotes the Möbius function. Since we want that d divides $\gcd(x_i,x_j)$, this means that d must divide both x_i and x_j and we can use the indicator notation to choose the d values that are counted:

$$\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{d=1}^{N}\mu(d)[d \mid x_i$$

$$d \mid x_j$$

]

Here N is the upper bound for the input values.

After that, we reorder the summation as follows:

$$\sum_{d=1}^{N}(\mu(d)(\sum_{i=1}^{n}[d \mid x_i$$

$$) \sum_{i=1}^{n}[d \mid x_i])]$$

Note that the last two sums are independent of each other so we can use the variable i in both sums and calculate them separately.

We precalculate all values of $\mu(d)$ and $\sum_{i=1}^{n} [d \mid x_i]$ using a sieve of Eratosthenes style technique, after which we can efficiently calculate the answer. Finally, we remove the number of pairs whose elements are at the same position and divide the answer by two to count each pair only once.

The solution works in $O(n + N \log N)$ time.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int N = 1000000;

int main() {
    vector<int> mu(N + 1, 1);
    vector<int> skip(N + 1);
    for (int d = 2; d <= N; d++) {
        if (skip[d]) continue;
        for (int i = d; i <= N; i += d) {
            skip[i] = 1;
            if (i % (d * d) == 0) mu[i] = 0;
            mu[i] = -mu[i];
        }
    }

    int n;
    cin >> n;
    vector<int> count(N + 1);
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        count[x]++;
    }

    vector<int> div(N + 1);
    for (int d = 1; d <= N; d++) {
        for (int i = d; i <= N; i += d) {
            div[d] += count[i];
        }
    }

    ll answer = 0;
    for (int d = 1; d <= N; d++) {
        answer += mu[d] * (ll)div[d] * div[d];
    }
    cout << (answer - count[1]) / 2 << "\n";
}
```

2416 · Increasing Array Queries

Task · source analysis length 7,678 characters

Problem statement

You are given an array that consists of n integers. The array elements are indexed $1, 2, \ldots, n$.

You can modify the array using the following operation: choose an array element and increase its value by one.

Your task is to process q queries of the form: when we consider a subarray from position a to position b, what is the minimum number of operations after which the subarray is increasing?

An array is increasing if each element is greater than or equal with the previous element.

Input

The first input line has two integers n and q: the size of the array and the number of queries.

The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Finally, there are q lines that describe the queries. Each line has two integers a and b: the starting and ending position of a subarray.

Output

For each query, print the minimum number of operations.

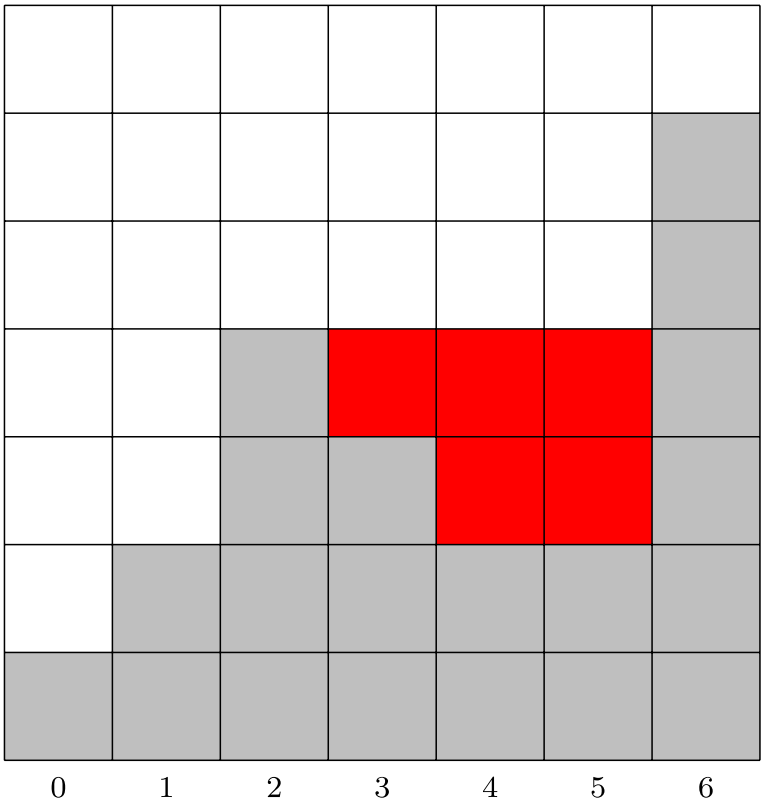
Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le a \le b \le n$

There are many approaches to this problem. Let us first analyze the situation before discussing any specific approach.

One important question is "When do we need to increase an element?" Let an element of the array be *special* if it is larger than the value following it. Only *special* elements can cause other elements to increase. Each smaller value after a *special* element will be increased to match the *special* element's size.

The picture following illustrates the idea – the heights of the towers represents the sizes of the array elements and the red squares are added using the operation.



First approach: Lazy segment tree

This is an offline approach as we answer the queries in the order of their starting index. We will go through the array in reverse order and keep track of the amount of increase each element needs. This way we can answer a query with a sum query in a lazy segment tree.

To each position in the tree, we will store the amount the element in this position will need to be increased, in other words, the increased value subtracted by the initial value.

To figure out the amount of increase each element needs we will use a stack of increasing values. For each element, we will remove all values smaller than it from the stack. Note that all of the values in the stack are *special* elements. The topmost element in the stack will be the next larger value and all of the values before it will need to be increased which can be done by adding the current value to the corresponding range in the tree.

We will use segment tree operations either when we remove an element from the stack or when we answer a query. Each index can be removed from the stack at most once – in other words, the number of removed items is $O(n)$. Further, each segment tree operation will take $O(\log n)$ time. Thus, the total time complexity of the solution is $O((n + q$\log n)).

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

// Lazy propagation segment tree.
// Supports the following operations:
//  1. `add`: add x to every element in range [l, r)
//  2. `query`: find the sum of values in range [l, r)
struct Tree {
    int n;
    vector<ll> tree;
    vector<ll> lazy;

    Tree(int n) : n(n), tree(n * 4), lazy(n * 4) {}

    void add(int s, int l, int r, int ql, int qr, ll x) {
        if (r <= ql || qr <= l) return;
        if (ql <= l && r <= qr) return apply(s, r - l, x);
        push(s, r - l);
```

```
        int m = (l + r) / 2;
        add(s * 2 + 0, l, m, ql, qr, x);
        add(s * 2 + 1, m, r, ql, qr, x);
        pull(s);
    }

    ll query(int s, int l, int r, int ql, int qr) {
        if (r <= ql || qr <= l) return 0;
        if (ql <= l && r <= qr) return tree[s];
        push(s, r - l);
        int m = (l + r) / 2;
        return query(s * 2, l, m, ql, qr) + query(s * 2 + 1, m, r, ql, qr);
    }

    void push(int s, int length) {
        if (lazy[s]) {
            apply(s * 2 + 0, length / 2, lazy[s]);
            apply(s * 2 + 1, (length + 1) / 2, lazy[s]);
            lazy[s] = 0;
        }
    }

    void apply(int s, int length, ll x) {
        tree[s] += x * length;
        lazy[s] += x;
    }

    void pull(int s) { tree[s] = tree[s * 2] + tree[s * 2 + 1]; }
    void add(int l, int r, ll x) { add(1, 0, n, l, r, x); }
    ll query(int l, int r) { return query(1, 0, n, l, r); }
};

int main() {
    int n, q;
    cin >> n >> q;

    Tree tree(n);

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
        tree.add(i, i + 1, -v[i]);
    }

    vector<vector<pair<int, int>>> queries(n);
    vector<ll> ans(q);

    for (int i = 0; i < q; ++i) {
        int l, r;
        cin >> l >> r;
        l--;
        queries[l].emplace_back(r, i);
    }

    vector<int> stack{n};

    for (int i = n - 1; i >= 0; --i) {
        while (stack.back() < n && v[stack.back()] <= v[i]) {
            int j = stack.back();
            stack.pop_back();
            tree.add(j, stack.back(), -v[j]);
        }
        tree.add(i, stack.back(), v[i]);
        stack.push_back(i);
        for (auto [r, j] : queries[i]) {
            ans[j] = tree.query(i, r);
        }
    }

    for (ll u : ans) {
        cout << u << '\n';
    }
}
```

Second approach: Regular segment tree

The above solution can be modified to work with a regular segment tree.

When working with a regular segment tree, we will store the increase amounts to the positions of the corresponding *special elements*. Note that the effective range of the last *special* element in the query range can extend beyond the query's right end. In this case, we have to subtract the surplus increase.

This approach has the same time complexity as the previous one.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int TREE_SIZE = 1 << 18;
ll tree[TREE_SIZE * 2];

// Adds x to position i
void add(int i, ll x) {
    i += TREE_SIZE;
    while (i > 0) {
        tree[i] += x;
        i /= 2;
    }
}

// Returns the sum of values in range [l, r)
ll query(int l, int r) {
    ll res = 0;
    l += TREE_SIZE;
    r += TREE_SIZE;
    while (l < r) {
        if (l % 2 == 1) res += tree[l++];
        if (r % 2 == 1) res += tree[--r];
        l /= 2;
        r /= 2;
    }
    return res;
}

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
        add(i, -v[i]);
    }

    vector<vector<pair<int, int>>> queries(n);
    vector<ll> ans(q);

    for (int i = 0; i < q; ++i) {
        int l, r;
        cin >> l >> r;
        l--;
        queries[l].emplace_back(r, i);
    }

    vector<pair<int, int>> stack{{n, 0}};

    for (int i = n - 1; i >= 0; --i) {
        while (stack.back().first < n && v[stack.back().first] <= v[i]) {
            auto [j, len] = stack.back();
            stack.pop_back();
            add(j, -ll(len) * v[j]);
        }
        add(i, ll(stack.back().first - i) * v[i]);
        stack.emplace_back(i, stack.back().first - i);
        for (auto [r, j] : queries[i]) {
            auto iter = lower_bound(stack.rbegin(), stack.rend(), pair{r, 0});
            auto [it, len] = *prev(iter);
            ans[j] = query(i, r) - ll(len - (r - it)) * v[it];
        }
    }

    for (ll u : ans) {
        cout << u << '\n';
    }
}
```

Third approach: Sparse table

There is a simple solution that uses a sparse table. We will, for each element, store a pointer to the next larger element. Later, a sparse table will be built for repeatedly following the pointers efficiently.

The amount of increase needed to make the range from the position i to the position j of the next larger element non-decreasing is calculated with the formula $(j - i$\cdot x_i-\sum_{i\le k < j}a_k). The formula represents the difference between the increased values and the initial values.

Note that, also in this solution, the final jump might exceed the query range. In this case, we know that each remaining element in the range will have the same value – as they are smaller than our current value. Thus we can calculate the amount of increase needed for these elements separately.

Building the sparse table takes $O(n \log n)$ time and answering a single query takes $O(\log n)$ time. Thus, the total time complexity is $O((n + q$\log n)).

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int INF = 1e9;

const int N = 200001;
const int L = 19;
int jump[L][N];
ll cost[L][N];

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> a(n + 1);
    vector<ll> prefix_sum(n + 1);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        prefix_sum[i + 1] = prefix_sum[i] + a[i];
    }

    a[n] = INF;
    jump[0][n] = n;

    vector<int> stack{n};

    for (int i = n - 1; i >= 0; --i) {
        while (a[stack.back()] <= a[i]) {
            stack.pop_back();
        }

        int j = stack.back();
        jump[0][i] = j;
        cost[0][i] = ll(j - i) * a[i];
        stack.push_back(i);
    }

    for (int j = 0; j < L - 1; ++j) {
        for (int i = 0; i <= n; ++i) {
            cost[j + 1][i] = cost[j][i] + cost[j][jump[j][i]];
            jump[j + 1][i] = jump[j][jump[j][i]];
        }
    }

    for (int qi = 0; qi < q; ++qi) {
        int l, r;
        cin >> l >> r;
        l--;
        ll ans = -(prefix_sum[r] - prefix_sum[l]);

        for (int j = L - 1; j >= 0; --j) {
            if (jump[j][l] < r) {
                ans += cost[j][l];
                l = jump[j][l];
            }
        }

        ans += ll(r - l) * a[l];

        cout << ans << '\n';
    }
}
```

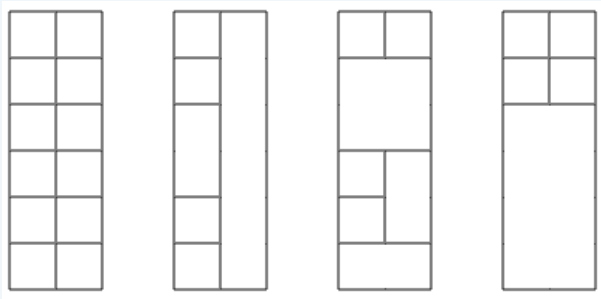
2413 · Counting Towers

Task · source analysis length 4,540 characters

Problem statement

Your task is to build a tower whose width is 2 and height is n. You have an unlimited supply of blocks whose width and height are integers.

For example, here are some possible solutions for $n = 6$:



Given n, how many different towers can you build? Mirrored and rotated towers are counted separately if they look different.

Input

The first input line contains an integer t: the number of tests.

After this, there are t lines, and each line contains an integer n: the height of the tower.

Output

For each test, print the number of towers modulo $10^9 + 7$.

Constraints

- $1 \le t \le 100$
- $1 \le n \le 10^6$

Solution 1

To solve this problem using dynamic programming, the choice of *state space* is not obvious.

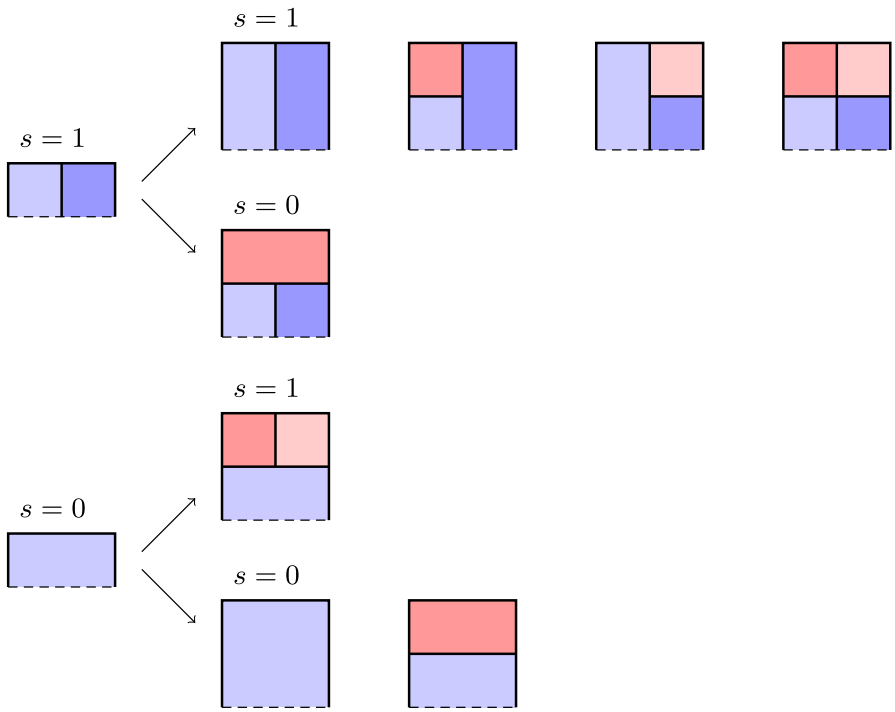
For example, using only the height of the tower n as state does not seem sufficient. A solution could try to split the tower into two pieces with heights $a + b = n$, but towers which cannot be split also exhibit complex patterns and are hard to count. This suggests that some more information about the tower needs to be encoded in the state.

One suitable state space is determined by the height of the tower, and whether or not the topmost two squares are disjoint. In other words, whether there is a "line" drawn between the squares.

Let the function $\mathbf{towers}(i, s)$ describe the amount of towers with height i, where $s \in \{1, 0\}$ indicates whether the topmost squares are disjoint, as 1, or not, as 0.

Define towers with height one as base cases: $\mathbf{towers}(1, 1) = 1$ (two 1×1 blocks) and $\mathbf{towers}(1, 0) = 1$ (one 2×1 block).

Now let us explore the *transitions* between states with height $i - 1$ and i. How many ways are there to add one layer to a tower in each case, by starting new blocks or extending existing ones?



The recurrence corresponding to this tabulation is as follows:

$$\mathbf{towers}(i, 1) = 4 \cdot \mathbf{towers}(i - 1, 1) + \mathbf{towers}(i - 1, 0)$$
$$\mathbf{towers}(i, 0) = \mathbf{towers}(i - 1, 1) + 2 \cdot \mathbf{towers}(i - 1, 0)$$

The implementation below precomputes these values for all $i \leq 10^6$, so each test can be answered efficiently.

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int N = 1000001;

ll towers[N][2];

int main() {
    towers[1][1] = 1;
    towers[1][0] = 1;

    for (int i = 2; i < N; ++i) {
        towers[i][1] = (4 * towers[i - 1][1] + towers[i - 1][0]) % M;
        towers[i][0] = (towers[i - 1][1] + 2 * towers[i - 1][0]) % M;
    }

    int t;
    cin >> t;
    for (int ti = 0; ti < t; ++ti) {
        int n;
        cin >> n;
        cout << (towers[n][0] + towers[n][1]) % M << "\n";
    }
}
```

Solution 2

Despite it seeming challenging, there is an explicit formula for counting towers which cannot be split further. Such a tower consists of either one $2 \times n$ block or two columns of $1 \times x$ blocks, where block boundaries do not coincide at any height.

In the latter case, there are three possible configurations at each intermediary height: neither of the columns have a block boundary, only the left one has a block boundary, or only the right one has a block boundary. If both columns had a boundary, the tower could be split at that height.

Let the height of this indivisible tower be a. The total amount of combinations for constructing it is

$$1 + 3^{a-1}.$$

Now we may formulate a recurrence with only one variable. The topmost indivisible part can have any height $1 \le a \le i$.

$$\textbf{towers}(i) = \sum_{a=1}^{i} (1 + 3^{a-1}) \textbf{towers}(i-a)$$

We may set $\textbf{towers}(0$ = 1) to fit the pattern: if $a = i$, then the part left over has height zero, and this case is valid.

Using this recurrence directly would yield a time complexity of $O(N^2)$, where $N = 10^6$ is the largest i needed. However, the equation can be simplified.

For brevity, $f(i)$ shall denote the same as $\textbf{towers}(i)$.

$$f(i) = \sum_{a=1}^{i} (1 + 3^{a-1}) f(i-a) \tag{1}$$

Observe that the first term is $(1 + 3^0$ f$i - 1$ = 2f$i - 1$). It can be moved to the left side of the equation.

$$f(i) - 2f(i-1) = \sum_{a=2}^{i} (1 + 3^{a-1}) f(i-a)$$

The powers of three may be canceled by subtracting $3f(i-1)$ on both sides, substituting $i-1$ for i in equation 1.

$$\begin{aligned} f(i) - 5f(i-1) &= \sum_{a=2}^{i} (1 + 3^{a-1} - 3(1 + 3^{a-2})) f(i-a) \\ &= \sum_{a=2}^{i} (1 + 3^{a-1} - 3 - 3 \cdot 3^{a-2}) f(i-a) \\ &= \sum_{a=2}^{i} (1 + 3^{a-1} - 3 - 3^{a-1}) f(i-a) \\ &= -2 \sum_{a=2}^{i} f(i-a) \end{aligned} \tag{2}$$

Finally, subtract $f(i-1$-5f$i-2)$ on both sides according to equation 2.

$$\begin{aligned} f(i) - 6f(i-1) + 5f(i-2) &= -2 \sum_{a=2}^{i} f(i-a) + 2 \sum_{a=2}^{i-1} f(i-1-a) \\ &= -2 \sum_{a=2}^{i} f(i-a) + 2 \sum_{a=3}^{i} f(i-a) \\ &= -2f(i-2) \end{aligned}$$

Rearrange to find

$$f(i) = 6f(i-1) - 7f(i-2).$$

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int N = 1000001;

ll towers[N];

int main() {
    towers[0] = 1;
    towers[1] = 2;

    for (int i = 2; i < N; ++i) {
        towers[i] = (6 * towers[i - 1] + (M - 7) * towers[i - 2]) % M;
    }

    int t;
    cin >> t;
    for (int ti = 0; ti < t; ++ti) {
        int n;
        cin >> n;
        cout << towers[n] << "\n";
    }
}
```

References

- The On-Line Encyclopedia of Integer Sequences, A034999

2229 · Permutation Inversions

Task · source analysis length 1,992 characters

Problem statement

Your task is to count the number of permutations of $1, 2, \ldots, n$ that have exactly k inversions $i.e., pairs of elements in the wrong order$.

For example, when $n = 4$ and $k = 3$, there are 6 such permutations:

- $[1, 4, 3, 2]$
- $[2, 3, 4, 1]$
- $[2, 4, 1, 3]$
- $[3, 1, 4, 2]$
- $[3, 2, 1, 4]$
- $[4, 1, 2, 3]$

Input

The only input line has two integers n and k.

Output

Print the answer modulo $10^9 + 7$.

Constraints

- $1 \le n \le 500$
- $0 \le k \le \frac{n(n-1)}{2}$

Solution 1: $O(n^4)$ approach

We will first present a $O(n^4)$ approach. The approach is a bit too slow, but it can be easily sped up. Let us use dynamic programming to compute the answer. Let $\mathrm{dp}[i][j]$ represent the number of permutations of the numbers $1 \ldots i$ with exactly j inversions.

Now, when we are placing the number $i + 1$, we have $i + 1$ places for it. If we place the number at the end of the permutation, no new inversion will be formed. Each place to the left of this will add a new inversion as all numbers are smaller than the new one.

These observations lead to a solution where we loop through the numbers, the place for each number and the number of previous inversions. This leads to the recurrence

$$\mathrm{dp}[i + 1]$$

$$j$$

= \sum_{0 \le \mathrm{pos} \le i} \mathrm{dp}

$$i$$

$$j - \mathrm{pos}$$

.]

The table may be filled with this recurrence in $O(n^4)$ time.

```
#include <iostream>
using namespace std;

const int M = 1e9 + 7;
const int N = 501;
const int K = 124751;
int dp[N][K];

int main() {
    int n, k;
    cin >> n >> k;

    dp[0][0] = 1;
    for (int i = 1; i <= n; ++i) {
        for (int h = 0; h <= k; ++h) {
            for (int j = 0; j <= min(h, i - 1); ++j) {
                dp[i][h] += dp[i - 1][h - j];
                dp[i][h] %= M;
            }
        }
    }

    cout << dp[n][k] << '\n';
}
```

Solution 2: $O(n^3)$ approach

Note that the value of each dp position corresponds to a sum of a range of values from the previous layer. Further, the ranges are all the same length and their starting positions differ by one. We can, thus, maintain the sum while filling the dp layer. This leads to a solution with a time complexity of $O(n^3)$.

```
#include <iostream>
using namespace std;

const int M = 1e9 + 7;
const int N = 501;
const int K = 124751;
int dp[N][K];

int main() {
    int n, k;
    cin >> n >> k;

    dp[0][0] = 1;
    for (int i = 1; i <= n; ++i) {
        int sum = 0;
        for (int h = 0; h <= k; ++h) {
            sum += dp[i - 1][h];
            sum %= M;
            if (h >= i) {
                sum -= dp[i - 1][h - i];
                sum += M;
                sum %= M;
            }
            dp[i][h] = sum;
        }
    }

    cout << dp[n][k] << '\n';
}
```

2220 · Counting Numbers

Task · source analysis length 2,540 characters

Problem statement

Your task is to count the number of integers between a and b where no two adjacent digits are the same.

Input

The only input line has two integers a and b.

Output

Print one integer: the answer to the problem.

Constraints

- $0 \le a \le b \le 10^{18}$

Let us first simplify the problem to only one variable. Define a function $\mathbf{count}(x)$ to equal the amount of integers on the interval $[0, x]$ that have no two adjacent equal digits. The answer can then be found by computing $\mathbf{count}(b$ - \mathrm{count}$a - 1)$.

We will compute the function by recursively choosing one digit of the integer at a time. The parameters of the recursive function, or the context needed to compute the total amount of integers that can still be chosen, are:

- i: The position of the next digit, starting from 0 at the most significant digit in x.
- $\mathbf{prev}$: The previous digit chosen $if applicable$.
- $\mathbf{equal}$: Whether all previously chosen digits are equal to the respective digits in x. This should be known so that no integers greater than x are counted. After the first nonequal digit, there is no limit to the rest of the digits that can be chosen.
- $\mathbf{empty}$: Whether there has been no nonzero digit chosen yet, meaning that the prefix is empty. Any amount of zeros can be adjacent to each other before the proper start of the integer.

We then try each option for the next digit, $0, \ldots, 9$, check that the total integer does not exceed x, and ensure that the digit is not the same as the previous one. The parameters are updated appropriately and we can then recurse to the next digit and add the result to the total.

The case $i = n$, where n is the amount of digits in x, means that we have successfully set all digits and found one valid integer.

The results of the function are memoized in a lookup table. There are at most $(n + 1$ \cdot 10 \cdot 2 \cdot 2) different combinations of parameters to the function, so the time complexity when using an `std::map` for the lookup table is $O(\log x \log \log x)$.

```
#include <array>
#include <iostream>
#include <map>
using namespace std;
using ll = long long;

string s;
map<array<int, 4>, ll> lookup;

ll recurse(int i, int prev, bool equal, bool empty) {
```

```
    if (i == (int)s.size()) return 1;

    array<int, 4> params{i, prev, equal, empty};
    if (lookup.count(params)) return lookup[params];

    ll result = 0;
    for (int d = 0; d < 10; ++d) {
        if (equal && d > s[i] - '0') break;
        if (!empty && d == prev) continue;
        result += recurse(i + 1, d, equal && d == s[i] - '0', empty && d == 0);
    }

    lookup[params] = result;
    return result;
}

ll count(ll x) {
    if (x < 0) return 0;
    s = to_string(x);
    lookup.clear();
    return recurse(0, 0, true, true);
}

int main() {
    ll a, b;
    cin >> a >> b;
    cout << count(b) - count(a - 1) << endl;
}
```

2217 · Collecting Numbers II

Task · source analysis length 1,316 characters

Problem statement

You are given an array that contains each number between $1 \ldots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order.

On each round, you go through the array from left to right and collect as many numbers as possible.

Given m operations that swap two numbers in the array, your task is to report the number of rounds after each operation.

Input

The first line has two integers n and m: the array size and the number of operations.

The next line has n integers $x_1, x_2, \ldots, x_n$: the numbers in the array.

Finally, there are m lines that describe the operations. Each line has two integers a and b: the numbers at positions a and b are swapped.

Output

Print m integers: the number of rounds after each swap.

Constraints

- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

In this problem we have to update the number of rounds after each swap. Fortunately, when the position of number x changes, it suffices to check the positions of numbers $x-1$ and $x+1$.

In the following code, before the swap, we first remove the new rounds caused by the numbers that will be swapped. Then, after the swap, we add the new rounds caused by the numbers that were swapped.

There is one special case when consecutive numbers are swapped, and some if statements are needed to make sure that the code works also in this case.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<int> x(n + 1);
    vector<int> pos(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> x[i];
        pos[x[i]] = i;
    }

    int rounds = 1;
    for (int i = 2; i <= n; i++) {
        if (pos[i] < pos[i - 1]) rounds++;
    }

    auto change = [&](int k, int c) {
```

```
        if (k <= n && pos[k - 1] > pos[k]) rounds += c;
    };

    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        change(x[a], -1);
        if (x[a] + 1 != x[b]) change(x[a] + 1, -1);
        change(x[b], -1);
        if (x[b] + 1 != x[a]) change(x[b] + 1, -1);
        swap(x[a], x[b]);
        pos[x[a]] = a;
        pos[x[b]] = b;
        change(x[a], 1);
        if (x[a] + 1 != x[b]) change(x[a] + 1, 1);
        change(x[b], 1);
        if (x[b] + 1 != x[a]) change(x[b] + 1, 1);
        cout << rounds << "\n";
    }
}
```

2214 · Inverse Inversions

Task · source analysis length 1,877 characters

Problem statement

Your task is to create a permutation of numbers $1, 2, \ldots, n$ that has exactly k inversions.

An inversion is a pair (a, b) where $a < b$ and $p_a > p_b$ where p_i denotes the number at position i in the permutation.

Input

The only input line has two integers n and k.

Output

Print a line that contains the permutation. You can print any valid solution.

Constraints

- $1 \le n \le 10^6$
- $0 \le k \le \frac{n(n-1)}{2}$

There is a straightforward greedy approach to this problem. However, it is not immediately clear that it works.

Let us construct the answer from the beginning, one element at a time. We will maintain two pointers: the lowest and highest numbers that have not been used, and at each point, append either of them to the answer and update the corresponding pointer.

Adding the lowest number yet will not form new inversions with any number following it as all of them will be larger. Adding the highest number yet, on the other hand, forms inversions with all of the numbers following it.

The greedy idea is to always pick the larger number while the inversion count remains under k. Now, we have to prove that the algorithm always finds a solution with exactly k inversions. In total, we have $n - 1$ choices: during the first step, we are offered $n - 1$ inversions, then $n - 2$ and so on until we are offered to take 1 inversion during the second-to-last step. The sum of the amounts is $1 + 2 + \cdots + (n - 1$=\frac{n$n - 1$}2), which means there are always enough inversions to form k.

It can be proven that because we take the inversions greedily, the amount of inversions needed is at all times at most $\frac{n(n-1)}{2}$, where n represents the number of elements remaining. It follows that when $n = 1$, the number of inversions needed is at most 0, thus equal to 0. The algorithm always finds a solution whose inversion count is never larger k nor is it lower than k.

```
#include <iostream>
using namespace std;
using ll = long long;

int main() {
    int n;
    ll k;
    cin >> n >> k;

    int l = 1, r = n;
    for (int i = 1; i <= n; ++i) {
        int left = n - i;  // The amount of numbers left.
        // As r is larger than all remaining numbers,
```

```
        // adding it will form inversions with each one of them.
        if (left <= k) {
            k -= left;
            cout << r-- << ' ';
        } else {
            cout << l++ << ' ';
        }
    }
    cout << '\n';
}
```

2210 · Counting Grids

Task · source analysis length 1,710 characters

Problem statement

Your task is to count the number of different $n \times n$ grids whose each square is black or white.

Two grids are considered to be different if it is not possible to rotate one of them so that they look the same.

Input

The only input line has an integer n: the size of the grid.

Output

Print one integer: the number of grids modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10^9$

We solve the problem using Burnside's lemma which states that the number of distinct combinations is

$$\frac{1}{n} \sum_{i=1}^{n} c(i)$$

when there are n ways to change the position of a combination and there are $c(i)$ combinations that remain the same when the ith way is applied.

In this problem there are 4 ways to rotate the grid: $0°$, $90°$, $180°$, and $270°$. Our task is to count the combinations that remain the same after each rotation. In each case we count the number of squares whose color we can choose.

When n is even, the numbers are as follows:

- $0°$: n^2
- $90°$: $\frac{n^2}{4}$
- $180°$: $\frac{n^2}{2}$
- $270°$: $\frac{n^2}{4}$

When n is odd, the numbers are as follows:

- $0°$: n^2
- $90°$: $\lfloor \frac{n}{2} \rfloor \lfloor \frac{n}{2} + 1 \rfloor + 1$
- $180°$: $n \lfloor \frac{n}{2} \rfloor + \lfloor \frac{n}{2} + 1 \rfloor$
- $270°$: $\lfloor \frac{n}{2} \rfloor \lfloor \frac{n}{2} + 1 \rfloor + 1$

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;

int power(int a, ll b) {
    if (b == 0) return 1;
```

```
    ll p = power(a, b / 2);
    p = p * p % M;
    if (b % 2 == 1) p = p * a % M;
    return p;
}

int inv(int x) {
    return power(x, M - 2);
}

int main() {
    ll n;
    cin >> n;

    ll sum = 0;
    if (n % 2 == 0) {
        sum += power(2, n * n);
        sum += power(2, n * n / 4);
        sum += power(2, n * n / 2);
        sum += power(2, n * n / 4);
    } else {
        sum += power(2, n * n);
        sum += power(2, (n / 2) * (n / 2 + 1) + 1);
        sum += power(2, n * (n / 2) + (n / 2 + 1));
        sum += power(2, (n / 2) * (n / 2 + 1) + 1);
    }
    ll answer = sum * inv(4) % M;
    cout << answer << "\n";
}
```

References

- Burnside's lemma *Wikipedia*

2209 · Counting Necklaces

Task · source analysis length 1,138 characters

Problem statement

Your task is to count the number of different necklaces that consist of n pearls and each pearl has m possible colors.

Two necklaces are considered to be different if it is not possible to rotate one of them so that they look the same.

Input

The only input line has two numbers n and m: the number of pearls and colors.

Output

Print one integer: the number of different necklaces modulo $10^9 + 7$.

Constraints

- $1 \le n, m \le 10^6$

We solve the problem using Burnside's lemma which states that the number of distinct combinations is

$$\frac{1}{n} \sum_{i=1}^{n} c(i)$$

when there are n ways to change the position of a combination and there are $c(i)$ combinations that remain the same when the ith way is applied.

In this problem there are n ways to rotate the necklace: we can rotate it $0, 1, \ldots, n-1$ steps. The number of combinations that remain the same is

$$m^{\gcd(i,n)}$$

because there is a repeating block of $\gcd(i, n)$ pearls and there are m ways to choose the color of each pearl inside the block.

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int power(int a, int b) {
    if (b == 0) return 1;
    ll p = power(a, b / 2);
    p = p * p % M;
    if (b % 2 == 1) p = p * a % M;
    return p;
}
```

```
int inv(int x) {
    return power(x, M - 2);
}

int main() {
    int n, m;
    cin >> n >> m;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += power(m, gcd(i, n));
        sum %= M;
    }
    ll answer = sum * inv(n) % M;
    cout << answer << "\n";
}
```

References

- Burnside's lemma *Wikipedia*

2207 · Grundy's Game

Task · source analysis length 2,261 characters

Problem statement

There is a heap of n coins and two players who move alternately. On each move, a player chooses a heap and divides into two nonempty heaps that have a different number of coins. The player who makes the last move wins the game.

Your task is to find out who wins if both players play optimally.

Input

The first input line contains an integer t: the number of tests.

After this, there are t lines that describe the tests. Each line has an integer n: the number of coins in the initial heap.

Output

For each test case, print "first" if the first player wins the game and "second" if the second player wins the game.

Constraints

- $1 \le t \le 10^5$
- $1 \le n \le 10^6$

Solution 1

Let $\mathbf{grundy}[x]$ denote the Grundy number of a heap that has x coins, and let $\oplus$ denote the xor operator.

When we divide a heap of x coins into heaps of a and b coins, there will be two subgames whose Grundy numbers are $\mathbf{grundy}[a]$ and $\mathbf{grundy}[b]$, so the Grundy number after the move is $\mathbf{grundy}[a] \oplus \mathbf{grundy}[b]$.

To calculate $\mathbf{grundy}[x]$, we go through all possible ways to divide a heap of x coins into two heaps, calculate the Grundy number of each move and then find their mex value, i.e., the smallest nonnegative integer that doesn't appear among the numbers.

The solution works but it is too slow because it is quadratic and N can be up to 10^6. However, we can study the problem using the solution.

```
#include <iostream>
#include <set>
using namespace std;
const int N = 1000000;
int grundy[N + 1];

int main() {
    for (int i = 1; i <= N; i++) {
        set<int> s;
        for (int j = 1; i - j > j; j++) {
            s.insert(grundy[j] ^ grundy[i - j]);
        }
        grundy[i] = 0;
        while (s.count(grundy[i])) {
            grundy[i]++;
```

```
        }
    }

    int t;
    cin >> t;
    for (int ti = 1; ti <= t; ti++) {
        int n;
        cin >> n;
        cout << (grundy[n] ? "first" : "second") << "\n";
    }
}
```

Solution 2

By studying the first solution, we can notice that 1222 is the last state whose Grundy number is zero, i.e., we are in a losing position. Thus, it suffices to calculate the Grundy numbers of states $0 \ldots 1222$ because we always win if we have more than 1222 coins.

Note that our search only proves that 1222 is the last losing position among states $0 \ldots 10^6$. However, it is known that this also holds for states $0 \ldots 2^{35}$. Nobody knows if there is a losing position after that.

```
#include <iostream>
#include <set>
using namespace std;
const int N = 1222;
int grundy[N + 1];

int main() {
    for (int i = 1; i <= N; i++) {
        set<int> s;
        for (int j = 1; i - j > j; j++) {
            s.insert(grundy[j] ^ grundy[i - j]);
        }
        grundy[i] = 0;
        while (s.count(grundy[i])) {
            grundy[i]++;
        }
    }

    int t;
    cin >> t;
    for (int ti = 1; ti <= t; ti++) {
        int n;
        cin >> n;
        bool win = n > N || grundy[n];
        cout << (win ? "first" : "second") << "\n";
    }
}
```

References

- Grundy's game *Wikipedia*
- A. Flammenkamp: Sprague-Grundy values of Grundy's game

2206 · Pizzeria Queries

Task · source analysis length 2,392 characters

Problem statement

There are n buildings on a street, numbered $1, 2, \ldots, n$. Each building has a pizzeria and an apartment.

The pizza price in building k is p_k. If you order a pizza from building a to building b, its price $withdelivery$ is $p_a + |a - b|$.

Your task is to process two types of queries:

1. The pizza price p_k in building k becomes x.
2. You are in building k and want to order a pizza. What is the minimum price?

Input

The first input line has two integers n and q: the number of buildings and queries.

The second line has n integers $p_1, p_2, \ldots, p_n$: the initial pizza price in each building.

Finally, there are q lines that describe the queries. Each line is either "1 k x" or "2 k".

Output

Print the answer for each query of type 2.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le p_i, x \le 10^9$
- $1 \le k \le n$

This one is a bit trickier to solve with regular segment trees.

Let us examine the price offers in the cases of the first few indices.

The offers for the first position will look like this

$$p_0 + 0,\ p_1 + 1,\ p_2 + 2,\ p_3 + 3 \ldots$$

and the offers for the second one like this

$$p_0 + 1,\ p_1 + 0,\ p_2 + 1,\ p_3 + 2 \ldots$$

As can be seen from the examples, the price offers for the second building look almost exactly the same as the prices for the first building. In fact, from the second building on, all of the prices differ by only one unit.

Motivated by these observations, an idea of a segment tree where the index of each element is added to its value comes to mind.

These segments trees would need to be constructed for both directions separately. It turns out this works.

In the attached solutions, two segment trees are used – **`left\_tree`** and **`right\_tree`**.

To **`left\_tree`** we will store, to each position i, the value $p_i - i$ and to **`right\_tree`** we will store, to each position i, the value $p_i + i$. Then, we can answer queries by taking the minimum values separately, shifting them by the current index and returning the smaller of them.

```
#include <iostream>
#include <vector>
using namespace std;
const int INF = (int)1e9;

struct Tree {
    int tree_size;
    vector<int> t;
    Tree(int n) {
        tree_size = 1;
        while (tree_size < n) {
            tree_size *= 2;
        }
        t.resize(tree_size * 2);
    }
    // Changes the value at position k to x.
    void change(int k, int x) {
        k += tree_size;
        t[k] = x;
        while (k > 1) {
            k /= 2;
            t[k] = min(t[k * 2], t[k * 2 + 1]);
        }
    }
    // Returns the minimum value in range [l, r]
    int query(int l, int r) {
        int ans = INF;
        l += tree_size;
        r += tree_size;
        while (l <= r) {
            if (l % 2 == 1) ans = min(ans, t[l++]);
            if (r % 2 == 0) ans = min(ans, t[r--]);
            l /= 2;
            r /= 2;
        }
        return ans;
    }
};

int main() {
    int n, q;
    cin >> n >> q;

    Tree left_tree(n), right_tree(n);

    vector<int> p(n);

    auto update = [&](int i) {
        left_tree.change(i, p[i] - i);
        right_tree.change(i, p[i] + i);
    };

    for (int i = 0; i < n; ++i) {
        cin >> p[i];
        update(i);
    }

    for (int qi = 0; qi < q; ++qi) {
        int t;
        cin >> t;
        if (t == 1) {
            int i;
            cin >> i;
            i--;
            cin >> p[i];
            update(i);
        } else {
            int i;
            cin >> i;
            i--;
            int left_ans = left_tree.query(0, i) + i;
            int right_ans = right_tree.query(i, n - 1) - i;
            cout << min(left_ans, right_ans) << '\n';
        }
    }
}
```

2205 · Gray Code

Task · source analysis length 1,022 characters

Problem statement

A Gray code is a list of all 2^n bit strings of length n, where any two successive strings differ in exactly one bit $i.e., their Hamming distance is one$.

Your task is to create a Gray code for a given length n.

Input

The only input line has an integer n.

Output

Print 2^n lines that describe the Gray code. You can print any valid solution.

Constraints

- $1 \le n \le 16$

One way to create a Gray code of length n is to use recursion. Suppose that we already have a list of strings which is a Gray code of length $n-1$. Using that list, we can create a Gray code of length n as follows:

- Go through the list and, for each string s, add a string $0s$ to the code.
- Reverse the list.
- Go through the list and, for each string s, add a string $1s$ to the code.

For example, given a Gray code $[00, 01, 11, 10]$ of length 2, we can create a Gray code $[000, 001, 011, 010, 110, 111, 101, 100]$ of length 3.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

vector<string> create(int n) {
    if (n == 1) {
        return {"0", "1"};
    }
    vector<string> code;
    auto prev = create(n - 1);
    for (auto s : prev) {
        code.push_back("0" + s);
    }
    reverse(prev.begin(), prev.end());
    for (auto s : prev) {
        code.push_back("1" + s);
    }
    return code;
}

int main() {
    int n;
    cin >> n;
    auto code = create(n);
    for (auto s : code) {
        cout << s << "\n";
    }
}
```

References

- Gray code *Wikipedia*

2187 · Bracket Sequences II

Task · source analysis length 1,915 characters

Problem statement

Your task is to calculate the number of valid bracket sequences of length n when a *prefix* of the sequence is given.

Input

The first input line has an integer n.

The second line has a string of k characters: the prefix of the sequence.

Output

Print the number of sequences modulo $10^9 + 7$.

Constraints

- $1 \le k \le n \le 10^6$

This analysis assumes that you already know how to count the number of valid bracket sequences of length n using the reversed prefix technique. We use almost the same idea to solve this problem.

We go through the given prefix of length k and maintain the depth of the sequence. Each left bracket increases the depth by one, and each right bracket decreases the depth by one. If the depth ever becomes negative or the final depth is more than $n - k$, there are no solutions.

Otherwise, let us calculate the number of valid sequences. Let d denote the final depth of the prefix. The remaining sequence consists of $n - k$ characters and the number of left brackets is

$$l = \frac{n - k - d}{2}$$

so the total number of such sequences is

$$\binom{n-k}{l}.$$

Then, the number of invalid sequences equals

$$\binom{n-k}{l+d+1}$$

because after reversing the minimal invalid prefix, the number of left brackets increases by $d + 1$. Thus, the answer to the problem is

$$\binom{n-k}{l} - \binom{n-k}{l+d+1}.$$

```
#include <iostream>
using namespace std;
using ll = long long;
```

```
const int M = 1000000007;
const int N = 1000000;

int power(int a, int b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

int inv(int x) {
    return power(x, M - 2);
}

ll fac[N + 1];

int ncr(int a, int b) {
    return fac[a] * inv(fac[b] * fac[a - b] % M) % M;
}

int main() {
    fac[0] = 1;
    for (int i = 1; i <= N; i++) {
        fac[i] = fac[i - 1] * i % M;
    }

    int n;
    cin >> n;
    string s;
    cin >> s;
    int k = s.size();

    int depth = 0;
    bool fail = false;
    for (auto c : s) {
        if (c == '(') depth++;
        if (c == ')') depth--;
        if (depth < 0) fail = true;
    }
    if (depth > n - k) fail = true;

    if (n % 2 == 0 && !fail) {
        int left = (n - k - depth) / 2;
        ll answer = ncr(n - k, left) - ncr(n - k, left + depth + 1);
        answer = (answer % M + M) % M;
        cout << answer << "\n";
    } else {
        cout << "0\n";
    }
}
```

2186 · Special Substrings

Task · source analysis length 1,822 characters

Problem statement

A substring is called *special* if every character that appears in the string appears the same number of times in the substring.

Your task is to count the number of special substrings in a given string.

Input

The only input line has a string of length n. Every character is between a...z.

Output

Print one integer: the number of special substrings.

Constraints

- $1 \le n \le 2 \cdot 10^5$

In this analysis, we assume that each character from **a** to **z** appears in the string. If this is not the case, some extra work is required to ignore the missing characters.

Let $C_{i,c}$ be the number of occurrences of character c among the first i characters. Now, character c appears in the substring of characters at indices $[i, j)$ exactly $C_{j,c} - C_{i,c}$ times.

For a substring to be special, every character needs to appear a constant amount of times. We're not interested in the specific constant, which means we can keep a state of character occurrences relative to some reference character, say **a**:

$$\begin{aligned}(&C_{i,\mathtt{a}} - C_{i,\mathtt{a}},\\ &C_{i,\mathtt{b}} - C_{i,\mathtt{a}},\\ &\dots,\\ &C_{i,\mathtt{z}} - C_{i,\mathtt{a}}),\end{aligned}$$

Let us look at any character x. Note that if we add a constant amount of occurrences of both x and the reference character **a**, the relative count stays constant, which means it suffices to count the number of previous occurrences of the same state.

We will store the current state in an `array<int, 26>` and previous states to a `map`. The time complexity of a map operation is $O(\log n \cdot \Sigma)$, where Σ represents the alphabet size. The time complexity of the complete solution is $O(n \log n \cdot \Sigma)$.

```
#include <array>
#include <iostream>
#include <map>
using namespace std;
using ll = long long;

using A = array<int, 26>;

int main() {
    string s;
```

```
    cin >> s;

    bool seen[26]{};
    char reference_char = s[0];

    for (char c : s) seen[c - 'a'] = true;

    map<A, int> m;
    A cur{};
    m[cur] = 1;
    ll ans = 0;

    for (char c : s) {
        if (c == reference_char) {
            for (int i = 0; i < 26; ++i) {
                if (seen[i]) cur[i]--;
            }
        }
        cur[c - 'a']++;
        ans += m[cur];
        m[cur]++;
    }

    cout << ans << '\n';
}
```

2185 · Prime Multiples

Task · source analysis length 1,295 characters

Problem statement

You are given k distinct prime numbers $a_1, a_2, \ldots, a_k$ and an integer n.

Your task is to calculate how many of the first n positive integers are divisible by at least one of the given prime numbers.

Input

The first input line has two integers n and k.

The second line has k prime numbers $a_1, a_2, \ldots, a_k$.

Output

Print one integer: the number integers within the interval $1, 2, \ldots, n$ that are divisible by at least one of the prime numbers.

Constraints

- $1 \le n \le 10^{18}$
- $1 \le k \le 20$
- $2 \le a_i \le n$

We solve the problem using the inclusion-exclusion technique.

We go through every nonempty subset of $\{a_1, a_2, \ldots, a_k\}$. The number of integers between 1 and n that are divisible by all primes in the chosen subset equals $x = \lfloor n/p \rfloor$ where p is the product of the primes in the subset.

If the subset has an odd number of primes, we increase the answer by x, and otherwise we decrease the answer by x. After processing all the subsets, we have counted the numbers that are divisible by at least one prime.

Note that p can be a large number and it can overflow, so we have to make sure that the code works also in this case.

The algorithm works in $O(2^k k)$ time because it goes through $O(2^k)$ subsets and processes each subset in $O(k)$ time.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

int main() {
    ll n;
    int k;
    cin >> n >> k;

    vector<ll> primes(k);
    for (int i = 0; i < k; i++) {
        cin >> primes[i];
    }

    ll answer = 0;
```

```
    for (int b = 1; b < (1 << k); b++) {
        ll prod = 1;
        bool fail = false;

        for (int i = 0; i < k; i++) {
            if (b & (1 << i)) {
                if (primes[i] > n / prod) fail = true;
                prod *= primes[i];
            }
        }

        if (fail) continue;

        ll add = n / prod;
        if (__builtin_parity(b) == 0) add = -add;
        answer += add;
    }

    cout << answer << "\n";
}
```

2184 · Missing Coin Sum Queries

Task · source analysis length 2,388 characters

Problem statement

You have n coins with positive integer values. The coins are numbered $1, 2, \ldots, n$.

Your task is to process q queries of the form: "if you can use coins $a \ldots b$, what is the smallest sum you cannot produce?"

Input

The first input line has two integers n and q: the number of coins and queries.

The second line has n integers $x_1, x_2, \ldots, x_n$: the value of each coin.

Finally, there are q lines that describe the queries. Each line has two values a and b: you can use coins $a \ldots b$.

Output

Print the answer for each query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le a \le b \le n$

We process each query in B steps where $B = \lfloor \log_2 10^9 \rfloor = 30$. In each step, $b = 0, 1, \ldots, B-1$, we consider all coins in the query range such that the most significant bit of the coin value is in position b.

For each query i, we maintain a value $\mathbf{result}[i]$, which indicates that we can produce all coin sums between 1 and $\mathbf{result}[i]$ using the coins. Initially $\mathbf{result}[i] = 0$ for each $i = 1, 2, \ldots, q$.

In each step, we update the result for each query. Let $\mathbf{sum}(i, b)$ and $\mathbf{min}(i, b)$ denote the sum and minimum of coin values in the range of query i whose most significant bit is in position b. Always when $\mathbf{min}(i, b$ \le \textrm{result}

$$i$$

+1), we can extend the range of possible sums by adding $\mathbf{sum}(i, b)$ to $\mathbf{result}[i]$. Finally, $\mathbf{result}[i] + 1$ is the smallest missing coin sum.

We use data structures for efficient range sum and range minimum queries. Using these data structures, we can process range sum queries in $O(1)$ time after $O(n)$ time preprocessing and range minimum queries in $O(1)$ time after $O(n \log n)$ time preprocessing.

The resulting algorithm works in $O(B(n \log n + q))$ time.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int INF = 1e9 + 1;
const int B = 30;
```

```
const int K = 18;

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> x(n + 1);
    vector<int> bit(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> x[i];
        bit[i] = 31 - __builtin_clz(x[i]);
    }

    vector<pair<int, int>> query(q + 1);
    for (int i = 1; i <= q; i++) {
        cin >> query[i].first >> query[i].second;
    }

    vector<ll> result(q + 1);
    vector<ll> sumq(n + 1);
    vector<vector<int>> minq(K, vector<int>(n + 1));

    for (int b = 0; b < B; b++) {
        for (int i = 1; i <= n; i++) {
            sumq[i] = sumq[i - 1];
            if (bit[i] == b) {
                sumq[i] += x[i];
            }
        }

        for (int i = 1; i <= n; i++) {
            minq[0][i] = bit[i] == b ? x[i] : INF;
        }
        for (int k = 1; k < K; k++) {
            for (int i = 1; i <= n - (1 << (k - 1)); i++) {
                minq[k][i] =
                    min(minq[k - 1][i], minq[k - 1][i + (1 << (k - 1))]);
            }
        }

        for (int i = 1; i <= q; i++) {
            auto [a, b] = query[i];
            int level = 31 - __builtin_clz(b - a + 1);
            int min_x = min(minq[level][a], minq[level][b - (1 << level) + 1]);
            if (min_x <= result[i] + 1) {
                result[i] += sumq[b] - sumq[a - 1];
            }
        }
    }

    for (int i = 1; i <= q; i++) {
        cout << result[i] + 1 << "\n";
    }
}
```

2183 · Missing Coin Sum

Task · source analysis length 927 characters

Problem statement

You have n coins with positive integer values. What is the smallest sum you cannot create using a subset of the coins?

Input

The first line has an integer n: the number of coins.

The second line has n integers $x_1, x_2, \ldots, x_n$: the value of each coin.

Output

Print one integer: the smallest coin sum.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$

Suppose that we can create all sums $1, 2, \ldots, k$ using a subset of coins. Then, if we add a new coin whose value x is at most $k+1$, we can create all sums $1, 2, \ldots, k+x$, but if the value is more than $k+1$, we can't create the sum $k+1$ after adding the coin.

We can start with an empty subset, go through the coins in increasing order of their values and try to extend the subset at each coin. Either we can add the coin to the subset and increase the sum, or detect that no more coins can be added and we have found the solution.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

int main() {
    int n;
    cin >> n;

    vector<int> values(n);
    for (int i = 0; i < n; i++) {
        cin >> values[i];
    }
    sort(values.begin(), values.end());

    ll largest = 0;
    for (int i = 0; i < n; i++) {
        if (values[i] > largest + 1) {
            break;
        }
        largest += values[i];
    }
    cout << largest + 1 << "\n";
}
```

2182 · Divisor Analysis

Task · source analysis length 4,783 characters

Problem statement

Given an integer, your task is to find the number, sum and product of its divisors. As an example, let us consider the number 12:

- the number of divisors is 6 (they are 1, 2, 3, 4, 6, 12)
- the sum of divisors is $1+2+3+4+6+12=28$
- the product of divisors is $1 \cdot 2 \cdot 3 \cdot 4 \cdot 6 \cdot 12 = 1728$

Since the input number may be large, it is given as a prime factorization.

Input

The first line has an integer n: the number of parts in the prime factorization.

After this, there are n lines that describe the factorization. Each line has two numbers x and k where x is a prime and k is its power.

Output

Print three integers modulo 10^9+7: the number, sum and product of the divisors.

Constraints

- $1 \le n \le 10^5$
- $2 \le x \le 10^6$
- each x is a distinct prime
- $1 \le k \le 10^9$

We are given a number

$$N = \prod_{i=1}^{n} x_i^{k_i}$$

and our task is to calculate the number, sum and product of its divisors.

Solution 1

We calculate the number of divisors as follows:

$$d(N) = \prod_{i=1}^{n} (k_i + 1)$$

The formula works because there are $k_i + 1$ ways to choose a factor x_i.

We calculate the sum of divisors as follows:

$$s(N) = \prod_{i=1}^{n}(1 + x_i + x_i^2 + \cdots + x_i^{k_i}) = \prod_{i=1}^{n} \frac{x_i^{k_i+1} - 1}{x_i - 1}$$

This formula corresponds to a sum where each divisor appears exactly once. Since each part of the product is a geometric sum, we can calculate the answer more efficiently using the geometric sum formula.

We calculate the product of divisors as follows:

$$p(N) = \prod_{i=1}^{n}(x_i^{k_i(k_i+1)/2})^{d(N)/(k_i+1)}$$

This formula works because if a factor x_i appears in a divisor, it can appear between 1 and k_i times, so we can use the sum formula $k_i(k_i+1/2)$ to count the total number of factor appearances. Moreover, there are $d(N/k_i+1)$ ways to choose the other factors.

Note that since we want to calculate the answer modulo M, we have to calculate $d(N/k_i+1)$ modulo $M-1$. We can't use modular multiplicative inverses here because k_i+1 and $M-1$ are not necessarily coprime. Instead, we create two additional vectors that store prefix and suffix products of k_i+1 values and use them in the last loop.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

ll power(ll a, ll b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

ll inv(ll x) {
    return power(x, M - 2);
}

int main() {
    int n;
    cin >> n;

    vector<ll> x(n), k(n);
    for (int i = 0; i < n; i++) {
        cin >> x[i] >> k[i];
    }

    ll count = 1;
    for (int i = 0; i < n; i++) {
        count = count * (k[i] + 1) % M;
    }

    ll sum = 1;
    for (int i = 0; i < n; i++) {
        sum = sum * (power(x[i], k[i] + 1) - 1) % M;
        sum = sum * inv(x[i] - 1) % M;
    }

    vector<ll> prefix(n + 1);
    prefix[0] = 1;
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] * (k[i] + 1) % (M - 1);
    }

    vector<ll> suffix(n);
    suffix[n - 1] = 1;
    for (int i = n - 1; i > 0; i--) {
        suffix[i - 1] = suffix[i] * (k[i] + 1) % (M - 1);
    }

    ll prod = 1;
    for (int i = 0; i < n; i++) {
        ll ways = prefix[i] * suffix[i] % (M - 1);
        prod *= power(power(x[i], k[i] * (k[i] + 1) / 2), ways);
        prod %= M;
    }
```

```
    cout << count << " " << sum << " " << prod << "\n";
}
```

Solution 2

A more elegant way to calculate the product of divisors is as follows:

$$p(N) = N^{d(N)/2}$$

We can derive this formula from the previous formula by simplifying the exponents:

$$p(N) = \prod_{i=1}^{n} (x_i^{k_i(k_i+1)/2})^{d(N)/(k_i+1)} = \prod_{i=1}^{n} (x_i^{k_i})^{d(N)/2} = N^{d(N)/2}$$

The following solution calculates the product of divisors using this formula and does not require additional vectors.

We need to calculate $d(N/2)$ modulo $M-1$ and we can't use modular multiplicative inverses because $M-1$ and 2 are not coprime. We handle two cases in the code:

Case 1: N is not square. There is an even value k_i+1 and we divide the first such value by two to calculate the exponent $d(N/2)$.

Case 2: N is square. Now $d(N)$ is odd and $d(N/2)$ is not an integer. However, $\sqrt{N}$ is an integer so we can use the formula $N^{d(N)/2} = \sqrt{N}^{d(N)}$ to calculate the answer. In this case all k_i values are even and we divide each of them by two in the last loop.

Another way to explain the formula $N^{d(N)/2}$ is to consider divisor pairs of the form $(k, N/k)$ where $k < N/k$. When N is not square, there are $d(N/2)$ pairs and the product of each pair is N. When N is square, there are $(d(N-1)/2)$ pairs with product N and then an additional divisor $\sqrt{N} = N^{1/2}$. The formula holds also in this case because

$$N^{(d(N)-1)/2} N^{1/2} = N^{d(N)/2}.$$

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

ll power(ll a, ll b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

ll inv(ll x) {
    return power(x, M - 2);
}

int main() {
    int n;
    cin >> n;

    vector<ll> x(n), k(n);
    for (int i = 0; i < n; i++) {
        cin >> x[i] >> k[i];
    }

    ll count = 1;
    for (int i = 0; i < n; i++) {
        count = count * (k[i] + 1) % M;
    }

    ll sum = 1;
    for (int i = 0; i < n; i++) {
        sum = sum * (power(x[i], k[i] + 1) - 1) % M;
        sum = sum * inv(x[i] - 1) % M;
    }
```

```
    ll count2 = 1;
    bool is_square = true;
    for (int i = 0; i < n; i++) {
        if (is_square && k[i] % 2 == 1) {
            count2 *= (k[i] + 1) / 2;
            is_square = false;
        } else {
            count2 *= (k[i] + 1);
        }
        count2 %= M - 1;
    }

    ll prod = 1;
    for (int i = 0; i < n; i++) {
        if (is_square) k[i] /= 2;
        prod *= power(power(x[i], k[i]), count2);
        prod %= M;
    }

    cout << count << " " << sum << " " << prod << "\n";
}
```

2181 · Counting Tilings

Task · source analysis length 4,023 characters

Problem statement

Your task is to count the number of ways you can fill an $n \times m$ grid using 1×2 and 2×1 tiles.

Input

The only input line has two integers n and m.

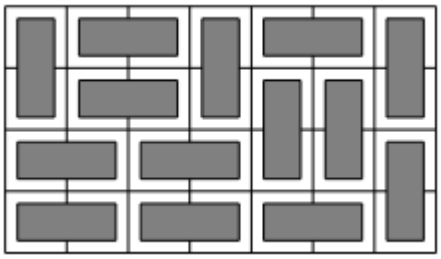
Output

Print one integer: the number of ways modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10$
- $1 \le m \le 1000$

We use dynamic programming to solve the problem. We represent each column pattern as a bitmask that indicates the rows where a horizontal tile begins. For example, consider the following tiling:



Here the bitmasks 0011, 1100, 0011, 0000, 1001, 0000 and 0000 correspond to the column patterns from left to right.

A pair of bitmasks (a, b) is valid if patterns a and b can occur next to each other in the tiling. For example, $(0011, 1100)$ is a valid pair which corresponds to the first two columns in the example. The pair $(0011, 1101)$ is not valid, because a horizontal tile begins in the last row in both bitmasks.

A simple solution would be to iterate over all pairs of bitmasks in every column. There are 4^n bitmask pairs and checking if a pair is valid takes $O(n)$ time, so such a solution would require $O(mn4^n)$ time. This can be optimized by only checking pairs of bitmasks which share no bits. There are exactly 3^n such bitmasks, which would result in an $O(mn3^n)$ solution.

An $O(mn3^n)$ solution can be made fast enough, but since in practice there are much less valid transitions than 3^n, and the same transitions are used for each column, it makes sense to precompute them.

Solution 1: Precomputed transitions

In this solution, the transitions are computed recursively before performing dynamic programming, one or two bits at a time. The bitmask `left` represents which rows have a horizontal tile going left, and `right`

represents which rows have a horizontal tile going right. Other rows are occupied by vertical tiles, which have a height of two.

Dynamic programming values from only the previous column need to be known, so two vectors are used and swapped between columns.

It can be shown [2] that the number of transitions is $O((1+\sqrt{2}$^n)). Therefore the time complexity of this solution is $O(m(1+\sqrt{2}$^n)), or approximately $O(m2.42^n)$.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
const int M = 1000000007;

int n;
vector<pair<int, int>> transitions;

void generate(int i, int left, int right) {
    if (i > n) return;
    if (i == n) {
        transitions.emplace_back(left, right);
        return;
    }
    generate(i + 1, left | 1 << i, right);
    generate(i + 1, left, right | 1 << i);
    generate(i + 2, left, right);
}

int main() {
    int m;
    cin >> n >> m;

    generate(0, 0, 0);

    vector<int> cur(1 << n), prev(1 << n);
    prev[0] = 1;

    for (int i = 0; i < m; ++i) {
        fill(cur.begin(), cur.end(), 0);

        for (auto [x, y] : transitions) {
            cur[y] += prev[x];
            cur[y] %= M;
        }

        swap(cur, prev);
    }

    cout << prev[0] << endl;
}
```

Solution 2: Sum over subsets

Let us take a closer look at the transitions.

One transition that is always valid is from the bitmask s to the complement of s. At every position where there is not a tile going to the left, we choose a tile going to the right. Complementing the bitmasks is equivalent to reversing the dynamic programming array.

The other transitions from s are formed by replacing some adjacent pairs of tiles going to the right with vertical pieces.

The sum over subsets technique can be used to compute the desired sum of valid transitions. More precisely, as the value associated with the bitmask s we sum all values of supersets of s that may have some added adjacent pairs of 1-bits. The technique works similarly even with this added restriction.

The time complexity of this solution is $O(mn2^n)$. For $n = 10$ it is very similar in performance to the first solution.

```
#include <algorithm>
#include <iostream>
using namespace std;
const int M = 1000000007;
```

```
int tilings[1 << 10];

int main() {
    int n, m;
    cin >> n >> m;

    tilings[0] = 1;

    for (int i = 0; i < m; ++i) {
        reverse(tilings, tilings + (1 << n));

        for (int j = 0; j < n - 1; ++j) {
            for (int s = 0; s < (1 << n); ++s) {
                if ((s & 0b11 << j) == 0) {
                    tilings[s] += tilings[s | 0b11 << j];
                    tilings[s] %= M;
                }
            }
        }
    }

    cout << tilings[0] << endl;
}
```

References

1. CPHB *Competitive Programmer's Handbook*, Chapters 7.6 and 10.6, Counting subsets.
2. Pell number *Wikipedia*

2177 · Strongly Connected Edges

Task · source analysis length 1,665 characters

Problem statement

Given an undirected graph, your task is to choose a direction for each edge so that the resulting directed graph is strongly connected.

Input

The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$.

After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b.

You may assume that the graph is simple, i.e., there are at most one edge between two nodes and every edge connects two distinct nodes.

Output

Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution.

If there are no solutions, only print `IMPOSSIBLE`.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

It is useful to approach this problem by focusing on a DFS tree of the undirected graph. The DFS contains *tree edges* and *back edges*.

We will direct each tree edge from the parent to the child. Further, let each *back edge* run upwards towards the root. Turns out this construction always works if it is possible to make the graph strongly connected overall.

There are two cases in which it is not possible to make the graph strongly connected. Firstly, it is not possible if the given graph is not connected. Secondly, if there is a subtree in the DFS tree from which there is not a single back-edge going outside of it, it is also not possible, because there is no way to reach other nodes from the subtree.

```
#include <iostream>
#include <vector>
using namespace std;

void fail() {
    cout << "IMPOSSIBLE" << '\n';
    exit(0);
}

const int N = 100001;
vector<int> g[N];
vector<pair<int, int>> ans;
bool visited[N];
int depth[N], min_depth[N];
```

```
void dfs(int node, int prev) {
    visited[node] = true;
    min_depth[node] = depth[node];
    for (int nxt : g[node]) {
        if (nxt == prev) continue;
        if (visited[nxt]) {
            if (depth[nxt] < depth[node]) {
                ans.emplace_back(node, nxt);
            }
            min_depth[node] = min(min_depth[node], depth[nxt]);
        } else {
            depth[nxt] = depth[node] + 1;
            dfs(nxt, node);
            min_depth[node] = min(min_depth[node], min_depth[nxt]);
            ans.emplace_back(node, nxt);
            if (min_depth[nxt] > depth[node]) fail();
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);

    if (ans.size() != m) fail();

    for (auto [a, b] : ans) {
        cout << a << ' ' << b << '\n';
    }
}
```

2169 · Nested Ranges Count

Task · source analysis length 1,584 characters

Problem statement

Given n ranges, your task is to count for each range how many other ranges it contains and how many other ranges contain it.

Range $[a,b]$ contains range $[c,d]$ if $a \leq c$ and $d \leq b$.

Input

The first input line has an integer n: the number of ranges.

After this, there are n lines that describe the ranges. Each line has two integers x and y: the range is $[x,y]$.

You may assume that no range appears more than once in the input.

Output

First print a line that describes for each range $in the input order$ how many other ranges it contains.

Then print a line that describes for each range $in the input order$ how many other ranges contain it.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x < y \leq 10^9$

This problem is like the previous problem, but instead of just checking if a range contains another range, we have to count the number of ranges.

Also in this problem we can use the idea that is explained in the analysis of the previous problem. The only difference is that instead of maintaining a rightmost position of an endpoint, we create a policy-based indexed set that contains all the endpoints. Then we can use the function `order_of_key` to calculate the number of endpoints after some point.

Since policy-based sets can't be multisets, we need a small trick: instead of just storing endpoints, we store a pair of an endpoint and the index of the corresponding range.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using indexed_set =
    tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

int n;

void solve(vector<vector<int>> ranges) {
    vector<int> count(n + 1);
    indexed_set<pair<int, int>> points;

    for (auto r : ranges) {
```

```
        count[r[2]] = points.size() - points.order_of_key({-r[1], 0});
        points.insert({-r[1], r[2]});
    }

    for (int i = 1; i <= n; i++) {
        cout << count[i] << " ";
    }
    cout << "\n";
}

int main() {
    cin >> n;

    vector<vector<int>> ranges1;
    vector<vector<int>> ranges2;

    for (int i = 1; i <= n; i++) {
        int x, y;
        cin >> x >> y;
        ranges1.push_back({y, -x, i});
        ranges2.push_back({x, -y, i});
    }

    sort(ranges1.begin(), ranges1.end());
    sort(ranges2.begin(), ranges2.end());

    solve(ranges1);
    solve(ranges2);
}
```

2168 · Nested Ranges Check

Task · source analysis length 1,728 characters

Problem statement

Given n ranges, your task is to determine for each range if it contains some other range and if some other range contains it.

Range $[a,b]$ contains range $[c,d]$ if $a \leq c$ and $d \leq b$.

Input

The first input line has an integer n: the number of ranges.

After this, there are n lines that describe the ranges. Each line has two integers x and y: the range is $[x,y]$.

You may assume that no range appears more than once in the input.

Output

First print a line that describes for each range $in the input order$ if it contains some other range 1 or not 0.

Then print a line that describes for each range $in the input order$ if some other range contains it 1 or not 0.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x < y \leq 10^9$

We can solve this problem by using a clever way to sort the ranges, and it turns out that both the subproblems can be solved using the same idea.

Let us first focus on the first subproblem of finding ranges that contain some other range. We sort the ranges in increasing order by right endpoint, and secondarily in decreasing order by left endpoint. Then we go through the ranges and maintain a value p: the rightmost position of a left endpoint. A range $[x,y]$ contains some other range exactly when $p \geq x$.

Then consider the second subproblem of finding ranges that are contained in other ranges. This time we sort the ranges in increasing order by left endpoint, and secondarily in decreasing order by right endpoint. We maintain a value p that stores the rightmost position of a right endpoint, and some other range contains range $[x,y]$ exactly when $y \leq p$.

In the following code we use the function `solve` for both subproblems. There is a small implementation trick: we use negative endpoints to easily sort the ranges secondarily in decreasing order.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int n;

void solve(vector<vector<int>> ranges) {
    vector<int> check(n + 1);
```

```
    int pos = 0;
    for (auto r : ranges) {
        if (pos >= -r[1]) {
            check[r[2]] = 1;
        }
        pos = max(pos, -r[1]);
    }

    for (int i = 1; i <= n; i++) {
        cout << check[i] << " ";
    }
    cout << "\n";
}

int main() {
    cin >> n;

    vector<vector<int>> ranges1;
    vector<vector<int>> ranges2;

    for (int i = 1; i <= n; i++) {
        int x, y;
        cin >> x >> y;
        ranges1.push_back({y, -x, i});
        ranges2.push_back({x, -y, i});
    }

    sort(ranges1.begin(), ranges1.end());
    sort(ranges2.begin(), ranges2.end());

    solve(ranges1);
    solve(ranges2);
}
```

2166 · Prefix Sum Queries

Task · source analysis length 2,348 characters

Problem statement

Given an array of n integers, your task is to process q queries of the following types:

1. update the value at position k to u
2. what is the maximum prefix sum in range $[a, b]$?

Empty prefixes $with sum 0$ are allowed.

Input

The first input line has two integers n and q: the number of values and queries.

The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has three integers: either "$1\ k\ u$" or "$2\ a\ b$".

Output

Print the result of each query of type 2.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $-10^9 \le x_i, u \le 10^9$
- $1 \le k \le n$
- $1 \le a \le b \le n$

A simple sum/max segment tree will not suffice for this problem. We need to store additional information in the nodes.

For each node, we will store the sum of the values in the range corresponding to that node and also the maximum prefix sum of those values.

A regular query function will not work because we have to keep track of the prefix sum. For this purpose, two variables are used:

1. **prefix_sum**, this variable will be used by the left pointer which will accumulate a prefix sum from the left. It will hold the sum of values from the beginning of the query range to the beginning of the current node.
2. **subfix_sum**, this variable will be used by the right pointer. We cannot accumulate the sum as we go because the right pointer starts from the very end of the query range. Thus, the sum needs to be calculated beforehand and the right pointer will subtract from it as it moves up in the tree.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int TREE_SIZE = 1 << 18;
struct Node {
    ll sum;
    ll max_prefix;
} tree[TREE_SIZE * 2];
```

```
Node merge(Node l, Node r) {
    return {l.sum + r.sum, max(l.max_prefix, l.sum + r.max_prefix)};
}

void change(int i, int x) {
    i += TREE_SIZE;
    tree[i] = {x, x};
    while (i > 1) {
        i /= 2;
        tree[i] = merge(tree[i * 2], tree[i * 2 + 1]);
    }
}

// Returns the sum of the value in range [l, r]
ll query_sum(int l, int r) {
    ll sum = 0;
    l += TREE_SIZE;
    r += TREE_SIZE;
    while (l <= r) {
        if (l % 2 == 1) sum += tree[l++].sum;
        if (r % 2 == 0) sum += tree[r--].sum;
        l /= 2;
        r /= 2;
    }
    return sum;
}

// Returns the maximum prefix sum in range [l, r]
ll query(int l, int r) {
    ll prefix_sum = 0;
    ll subfix_sum = query_sum(l, r);
    ll res = 0;
    l += TREE_SIZE;
    r += TREE_SIZE;
    while (l <= r) {
        if (l % 2 == 1) {
            res = max(res, prefix_sum + tree[l].max_prefix);
            prefix_sum += tree[l].sum;
            l++;
        }
        if (r % 2 == 0) {
            subfix_sum -= tree[r].sum;
            res = max(res, subfix_sum + tree[r].max_prefix);
            r--;
        }
        l /= 2;
        r /= 2;
    }
    return res;
}

int main() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; ++i) {
        int x;
        cin >> x;
        change(i, x);
    }

    for (int qi = 0; qi < q; ++qi) {
        int t;
        cin >> t;
        if (t == 1) {
            int k, u;
            cin >> k >> u;
            change(k, u);
        } else {
            int l, r;
            cin >> l >> r;
            cout << query(l, r) << '\n';
        }
    }
}
```

2165 · Tower of Hanoi

Task · source analysis length 1,405 characters

Problem statement

The Tower of Hanoi game consists of three stacks $left, middle and right$ and n round disks of different sizes. Initially, the left stack has all the disks, in increasing order of size from top to bottom.

The goal is to move all the disks to the right stack using the middle stack. On each move you can move the uppermost disk from a stack to another stack. In addition, it is not allowed to place a larger disk on a smaller disk.

Your task is to find a solution that minimizes the number of moves.

Input

The only input line has an integer n: the number of disks.

Output

First print an integer k: the minimum number of moves.

After this, print k lines that describe the moves. Each line has two integers a and b: you move a disk from stack a to stack b.

Constraints

- $1 \le n \le 16$

The following solution uses a recursive function that moves n disks from stack a to stack b using stack c. The function assumes that all disks in stacks b and c are larger than the n disks in stack a which will be moved.

If $n = 1$, the function just moves a disk from stack a to stack b. Otherwise, the function recursively moves $n - 1$ disks from stack a to stack c, then one disk from stack a to stack b and finally $n - 1$ disks from stack c to stack b.

Using the function, we can solve the problem by moving n disks from the left stack to the right stack using the middle stack. The total number of moves will be $2^n - 1$ which is the optimal solution.

Why is the solution optimal? Since we have to move the largest disk from the left stack to the right stack, we have to move all other disks to the middle stack so that the right stack is empty and we can move the largest disk. After that, we have to move all other disks from the middle stack to the right stack.

```
#include <iostream>
#include <vector>
using namespace std;

vector<pair<int, int>> moves;

void move(int n, int a, int b, int c) {
    if (n == 1) {
        moves.emplace_back(a, b);
    } else {
        move(n - 1, a, c, b);
        move(1, a, b, c);
        move(n - 1, c, b, a);
    }
}
```

```
int main() {
    int n;
    cin >> n;
    move(n, 1, 3, 2);
    cout << moves.size() << "\n";
    for (auto [a, b] : moves) {
        cout << a << " " << b << "\n";
    }
}
```

References

- Tower of Hanoi *Wikipedia*

2164 · Josephus Queries

Task · source analysis length 1,788 characters

Problem statement

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, every second child is removed from the circle, until there are no children left.

Your task is to process q queries of the form: "when there are n children, who is the kth child that will be removed?"

Input

The first input line has an integer q: the number of queries.

After this, there are q lines that describe the queries. Each line has two integers n and k: the number of children and the position of the child.

Output

Print q integers: the answer for each query.

Constraints

- $1 \le q \le 10^5$
- $1 \le k \le n \le 10^9$

We can divide the process of removing children into rounds where we go through the children in increasing order and remove every second of them. For example, when $n = 7$, the rounds are as follows:

- Round 1: $[1, \underline{2}, 3, \underline{4}, 5, \underline{6}, 7]$
- Round 2: $[1, \underline{3}, \underline{5}, 7]$
- Round 3: $[\underline{3}, 7]$
- Round 4: $[\underline{7}]$

To efficiently find the kth removed child, we simulate the rounds using a recursive function whose each call corresponds to a round.

Let $\mathrm{child}(n, k, p)$ denote the kth child who will be removed when there are n children and the parity of the first child to be removed is p (0 means we first remove child 2, and 1 means we first remove child 1).

The number of children removed on the first round is $r = \lfloor n + p \rfloor / 2$. If $k \le r$, we remove the kth child on the first round and just return the number of the child which is $2k - p$.

Otherwise we call the function recursively: we have $n - r$ children left and we should find the $(k - r)$th child who will be removed. Here u denotes the new parity: if n is even, the parity remains the same, and if n is odd, the parity changes. Since the numbering of the children changes in the subproblem, we have to convert it after the recursive call.

Since we remove half of the children on each round, the number of rounds is $O(\log n)$ and the function works in $O(\log n)$ time.

```
#include <iostream>
using namespace std;

int child(int n, int k, int p) {
```

```
    int r = (n + p) / 2;
    if (k <= r) return k * 2 - p;
    int u = n % 2 == 0 ? p : 1 - p;
    return child(n - r, k - r, u) * 2 - 1 + p;
}

int main() {
    int q;
    cin >> q;

    for (int qi = 1; qi <= q; qi++) {
        int n, k;
        cin >> n >> k;
        cout << child(n, k, 0) << "\n";
    }
}
```

References

- Josephus problem *Wikipedia*

2163 · Josephus Problem II

Task · source analysis length 940 characters

Problem statement

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, repeatedly k children are skipped and one child is removed from the circle. In which order will the children be removed?

Input

The only input line has two integers n and k.

Output

Print n integers: the removal order.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $0 \le k \le 10^9$

A convenient way to solve this problem is to use a policy-based indexed set data structure. This data structure has a function `find_by_order` which can be used to find an element at a given index in the ordered set.

At each step, we calculate the next position and print and remove the element at that position. Each step requires $O(\log n)$ time, so the algorithm works in $O(n \log n)$ time.

```
#include <iostream>
using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using indexed_set =
    tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

int main() {
    int n, k;
    cin >> n >> k;

    indexed_set<int> s;
    for (int i = 1; i <= n; i++) {
        s.insert(i);
    }

    int pos = 0;
    for (int i = 1; i <= n; i++) {
        pos = (pos + k) % (n - i + 1);
        auto it = s.find_by_order(pos);
        cout << *it << " ";
        s.erase(it);
    }
    cout << "\n";
}
```

References

- Josephus problem *Wikipedia*

2143 · Reachability Queries

Task · source analysis length 2,671 characters

Problem statement

A directed graph consists of n nodes and m edges. The edges are numbered $1, 2, \ldots, n$.

Your task is to answer q queries of the form "can you reach node b from node a?"

Input

The first input line has three integers n, m and q: the number of nodes, edges and queries.

Then there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge from node a to node b.

Finally there are q lines describing the queries. Each line consists of two integers a and b: "can you reach node b from node a?"

Output

Print the answer for each query: either "YES" or "NO".

Constraints

- $1 \le n \le 5 \cdot 10^4$
- $1 \le m, q \le 10^5$

It is useful to think of the problem in terms of strongly connected components. In particular, if two nodes are in the same in the same component, they are both reachable from each other.

The strongly connected components of the graph also form a directed acyclic graph DAG of their own. There cannot be cycles in the graph condensed from the components because if it contained a cycle of multiple components, the nodes of the cycle would, by definition, form only a single component.

Once we have found the strongly connected components using Tarjan's algorithm, we have reduced the problem to a DAG. Because there are no cycles in a DAG, we can make a dynamic programming solution. In particular, we may store, for each node, information about which other nodes are reachable from it. The reachability information can be copied to each *parent* of a node, once all the children of the node have been fully processed.

If we were to store the reachability information in a boolean array, the solution would turn out too slow. We may, however, use bitsets to speed up the transformation of the information as bitsets allow us to update 64 elements at once.

```
#include <bitset>
#include <iostream>
#include <vector>
using namespace std;

const int N = 5e4 + 1;
vector<int> g[N];
int cur_depth, comp_cnt;
int comp[N], depth[N], low[N];
vector<int> stack;
bool in_stack[N];

// Tarjan's strongly connected components algorithm
```

```
void scc_dfs(int node) {
    low[node] = depth[node] = ++cur_depth;
    stack.push_back(node);
    in_stack[node] = true;
    for (int nxt : g[node]) {
        if (!depth[nxt]) {
            scc_dfs(nxt);
            low[node] = min(low[node], low[nxt]);
        } else if (in_stack[nxt]) {
            low[node] = min(low[node], depth[nxt]);
        }
    }
    if (depth[node] == low[node]) {
        int i = -1;
        ++comp_cnt;
        while (i != node) {
            i = stack.back();
            stack.pop_back();
            in_stack[i] = false;
            comp[i] = comp_cnt;
        }
    }
}

using B = bitset<N>;
// `cg` is a condensed graph constructed from the SCC's.
vector<int> cg[N];
B reach[N];
bool visited[N];

void dfs(int node) {
    if (visited[node]) return;
    visited[node] = true;
    reach[node][node] = true;
    for (int nxt : cg[node]) {
        dfs(nxt);
        reach[node] |= reach[nxt];
    }
}

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }

    for (int i = 1; i <= n; ++i) {
        if (!depth[i]) scc_dfs(i);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j : g[i]) {
            if (comp[i] != comp[j]) {
                cg[comp[i]].push_back(comp[j]);
            }
        }
    }

    for (int i = 1; i <= comp_cnt; ++i) {
        dfs(i);
    }

    for (int qi = 1; qi <= q; ++qi) {
        int a, b;
        cin >> a >> b;
        cout << (reach[comp[a]][comp[b]] ? "YES" : "NO") << '\n';
    }
}
```

2138 · Reachable Nodes

Task · source analysis length 1,376 characters

Problem statement

A directed acyclic graph consists of n nodes and m edges. The nodes are numbered $1, 2, \ldots, n$.

Calculate for each node the number of nodes you can reach from that node $including the node itself$.

Input

The first input line has two integers n and m: the number of nodes and edges.

Then there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge from node a to node b.

Output

Print n integers: for each node the number of reachable nodes.

Constraints

- $1 \le n \le 5 \cdot 10^4$
- $1 \le m \le 10^5$

A directed acyclic graph does not contain cycles which means that we may use a dynamic programming approach to solve the problem. Let us store a boolean array for each node, which tells us which nodes are reachable from the node. A straightforward solution would perform a DFS from each node and update the arrays after each edge. Each edge would correspond to $O(n)$ operations and thus, the solution would result in a time complexity of $O(nm)$ and would perform roughly $5 \cdot 10^4 \cdot 10^5 = 5 \cdot 10^9$ operations, which makes the solution a bit too slow.

Fortunately, the approach can be sped up using bitsets. Bitsets allow us to update 64 positions of the array at once and thus, provide a factor of $1/64$ to the number of operations, bringing the total down to the order of $5 \cdot 10^4 \cdot 10^5 \cdot \frac{1}{64} \approx 7.8 \cdot 10^7$.

```
#include <bitset>
#include <iostream>
#include <vector>
using namespace std;

const int N = 5e4 + 1;

vector<int> g[N];
bitset<N> reach[N];
bool visited[N];

void dfs(int node) {
    if (visited[node]) return;
    visited[node] = true;
    reach[node][node] = true;
    for (int nxt : g[node]) {
        dfs(nxt);
        reach[node] |= reach[nxt];
    }
}

int main() {
    int n, m;
```

```
    cin >> n >> m;

    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }

    for (int i = 1; i <= n; ++i) {
        dfs(i);
        cout << reach[i].count() << ' ';
    }
    cout << '\n';
}
```

2137 · Corner Subgrid Count

Task · source analysis length 2,437 characters

Problem statement

You are given an $n \times n$ grid whose each square is either black or white. A subgrid is called *beautiful* if its height and width is at least two and all of its corners are black. How many beautiful subgrids are there within the given grid?

Input

The first input line has an integer n: the size of the grid.

Then there are n lines describing the grid: `1` means that a square is black and `0` means it is white.

Output

Print the number of beautiful subgrids.

Constraints

- $1 \le n \le 3000$

Going through all possible corners of a subgrid would run in $O(n^4)$ and thus, be too slow. There is, however, a way to calculate the number of subgrids by only going through each pair of rows.

Let us go through each pair of rows and find each subgrid which spans from one of the selected rows to another. For a pair of rows to form a subgrid, the rows need to share two distinct positions which have black squares in both of the rows. If we know the number of such positions is c, the number of subgrids the rows form, is $\binom{c}{2} = \frac{c(c-1)}{2}$, as we may choose any two of them as the borders of the subgrid. This algorithm runs in $O(n^3)$ as we go through each pair of rows in $O(n)$ time and there are in total $O(n^2)$ pairs of rows.

With $n = 3000$, the solution would perform roughly $3000^3 \approx 3 \cdot 10^{10}$ operations and would still be too slow. We may, however, use bitsets to improve the constant factor of the solution. Bitsets allow us to check multiple positions at once. On a Linux system, bitsets usually contain 64-bit integers. This means that, with constant time bit operations, we can check 64 positions at once. Thus, the number of operations becomes $3000^3 \cdot \frac{1}{64} \approx 4 \cdot 10^8$.

To find the positions where the two rows both have a black square, we will use the bitwise AND operation as it discards the positions where only one or neither of the squares are black. Finally, we will count the number of bits set in the bitset using the method `.count()`. Note that even with the bitset optimization the solution will turn out too slow without the line

```
#pragma GCC target("popcnt")
```

which tells the compiler that our machine has an instruction for calculating the popcount of an integer, that is, our processor is able to count the number of bits set in an integer efficiently. The popcount instruction is available on CSES judges and has been included in most of Intel's x86-64 processors since 2008.

```
#pragma GCC target("popcnt")
#include <bitset>
#include <iostream>
#include <vector>
```

```
using namespace std;
using ll = long long;
using B = bitset<3000>;

int main() {
    int n;
    cin >> n;

    vector<B> b(n);
    for (int i = 0; i < n; ++i) {
        string s;
        cin >> s;
        for (int j = 0; j < n; ++j) {
            if (s[j] == '1') {
                b[i][j] = true;
            }
        }
    }

    ll r = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            int c = (b[i] & b[j]).count();
            r += c * (c - 1) / 2;
        }
    }

    cout << r << '\n';
}
```

2136 · Hamming Distance

Task · source analysis length 2,192 characters

Problem statement

The Hamming distance between two strings a and b of equal length is the number of positions where the strings differ.

You are given n bit strings, each of length k and your task is to calculate the minimum Hamming distance between two strings.

Input

The first input line has two integers n and k: the number of bit strings and their length.

Then there are n lines each consisting of one bit string of length k.

Output

Print the minimum Hamming distance between two strings.

Constraints

- $2 \le n \le 2 \cdot 10^4$
- $1 \le k \le 30$

If we were to store each bit separately in a `bool` for example, and then go through each of the bits when comparing two bit strings, the code would turn out way too slow. This solution has a time complexity of $O(n^2k)$, as it goes through each bit in each pair of strings. Fortunately, there is a much more efficient way of storing the bit strings. What would be a better way to store them than regular integers?

A C++ `int` has a width of 32 bits, which is more than enough for this task. The most important observation is that the hamming distance between two bit strings can be calculated efficiently using the bitwise xor operation. In fact, the hamming distance corresponds to the amount of bits set in the bitwise xor. Note that in a bitwise xor, matching bits will result in a zero and differing bits in a one bit. This is clearly visible in the sample input.

$$\begin{aligned} & 101000 \\ \oplus\ & 001000 \\ \hline & 100000 \end{aligned}$$

As each set bit corresponds to a position of differing bits, the amount of bits set corresponds to the hamming distance. In the example, the number of bits set is 1 and thus, the answer in 1 as well.

An important function to know is `__builtin_popcount`, which calculates the popcount of an `int` efficiently. The aforementioned function works even faster if the compiler knows that we are on a machine which has an instruction for calculating the popcount. Most modern processors have this instruction, including the CSES judges, and the following line may be used to convey this information to the compiler.

```
#pragma GCC target("popcnt")
```

If we assume that k is proportional to the machine word size, we can say that xor and popcount operations take $O(1)$ time, resulting in a time complexity of $O(n^2)$.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    vector<int> b(n);
    for (int i = 0; i < n; ++i) {
        string s;
        cin >> s;
        for (int j = 0; j < k; ++j) {
            if (s[j] == '1') {
                b[i] += 1 << j;
            }
        }
    }

    int r = 1e9;
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            r = min(r, __builtin_popcount(b[i] ^ b[j]));
        }
    }

    cout << r << '\n';
}
```

2134 · Path Queries II

Task · source analysis length 5,180 characters

Problem statement

You are given a tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$. Each node has a value.

Your task is to process following types of queries:

1. change the value of node s to x
2. find the maximum value on the path between nodes a and b.

Input

The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$.

The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node.

Then there are $n - 1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 a b".

Output

Print the answer to each query of type 2.

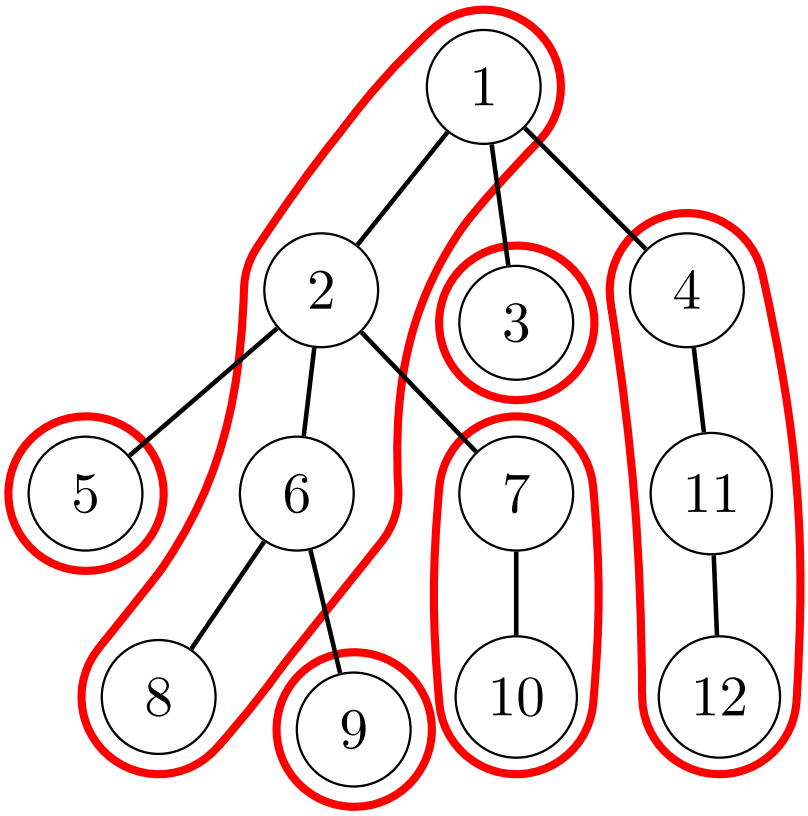
Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le a, b, s \le n$
- $1 \le v_i, x \le 10^9$

This problem can be solved with a technique known as *heavy-light decomposition*.

We will call, for each node, the child with the largest bysize subtree a *heavy child*. Every other child is a *light child*.

Heavy and *light* children will partition the tree into paths. A path consists of repeated *heavy* nodes. For each node, let us store a pointer that points to the end of the path the node belongs to.



The key observation here is that the path from any node to the root will consist of $O(\log n)$ of these *heavy* paths. This, in itself, does not seem like a powerful technique. We need a few more observations.

We will move the *heavy* child the first place in the adjacency list for each node. After this, we will make a depth-first search and store the nodes – in the order they are visited – to an array. Any heavy child will follow its parent in the DFS array. Furthermore, each heavy path corresponds to a contiguous range in the array.

Now, how do we partition a path between two nodes into heavy paths? We can think of this similarly to finding the lowest common ancestor of the two nodes. We greedily move the lower of the nodes up until the two nodes meet. If we pair the DFS array with a segment tree, we can efficiently query the maximum value during this process. Whenever we make move a node, we will do a segment tree query.

A single query will split the path between two nodes into $O(\log n)$ parts. Each of the $O(\log n)$ segment tree queries takes $O(\log n)$ time. Thus, the total time complexity is $O(n + q \log^2 n)$.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

// This segment tree supports the following operations:
// 1. SegmentTree(v) constructs the tree from the vector v.
// 2. set(k, x) sets the value in position k to x.
// 3. query(l, r) returns the maximum value in range [l, r].
struct SegmentTree {
    int n;
    vector<int> tree;

    SegmentTree(vector<int> &v) {
        n = 1;
        while (n < v.size()) n *= 2;
        tree.resize(n * 2);
        for (int i = 0; i < v.size(); ++i) {
            tree[i + n] = v[i];
        }
        for (int i = n - 1; i > 0; --i) {
            tree[i] = max(tree[i * 2], tree[i * 2 + 1]);
        }
    }

    void set(int k, int x) {
        k += n;
        tree[k] = x;
```

```
        while (k > 1) {
            k /= 2;
            tree[k] = max(tree[k * 2], tree[k * 2 + 1]);
        }
    }

    int query(int l, int r) {
        int ans = 0;
        for (l += n, r += n; l <= r; l /= 2, r /= 2) {
            if (l % 2 == 1) ans = max(ans, tree[l++]);
            if (r % 2 == 0) ans = max(ans, tree[r--]);
        }
        return ans;
    }
};

using graph = vector<vector<int>>;

// Heavy-light decomposition.
// Decomposes the given tree into heavy and light paths.
// Provides the following functions:
// 1. `position(x)`: Returns the position of node x in the DFS order array.
// 2. `subtree(x)`: Returns range in the DFS order array that corresponds to the
//                  subtree of node x.
// 3. `path_query(a, b, f)`: Partitions the path between nodes a and b into
//                           O(log n) contiguous ranges of the DFS order array.
//                           Calls f(l, r) for each such range [l, r].
//                           The function f may, for example, query/update a
//                           segment tree.
struct HLD {
    int n;
    graph &g;
    vector<int> parent;
    vector<int> jump;
    vector<int> total;
    vector<int> order;

    HLD(graph &g) : n(g.size()), g(g), parent(n), jump(n), total(n), order(n) {
        dfs_size(0);
        int order_index = 0;
        dfs_hld(0, order_index);
    }

    template <typename F>
    void path_query(int a, int b, F f) {
        for (;; a = parent[jump[a]]) {
            if (order[a] < order[b]) swap(a, b);
            if (order[jump[a]] <= order[b]) {
                f(order[b], order[a]);
                return;
            }
            f(order[jump[a]], order[a]);
        }
    }

    int position(int x) { return order[x]; }
    pair<int, int> subtree(int x) {
        return {order[x], order[x] + total[x] - 1};
    }

private:
    void dfs_size(int node) {
        total[node] = 1;
        for (auto &child : g[node]) {
            g[child].erase(find(g[child].begin(), g[child].end(), node));
            dfs_size(child);
            total[node] += total[child];
            if (total[child] > total[g[node][0]]) {
                swap(child, g[node][0]);
            }
        }
    }

    void dfs_hld(int node, int &order_index) {
        order[node] = order_index++;
        for (auto child : g[node]) {
            parent[child] = node;
            if (child == g[node][0]) {
                jump[child] = jump[node];
            } else {
                jump[child] = child;
            }
            dfs_hld(child, order_index);
        }
    }
};
```

```
int main() {
    int n, q;
    cin >> n >> q;

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
    }

    graph g(n);
    for (int i = 0; i < n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    HLD hld(g);

    vector<int> initial(n);
    for (int i = 0; i < n; ++i) {
        initial[hld.position(i)] = v[i];
    }
    SegmentTree tree(initial);

    for (int qi = 0; qi < q; ++qi) {
        int t;
        cin >> t;
        if (t == 1) {
            int s, x;
            cin >> s >> x;
            tree.set(hld.position(s - 1), x);
        } else {
            int a, b;
            cin >> a >> b;
            int ans = 0;
            auto update_ans = [&](int l, int r) {
                ans = max(ans, tree.query(l, r));
            };
            hld.path_query(a - 1, b - 1, update_ans);
            cout << ans << '\n';
        }
    }
}
```

Note that the way we implemented HLD, modifying the order of the adjacency lists, allows us to make subtree queries in addition to path queries. Not all implementations support this.

References

- Codeforces, *Easiest HLD with subtree queries* by adamant
- CP-Algorithms, Heavy-light decomposition

2133 · Dynamic Connectivity

Task · source analysis length 3,761 characters

Problem statement

Consider an undirected graph that consists of n nodes and m edges. There are two types of events that can happen:

1. A new edge is created between nodes a and b.
2. An existing edge between nodes a and b is removed.

Your task is to report the number of components after every event.

Input

The first input line has three integers n, m and k: the number of nodes, edges and events.

After this there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. There is at most one edge between any pair of nodes.

Then there are k lines describing the events. Each line has the form "$t\ a\ b$" where t is 1 $createanewedge$ or 2 $removeanedge$. A new edge is always created between two nodes that do not already have an edge between them, and only existing edges can get removed.

Output

Print $k+1$ integers: first the number of components before the first event, and after this the new number of components after each event.

Constraints

- $2 \le n \le 10^5$
- $1 \le m, k \le 10^5$
- $1 \le a, b \le n$

This problem may be solved using the union find structure, since it provides information about the number of components in a given network. By default, union find does not support the removal of edges, but it may be modified to support undoing previous operations. This may be done by keeping a stack of changes made to the structure. The addition of an edge modifies only $O(1)$ array values, and thus any operation may be undone with $O(1)$ work.

It is useful to think of the edges in terms of the intervals $[l, r)$ during which they are *active*, that is, the time range an edge is part of the graph. If an edge belongs to the initial graph, its interval starts at time 0, and if an edge is never removed its time range ends at $r = k + 1$. Our task is to find the number of connected components at single points of time.

To do this, we will create a divide and conquer-style recursive function that always splits the search range in half and processes the halves recursively. During the search, we will keep a list of all edge intervals that have some point in common with our search range, and propagate the list to the subsequent ranges. Whenever an interval contains the whole search range, it will be removed from the list and added to the union find structure. After the recursive calls are finished, we will undo the changes the added edges made to the union find structure.

This process ensures that each edge will be added and removed at most $O(\log k)$ times. Since each operation works in $O(\log n)$ time, the total time complexity of the search is $O(k \log k \log n + m \log n)$.

The technique used here generalizes to other problems as well. It may be used, for example, with Li Chao Trees, allowing deletions of lines.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

// Union find structure with rollback functionality.
struct UnionFind {
    // Stores the number of disjoint trees.
    int n;
    // If p[i] < 0, i is a root node with a tree size of -p[i].
    // Otherwise, p[i] represents the parent of i.
    vector<int> p;
    vector<pair<int *, int>> changes;
    UnionFind(int n) : n(n), p(n, -1) {}
    int root(int s) {
        while (p[s] >= 0) {
            s = p[s];
        }
        return s;
    }
    void merge(int a, int b) {
        a = root(a);
        b = root(b);
        if (a == b) return;
        if (p[a] < p[b]) swap(a, b);
        changes.emplace_back(&p[a], p[a]);
        changes.emplace_back(&p[b], p[b]);
        changes.emplace_back(&n, n);
        n--;
        p[b] += p[a];
        p[a] = b;
    }
    void undo(int cnt) {
        while (cnt--) {
            auto [a, b] = changes.back();
            changes.pop_back();
            *a = b;
        }
    }
};

using Event = tuple<int, int, int, int>;

const int N = 100001;
int ans[N];
UnionFind uf(0);

void answer(int l, int r, vector<Event> edges) {
    int old_size = uf.changes.size();
    int mid = (l + r) / 2;
    vector<Event> left, right;
    for (auto &e : edges) {
        auto [t0, t1, a, b] = e;
        if (t0 <= l && r <= t1) {
            uf.merge(a, b);
        } else {
            if (t0 < mid) {
                left.emplace_back(e);
            }
            if (mid < t1) {
                right.emplace_back(e);
            }
        }
    }
    if (r - l == 1) {
        ans[l] = uf.n;
    } else {
        answer(l, mid, left);
        answer(mid, r, right);
    }
    uf.undo(uf.changes.size() - old_size);
}

int main() {
    int n, m, k;
    cin >> n >> m >> k;

    map<pair<int, int>, int> edge_time;
```

```
    vector<Event> edges;

    for (int i = 0; i < m + k; ++i) {
        int _, a, b;
        if (i >= m) cin >> _;
        cin >> a >> b;
        a--;
        b--;
        if (b < a) swap(a, b);
        int t = max(0, i - (m - 1));
        if (auto it = edge_time.find({a, b}); it != edge_time.end()) {
            edges.emplace_back(it->second, t, a, b);
            edge_time.erase(it);
        } else {
            edge_time.insert(it, {{a, b}, t});
        }
    }

    for (auto [e, t] : edge_time) {
        auto [a, b] = e;
        edges.emplace_back(t, k + 1, a, b);
    }

    uf = UnionFind(n);
    answer(0, k + 1, edges);

    for (int i = 0; i < k + 1; ++i) {
        cout << ans[i] << ' ';
    }
    cout << '\n';
}
```

2110 · Substring Distribution

Task · source analysis length 3,990 characters

Problem statement

You are given a string of length n. For every integer between $1 \ldots n$ you need to print the number of distinct substrings of that length.

Input

The only input line has a string of length n that consists of characters a–z.

Output

For each integer between $1 \ldots n$ print the number of distinct substrings of that length.

Constraints

- $1 \le n \le 10^5$

Solution 1: suffix array

We go through the suffix array and determine, for each position i, the lengths of prefixes that should be counted.

Since we want to count each distinct substring only once, the minimum prefix length is $\mathrm{lcp}[i] + 1$. The maximum prefix length is $n - \mathrm{sa}[i]$, which is the length of the suffix. Thus, we always have a contiguous range of lengths.

We first construct a vector `diff` that contains the differences between consecutive lengths. By going through that vector, we can efficiently determine the distribution of lengths.

The following code creates the suffix array in $O(n \log^2 n)$ time and the LCP array in $O(n)$ time. Then, it calculates the distribution in $O(n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;
    vector<int> lcp;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n), lcp(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
```

```
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            inv[i]--;
            sa[inv[i]] = i;
        }
        for (int i = 0; i < n - 1; i++) {
            int k = inv[i];
            if (i) lcp[k] = max(0, lcp[inv[i - 1]] - 1);
            while (s[sa[k - 1] + lcp[k]] == s[sa[k] + lcp[k]]) lcp[k]++;
        }
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    SuffixArray sa(s);

    vector<int> diff(n + 2);
    for (int i = 1; i <= n; i++) {
        diff[sa.lcp[i] + 1]++;
        diff[n - sa.sa[i] + 1]--;
    }
    ll count = 0;
    for (int i = 1; i <= n; i++) {
        count += diff[i];
        cout << count << " ";
    }
    cout << "\n";
}
```

Solution 2: suffix automaton

Each suffix automaton state corresponds to a set of substrings whose lengths form a contiguous range.

The field `len` contains the maximum length of a substring in a state, and all other substrings are suffixes of that substring. Moreover, the field `link` points to a previous state that contains smaller suffixes of the substring.

Using the fields `len` and `link`, we can construct a vector `diff` like in the previous solution, and then construct the distribution of lengths.

The construction of the suffix automaton takes $O(n)$ time, and calculating the distribution also takes $O(n)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        bool seen;
    };

    vector<int> diff;

    State *first, *last;

    SuffixAutomaton(string s) : diff(s.size() + 2) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
```

```
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->seen) return;
        cur->seen = true;
        if (cur != first) {
            diff[cur->len + 1]--;
            diff[cur->link->len + 1]++;
        }
        for (auto [c, p] : cur->next) {
            dfs(p);
        }
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    SuffixAutomaton sa(s);

    ll count = 0;
    for (int i = 1; i <= n; i++) {
        count += sa.diff[i];
        cout << count << " ";
    }
    cout << "\n";
}
```

References

- Suffix array *Wikipedia*
- LCP array *Wikipedia*
- Suffix automaton *Wikipedia*

2109 · Substring Order II

Task · source analysis length 4,918 characters

Problem statement

You are given a string of length n. If all of its substrings $not necessarily distinct$ are ordered lexicographically, what is the kth smallest of them?

Input

The first input line has a string of length n that consists of characters a–z.

The second input line has an integer k.

Output

Print the kth smallest substring in lexicographical order.

Constraints

- $1 \le n \le 10^5$
- $1 \le k \le \frac{n(n+1)}{2}$

Solution 1: suffix array

We create two additional data structures:

- `sums` is a prefix sum array, using which we can calculate the sum of the lengths of suffixes in a suffix array range
- `lcps` is a set of all LCP values, using which we can find the next suffix array position that has a given LCP value

We contruct the substring step by step using the data structures, and maintain the corresponding suffix array range. When we have created a substring of length l, we divide the range into subranges, each of which has suffixes with a fixed character at position l. We create the subranges by finding positions where the LCP value is l. Then, we can determine the next character by calculating the total length of suffixes in each range.

Creating the suffix array takes $O(n \log^2 n)$ time and creating the LCP-array takes $O(n)$ time. After that, we can calculate the answer in $O(n \log n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <set>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;
    vector<int> lcp;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n), lcp(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
```

```
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            inv[i]--;
            sa[inv[i]] = i;
        }
        for (int i = 0; i < n - 1; i++) {
            int k = inv[i];
            if (i) lcp[k] = max(0, lcp[inv[i - 1]] - 1);
            while (s[sa[k - 1] + lcp[k]] == s[sa[k] + lcp[k]]) lcp[k]++;
        }
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    ll k;
    cin >> k;

    SuffixArray sa(s);

    vector<ll> sums(n + 1);
    for (int i = 1; i <= n; i++) {
        sums[i] = sums[i - 1] + n - sa.sa[i];
    }

    set<pair<int, int>> lcps;
    for (int i = 1; i <= n; i++) {
        lcps.emplace(sa.lcp[i], i);
    }
    lcps.emplace(0, n + 1);

    int left = 1;
    int right = n + 1;
    int len = 0;
    while (k > 0) {
        int end = right;
        while (true) {
            auto it = lcps.upper_bound({len, left});
            if (it != lcps.end() && it->first == len && it->second < end) {
                right = it->second;
            } else {
                right = end;
            }
            ll count = sums[right - 1] - sums[left - 1];
            count -= len * (right - left);
            if (k > count) {
                k -= count;
                left = right;
            } else {
                cout << s[sa.sa[left] + len];
                k -= right - left;
                len++;
                break;
            }
        }
    }
    cout << "\n";
}
```

Solution 2: suffix automaton

We calculate two values for each automaton state: `count` denotes the total number of $not necessarily distinct$ substrings that begin at the state, and `to_final` denotes the number of paths from the state to a final state $i.e., the number of suffixes that begin at the state$.

We construct the substring step by step and we always determine the next character by going through the characters in alphabetical order and checking the `count` values. Since we consider all substrings, the `to_final` values correspond to the number of substrings after adding a character.

We can both construct the automaton and generate the substring in $O(n)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        bool final;
        ll count;
        ll to_final;
    };

    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        State *cur = last;
        while (cur->len) {
            cur->final = true;
            cur = cur->link;
        }
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->count) return;
        if (cur->final) {
            cur->count = cur->to_final = 1;
        }
        for (auto [c, p] : cur->next) {
            dfs(p);
            cur->count += p->count + p->to_final;
            cur->to_final += p->to_final;
        }
    }
};

int main() {
    string s;
    cin >> s;
    ll k;
    cin >> k;

    SuffixAutomaton sa(s);
    auto cur = sa.first;
    while (k > 0) {
        for (auto [c, p] : cur->next) {
            if (p->count < k) {
                k -= p->count;
```

```
            } else {
                cout << c;
                k -= p->to_final;
                cur = p;
                break;
            }
        }
    }
    cout << "\n";
}
```

References

- Suffix array $Wikipedia$
- LCP array $Wikipedia$
- Suffix automaton $Wikipedia$

2108 · Substring Order I

Task · source analysis length 3,875 characters

Problem statement

You are given a string of length n. If all of its distinct substrings are ordered lexicographically, what is the kth smallest of them?

Input

The first input line has a string of length n that consists of characters a–z.

The second input line has an integer k.

Output

Print the kth smallest distinct substring in lexicographical order.

Constraints

- $1 \le n \le 10^5$
- $1 \le k \le \frac{n(n+1)}{2}$
- It is guaranteed that k does not exceed the number of distinct substrings.

Solution 1: suffix array

We go through the suffix array and calculate the number of distinct substrings each suffix produces. This can be done using the formula

$$n - \mathrm{sa}[i$$

- \textrm{lcp}

$$i$$

]

where $n - \mathrm{sa}[i]$ is the length of the suffix and $\mathrm{lcp}[i]$ gives the number of prefixes that have already been counted.

We first locate the suffix that contains the kth smallest substring and then print the prefix that has the correct length.

Creating the suffix array takes $O(n \log^2 n)$ time and creating the LCP-array takes $O(n)$ time. After that, we can calculate the answer in $O(n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixArray {
    string s;
```

```
    int n;
    vector<int> sa;
    vector<int> lcp;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n), lcp(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            inv[i]--;
            sa[inv[i]] = i;
        }
        for (int i = 0; i < n - 1; i++) {
            int k = inv[i];
            if (i) lcp[k] = max(0, lcp[inv[i - 1]] - 1);
            while (s[sa[k - 1] + lcp[k]] == s[sa[k] + lcp[k]]) lcp[k]++;
        }
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    ll k;
    cin >> k;

    SuffixArray sa(s);
    for (int i = 1; i <= n; i++) {
        int add = n - sa.sa[i] - sa.lcp[i];
        if (k > add) {
            k -= add;
        } else {
            cout << s.substr(sa.sa[i], sa.lcp[i] + k) << "\n";
            break;
        }
    }
}
```

Solution 2: suffix automaton

We calculate a value `count` for each automaton state: the number of paths that begin at that state and end at some state. After that, we generate the kth smallest substring step by step.

Suppose we are in some automaton state and we want to choose the next character. We can go through the characters in alphabetical order and check, for each character, the `count` value of the next state which corresponds to the number of substrings that begin at that character. This allows us to determine the next character in the substring and continue in the same way.

We can both construct the automaton and generate the substring in $O(n)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        ll count;
    };
```

```
    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->count) return;
        cur->count = 1;
        for (auto [c, p] : cur->next) {
            dfs(p);
            cur->count += p->count;
        }
    }
};

int main() {
    string s;
    cin >> s;
    ll k;
    cin >> k;

    SuffixAutomaton sa(s);
    auto cur = sa.first;
    while (k) {
        for (auto [c, p] : cur->next) {
            if (p->count < k) {
                k -= p->count;
            } else {
                cout << c;
                k--;
                cur = p;
                break;
            }
        }
    }
    cout << "\n";
}
```

References

- Suffix array *Wikipedia*
- LCP array *Wikipedia*
- Suffix automaton *Wikipedia*

2107 · String Functions

Task · source analysis length 1,438 characters

Problem statement

We consider a string of n characters, indexed $1, 2, \ldots, n$. Your task is to calculate all values of the following functions:

- $z(i)$ denotes the maximum length of a substring that begins at position i and is a prefix of the string. In addition, $z(1$=0).
- $\pi(i)$ denotes the maximum length of a substring that ends at position i, is a prefix of the string, and whose length is at most $i - 1$.

Note that the function z is used in the Z-algorithm, and the function π is used in the KMP algorithm.

Input

The only input line has a string of length n. Each character is between a–z.

Output

Print two lines: first the values of the z function, and then the values of the π function.

Constraints

- $1 \le n \le 10^6$

We first calculate the z values using the Z-algorithm. After that, we calculate the π values using the z values.

Observation 1: If $z(i$>0), then $\pi(i + z(i$-1) \ge zi), because a prefix of length $z(i)$ ends at position $i + z(i$-1).

Observation 2: $\pi(i$ \ge \pi$i + 1$-1), because if a prefix of length $\pi(i + 1)$ ends at position $i + 1$, then also a prefix of length $\pi(i + 1$-1) ends at position i.

The following code calculates the π values in two phases. First we calculate the "peak" values of π using Observation 1, and after that the remaining values using Observation 2. Note that 0-indexing is used in the code.

The code works in $O(n)$ time, because the Z-algorithm works in $O(n)$ time and the π values are calculated using two linear loops.

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> z_algo(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        z[i] = max(0, min(z[i - l], r - i));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

```
int main() {
    string s;
    cin >> s;
    int n = s.size();

    vector<int> z = z_algo(s);

    vector<int> p(n);
    for (int i = n - 1; i > 0; i--) {
        p[i + z[i] - 1] = z[i];
    }
    for (int i = n - 2; i > 0; i--) {
        p[i] = max(p[i], p[i + 1] - 1);
    }

    for (auto x : z) {
        cout << x << " ";
    }
    cout << "\n";

    for (auto x : p) {
        cout << x << " ";
    }
    cout << "\n";
}
```

2106 · Repeating Substring

Task · source analysis length 6,059 characters

Problem statement

A repeating substring is a substring that occurs in two $ormore$ locations in the string. Your task is to find the longest repeating substring in a given string.

Input

The only input line has a string of length n that consists of characters a–z.

Output

Print the longest repeating substring. If there are several possibilities, you can print any of them. If there is no repeating substring, print -1.

Constraints

- $1 \le n \le 10^5$

Solution 1: suffix array

We create an additional data structure for the suffix array: a LCP-array that contains, for each suffix array position $i > 0$, the length of the longest common prefix of suffixes whose positions in the suffix array are $i-1$ and i.

The maximum length of a repeating pattern equals the maximum value in the LCP-array. We can find the maximum value and the corresponding pattern by going through the LCP-array.

Creating the suffix array takes $O(n \log^2 n)$ time and creating the LCP-array takes $O(n)$ time. After that, we can calculate the answer in $O(n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;
    vector<int> lcp;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n), lcp(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
```

```
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            inv[i]--;
            sa[inv[i]] = i;
        }
        for (int i = 0; i < n - 1; i++) {
            int k = inv[i];
            if (i) lcp[k] = max(0, lcp[inv[i - 1]] - 1);
            while (s[sa[k - 1] + lcp[k]] == s[sa[k] + lcp[k]]) lcp[k]++;
        }
    }
};

int main() {
    string s;
    cin >> s;
    SuffixArray sa(s);

    int n = s.size();
    int len = 0;
    int pos = 0;
    for (int i = 1; i <= n; i++) {
        if (sa.lcp[i] > len) {
            len = sa.lcp[i];
            pos = sa.sa[i];
        }
    }
    if (len == 0) {
        cout << "-1\n";
    } else {
        cout << s.substr(pos, len) << "\n";
    }
}
```

Solution 2: suffix automaton

We calculate two values for each automaton state: `count` denotes the number of paths that begin at the state and end at a final state, and `max` denotes the maximum length of a path that begins at the state and ends at a state whose `count` is at least two.

After calculating the values, the `max` value of the start state equals the length of the longest repeating pattern. We can find the pattern step by step by always moving to a next state whose `max` value is one smaller than the `max` value of the current state.

We can construct the automaton, calculate the values and find the pattern in $O(n)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        bool final;
        int count;
        int max;
    };

    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
```

```
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        State *cur = last;
        while (cur->len) {
            cur->final = true;
            cur = cur->link;
        }
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->count) return;
        if (cur->final) cur->count = 1;
        for (auto [c, p] : cur->next) {
            dfs(p);
            cur->count += p->count;
            if (p->count > 1) {
                cur->max = max(cur->max, p->max + 1);
            }
        }
    }
};

int main() {
    string s;
    cin >> s;
    SuffixAutomaton sa(s);

    auto cur = sa.first;
    if (cur->max == 0) {
        cout << "-1\n";
    } else {
        while (cur->max) {
            for (auto [c, p] : cur->next) {
                if (p->max == cur->max - 1) {
                    cout << c;
                    cur = p;
                    break;
                }
            }
        }
        cout << "\n";
    }
}
```

Solution 3: binary search and hashing

Here is yet another solution that finds the longest repeating pattern length using binary search. To check if a length k is valid, we create a set of all hash values of substrings of length k. If the set contains the same hash value two times, then we have found a repeating pattern.

In this problem a 32-bit hash value is not safe enough because we actually compare hash values of *all* pairs of substrings of length k and there could easily be collisions $thebirthdayparadoxproblem$. For this reason, we work with 64-bit hash values.

The algorithm works in $O(n \log^2 n)$ time because it checks $O(\log n)$ pattern lengths and each check takes $O(n \log n)$ time. Note that this solution is much slower than the two previous solutions and it barely fits in the time limit.

```
#include <chrono>
#include <iostream>
#include <random>
#include <set>
#include <vector>
using namespace std;
using ll = long long;
```

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct HashString {
    const ll M = 1111111111111111111;  // M is a prime.
    const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9 * M)(rng);

    ll mul(ll a, ll b) { return __int128(a) * b % M; }

    int n;
    vector<ll> pows, sums;

    HashString(string s) : n(s.size()), pows(n + 1, 1), sums(n + 1) {
        for (int i = 1; i <= n; i++) {
            pows[i] = mul(pows[i - 1], C);
            sums[i] = (mul(sums[i - 1], C) + s[i - 1]) % M;
        }
    }

    // Returns the hash of the substring [l, r)
    ll hash(int l, int r) {
        ll h = sums[r] - mul(sums[l], pows[r - l]);
        return (h % M + M) % M;
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    HashString hs(s);

    auto find = [&](int len) {
        set<ll> hashes;
        for (int i = 0; i <= n - len; i++) {
            ll h = hs.hash(i, i + len);
            if (hashes.count(h)) return i;
            hashes.insert(h);
        }
        return -1;
    };

    int len = 0;
    int first = 0;
    for (int b = 1 << 16; b >= 1; b /= 2) {
        int pos = find(len + b);
        if (pos == -1) continue;
        first = pos;
        len += b;
    }

    if (len == 0) {
        cout << "-1\n";
    } else {
        cout << s.substr(first, len) << "\n";
    }
}
```

References

- Suffix array *Wikipedia*
- LCP array *Wikipedia*
- Suffix automaton *Wikipedia*

2105 · Distinct Substrings

Task · source analysis length 3,605 characters

Problem statement

Count the number of distinct substrings that appear in a string.

Input

The only input line has a string of length n that consists of characters a–z.

Output

Print one integer: the number of substrings.

Constraints

- $1 \le n \le 10^5$

Solution 1: suffix array

We create an additional data structure for the suffix array: the LCP-array. It contains, for each suffix array position $i > 0$, the length of the longest common prefix of suffixes whose positions in the suffix array are $i-1$ and i.

Let $\mathrm{lcp}[i]$ denote the LCP value at position i. The answer to the problem is

$$\frac{n(n+1)}{2} - \sum_{i=1}^{n} \mathrm{lcp}[i$$

]

where we first count the number of all substrings and then subtract the number of repeating substrings. Note that the length of the LCP-array is $n+1$ because we add $ to the string before creating the arrays.

Creating the suffix array takes $O(n \log^2 n)$ time and creating the LCP-array takes $O(n)$ time. After that, we can calculate the answer in $O(n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;
    vector<int> lcp;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n), lcp(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
```

```
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            inv[i]--;
            sa[inv[i]] = i;
        }
        for (int i = 0; i < n - 1; i++) {
            int k = inv[i];
            if (i) lcp[k] = max(0, lcp[inv[i - 1]] - 1);
            while (s[sa[k - 1] + lcp[k]] == s[sa[k] + lcp[k]]) lcp[k]++;
        }
    }
};

int main() {
    string s;
    cin >> s;
    SuffixArray sa(s);

    int n = s.size();
    ll ans = (ll)n * (n + 1) / 2;
    for (int i = 1; i <= n; i++) {
        ans -= sa.lcp[i];
    }
    cout << ans << "\n";
}
```

Solution 2: suffix automaton

We calculate a value `count` for each automaton state: the number of paths that begin at that state and end at some state. Since each path corresponds to a distinct string, the answer to the problem is the number of such paths that begin at the first state of the automaton.

Note that we should not count the empty string, so we decrease the number of strings by one before printing the result.

We can both construct the automaton and count the paths in $O(n)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        ll count;
    };

    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
```

```
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->count) return;
        cur->count = 1;
        for (auto [c, p] : cur->next) {
            dfs(p);
            cur->count += p->count;
        }
    }
};

int main() {
    string s;
    cin >> s;
    SuffixAutomaton sa(s);
    cout << sa.first->count - 1 << "\n";
}
```

References

- Suffix array $Wikipedia$
- LCP array $Wikipedia$
- Suffix automaton $Wikipedia$

2104 · Pattern Positions

Task · source analysis length 4,529 characters

Problem statement

Given a string and patterns, find for each pattern the first position $1-indexed$ where it appears in the string.

Input

The first input line has a string of length n.

The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns.

The string and the patterns consist of characters a–z.

Output

Print the first position for each pattern (or -1 if it does not appear at all).

Constraints

- $1 \le n \le 10^5$
- $1 \le k \le 5 \cdot 10^5$
- the total length of the patterns is at most $5 \cdot 10^5$

Solution 1: suffix array

To find the first pattern position, we should find the minimum value in the suffix array range that corresponds to the pattern. We can do that by creating a segment tree for the suffix array.

The following code creates the suffix array in $O(n \log^2 n)$ time and the segment tree in $O(n)$ time. For each query, we first find the suffix array range in $O(m \log n)$ time and then the minimum position in $O(\log n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
```

```
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            sa[inv[i] - 1] = i;
        }
    }

    // Returns the (half-open) range where pattern p appears in the suffix array
    pair<int, int> find(string p) {
        int m = p.size();
        auto left = sa.begin();
        auto right = sa.end();
        for (int i = 0; i < m; i++) {
            auto cmp = [&](int e, int v) {
                return s[e + i] < v;
            };
            auto new_left = lower_bound(left, right, p[i], cmp);
            auto new_right = lower_bound(left, right, p[i] + 1, cmp);
            left = new_left;
            right = new_right;
        }
        return {left - sa.begin(), right - sa.begin()};
    }
};

struct MinTree {
    int n;
    vector<int> tree;

    MinTree(vector<int> v) {
        n = 1;
        while (n < v.size()) n *= 2;
        tree.resize(2 * n);
        for (int i = 0; i < v.size(); i++) {
            tree[n + i] = v[i];
        }
        for (int i = n - 1; i >= 1; i--) {
            tree[i] = min(tree[2 * i], tree[2 * i + 1]);
        }
    }

    int find(int l, int r) {
        l += n;
        r += n;
        int ans = tree[l];
        while (l <= r) {
            if (l % 2 == 1) ans = min(ans, tree[l++]);
            if (r % 2 == 0) ans = min(ans, tree[r--]);
            l /= 2;
            r /= 2;
        }
        return ans;
    }
};

int main() {
    string s;
    cin >> s;
    SuffixArray sa(s);
    MinTree min(sa.sa);

    int k;
    cin >> k;
    for (int ki = 1; ki <= k; ki++) {
        string p;
        cin >> p;
        auto [l, r] = sa.find(p);
        if (l == r) {
            cout << "-1\n";
        } else {
            cout << min.find(l, r - 1) + 1 << "\n";
        }
    }
}
```

Solution 2: suffix automaton

We calculate a value pos for each automaton state: the minimum position of a suffix that begins at that state. The position of the last state is n, and we calculate the values for other states using depth-first search.

After finding the state of a pattern, the first position where the pattern appears can be calculated using the formula $x - m$ where x is the position of the state and m is the length of the pattern.

The construction of the suffix automaton takes $O(n)$ time and processing each pattern takes $O(m)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        bool final;
        int pos;
    };

    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        State *cur = last;
        while (cur->len) {
            cur->final = true;
            cur = cur->link;
        }
        last->pos = s.size();
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->pos) return;
        cur->pos = last->pos;
        for (auto [c, p] : cur->next) {
            dfs(p);
            cur->pos = min(cur->pos, p->pos - 1);
        }
    }

    int pos(string p) {
        State *cur = first;
        for (auto c : p) {
            if (!cur->next[c]) return -1;
            cur = cur->next[c];
        }
        return cur->pos - p.size();
    }
};

int main() {
    string s;
    cin >> s;
    SuffixAutomaton sa(s);
```

```
    int k;
    cin >> k;
    for (int ki = 1; ki <= k; ki++) {
        string p;
        cin >> p;
        int pos = sa.pos(p);
        cout << (pos != -1 ? pos + 1 : -1) << "\n";
    }
}
```

References

- Suffix array $Wikipedia$
- Suffix automaton $Wikipedia$

2103 · Counting Patterns

Task · source analysis length 3,772 characters

Problem statement

Given a string and patterns, count for each pattern the number of positions where it appears in the string.

Input

The first input line has a string of length n.

The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns.

The string and the patterns consist of characters a–z.

Output

For each pattern, print the number of positions.

Constraints

- $1 \le n \le 10^5$
- $1 \le k \le 5 \cdot 10^5$
- the total length of the patterns is at most $5 \cdot 10^5$

Solution 1: suffix array

We create a suffix array for the string, after which we can efficiently count the number of pattern occurrences in the string.

The following code creates the suffix array using the prefix doubling method and works in $O(n \log^2 n)$ time. We use binary search to find the range of a pattern in the suffix array in $O(m \log n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
            }
            sort(order.begin(), order.end(),
```

```
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            sa[inv[i] - 1] = i;
        }
    }

    // Returns the (half-open) range where pattern p appears in the suffix array
    pair<int, int> find(string p) {
        int m = p.size();
        auto left = sa.begin();
        auto right = sa.end();
        for (int i = 0; i < m; i++) {
            auto cmp = [&](int e, int v) {
                return s[e + i] < v;
            };
            auto new_left = lower_bound(left, right, p[i], cmp);
            auto new_right = lower_bound(left, right, p[i] + 1, cmp);
            left = new_left;
            right = new_right;
        }
        return {left - sa.begin(), right - sa.begin()};
    }
};

int main() {
    string s;
    cin >> s;
    SuffixArray sa(s);

    int k;
    cin >> k;
    for (int ki = 1; ki <= k; ki++) {
        string p;
        cin >> p;
        auto [l, r] = sa.find(p);
        cout << r - l << "\n";
    }
}
```

Solution 2: suffix automaton

We can also solve the problem using a suffix automaton that stores all suffixes of a string in compact form.

The following code first creates the suffix automaton and then uses depth-first search to calculate a value `count` for each state: the number of ways to reach a final state when we begin at that state. This corresponds to the number of occurrences of a pattern in the string.

The construction of the suffix automaton takes $O(n)$ time and processing each pattern takes $O(m)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
        bool final;
        int count;
    };

    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
```

```
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
        State *cur = last;
        while (cur->len) {
            cur->final = true;
            cur = cur->link;
        }
        dfs(first);
    }

    void dfs(State *cur) {
        if (cur->count) return;
        if (cur->final) cur->count = 1;
        for (auto [c, p] : cur->next) {
            dfs(p);
            cur->count += p->count;
        }
    }

    int count(string p) {
        State *cur = first;
        for (auto c : p) {
            if (!cur->next[c]) return 0;
            cur = cur->next[c];
        }
        return cur->count;
    }
};

int main() {
    string s;
    cin >> s;
    SuffixAutomaton sa(s);

    int k;
    cin >> k;
    for (int ki = 1; ki <= k; ki++) {
        string p;
        cin >> p;
        cout << sa.count(p) << "\n";
    }
}
```

References

- Suffix array *Wikipedia*
- Suffix automaton *Wikipedia*

2102 · Finding Patterns

Task · source analysis length 3,380 characters

Problem statement

Given a string and patterns, check for each pattern if it appears in the string.

Input

The first input line has a string of length n.

The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns.

The string and the patterns consist of characters a–z.

Output

For each pattern, print "YES" if it appears in the string and "NO" otherwise.

Constraints

- $1 \le n \le 10^5$
- $1 \le k \le 5 \cdot 10^5$
- the total length of the patterns is at most $5 \cdot 10^5$

Solution 1: suffix array

We create a suffix array for the string, after which we can efficiently check whether a pattern appears in the string.

The following code creates the suffix array using the prefix doubling method and works in $O(n \log^2 n)$ time. We use binary search to find the range of a pattern in the suffix array in $O(m \log n)$ time.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;

struct SuffixArray {
    string s;
    int n;
    vector<int> sa;

    SuffixArray(string x) : s(x + "$"), n(s.size()), sa(n) {
        int bound = 1;
        while (bound < n) bound *= 2;
        vector<int> inv(2 * bound);
        copy(s.begin(), s.end(), inv.begin());
        vector<int> order(n);
        iota(order.begin(), order.end(), 0);
        for (int add = 1; add < bound; add *= 2) {
            vector<pair<int, int>> pairs(n);
            for (int i = 0; i < n; i++) {
                pairs[i] = {inv[i], inv[i + add]};
            }
            sort(order.begin(), order.end(),
                 [&](int a, int b) { return pairs[a] < pairs[b]; });
```

```
            int label = 1;
            for (int i = 0; i < n; i++) {
                label += i && pairs[order[i - 1]] != pairs[order[i]];
                inv[order[i]] = label;
            }
        }
        for (int i = 0; i < n; i++) {
            sa[inv[i] - 1] = i;
        }
    }

    // Returns the (half-open) range where pattern p appears in the suffix array
    pair<int, int> find(string p) {
        int m = p.size();
        auto left = sa.begin();
        auto right = sa.end();
        for (int i = 0; i < m; i++) {
            auto cmp = [&](int e, int v) {
                return s[e + i] < v;
            };
            auto new_left = lower_bound(left, right, p[i], cmp);
            auto new_right = lower_bound(left, right, p[i] + 1, cmp);
            left = new_left;
            right = new_right;
        }
        return {left - sa.begin(), right - sa.begin()};
    }
};

int main() {
    string s;
    cin >> s;
    SuffixArray sa(s);

    int k;
    cin >> k;
    for (int ki = 1; ki <= k; ki++) {
        string p;
        cin >> p;
        auto [l, r] = sa.find(p);
        cout << (r > l ? "YES" : "NO") << "\n";
    }
}
```

Solution 2: suffix automaton

We can also solve the problem using a suffix automaton that stores all suffixes of a string in compact form. The following code first creates the suffix automaton, after which we can efficiently check whether a pattern appears in the string.

The construction of the suffix automaton takes $O(n)$ time and processing each pattern takes $O(m)$ time.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int len;
        State *link;
        map<char, State *> next;
    };

    State *first, *last;

    SuffixAutomaton(string s) {
        first = last = new State{};
        for (auto c : s) {
            auto *add = new State{};
            add->len = last->len + 1;
            add->link = first;
            State *cur = last;
            while (cur && !cur->next[c]) {
                cur->next[c] = add;
                cur = cur->link;
            }
            if (cur && cur->next[c] != add) {
                State *mid = cur->next[c];
                if (cur->len + 1 == mid->len) {
                    add->link = mid;
```

```
                } else {
                    auto *copy = new State(*mid);
                    copy->len = cur->len + 1;
                    add->link = mid->link = copy;
                    while (cur && cur->next[c] == mid) {
                        cur->next[c] = copy;
                        cur = cur->link;
                    }
                }
            }
            last = add;
        }
    }

    bool check(string p) {
        State *cur = first;
        for (auto c : p) {
            if (!cur->next[c]) return false;
            cur = cur->next[c];
        }
        return true;
    }
};

int main() {
    string s;
    cin >> s;
    SuffixAutomaton sa(s);

    int k;
    cin >> k;
    for (int ki = 1; ki <= k; ki++) {
        string p;
        cin >> p;
        cout << (sa.check(p) ? "YES" : "NO") << "\n";
    }
}
```

References

- Suffix array *Wikipedia*
- Suffix automaton *Wikipedia*

2101 · New Roads Queries

Task · source analysis length 8,070 characters

Problem statement

There are n cities in Byteland but no roads between them. However, each day, a new road will be built. There will be a total of m roads.

Your task is to process q queries of the form: "after how many days can we travel from city a to city b for the first time?"

Input

The first input line has three integers n, m and q: the number of cities, roads and queries. The cities are numbered $1, 2, \ldots, n$.

After this, there are m lines that describe the roads in the order they are built. Each line has two integers a and b: there will be a road between cities a and b.

Finally, there are q lines that describe the queries. Each line has two integers a and b: we want to travel from city a to city b.

Output

For each query, print the number of days, or -1 if it is never possible.

Constraints

- $1 \le n, m, q \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Solution 1: Parallel Binary Search

This problem may be solved efficiently with a technique called *parallel binary search*. This means that we binary search the answers to all queries simultaneously.

Were we to use binary search for answering each query separately, we would have to construct a union find structure for checking each midpoint during each search. The time complexity of constructing a union find structure is $O(m \log n)$, making the total time complexity of this solution $O(qm \log^2 n)$, which is too slow.

The key idea is that we need only $O(\log m)$ of such structures to answer all queries. We will start by assigning each query a range $[l, r]$ which means that the cities will become connected at earliest on day l and at latest on day r. Initially the range will be $[0, m]$.

Then, we will simulate the construction of the roads $\log m$ times. Just like with regular binary search, we will calculate the midpoint of each range, and check if the cities at connected at that time. Depending on the result, we will change the lower or upper bound of each range to the midpoint, effectively halving each range.

Adding m roads to the network takes $O(m \log n)$ time and updating the ranges $O(q \log n)$ time, making the total time complexity $O(\log m \cdot (m + q$ \log n)).

```
#include <iostream>
#include <vector>
using namespace std;
```

```
struct UnionFind {
    int n;
    // If p[i] < 0, i is a root node with a tree size of -p[i].
    // Otherwise, p[i] represents the parent of i.
    vector<int> p;
    UnionFind(int n) : n(n), p(n, -1) {}
    int root(int s) {
        while (p[s] >= 0) {
            s = p[s];
        }
        return s;
    }
    void merge(int a, int b) {
        a = root(a);
        b = root(b);
        if (a == b) return;
        if (p[a] < p[b]) swap(a, b);
        p[b] += p[a];
        p[a] = b;
    }
};

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    UnionFind graph(n);
    vector<pair<int, int>> edges(m);
    for (auto &[a, b] : edges) {
        cin >> a >> b;
        a--;
        b--;
        graph.merge(a, b);
    }

    vector<int> qa(q), qb(q), ql(q), qr(q);
    for (int i = 0; i < q; ++i) {
        int &a = qa[i], &b = qb[i];
        cin >> a >> b;
        a--;
        b--;
        ql[i] = 0;
        qr[i] = m;
    }

    int log2 = __lg(m + 1);

    for (int t = 0; t < log2 + 1; ++t) {
        vector<vector<int>> queries(m + 1);
        for (int i = 0; i < q; ++i) {
            if (ql[i] < qr[i]) {
                queries[(ql[i] + qr[i]) / 2].push_back(i);
            }
        }
        UnionFind uf(n);
        for (int i = 0; i < m; ++i) {
            for (int qi : queries[i]) {
                if (uf.root(qa[qi]) == uf.root(qb[qi])) {
                    qr[qi] = i;
                } else {
                    ql[qi] = i + 1;
                }
            }
            uf.merge(edges[i].first, edges[i].second);
        }
    }

    for (int i = 0; i < q; ++i) {
        if (graph.root(qa[i]) != graph.root(qb[i])) {
            cout << -1 << '\n';
        } else {
            cout << ql[i] << '\n';
        }
    }
}
```

Solution 2: Binary Lifting

Another approach, which is able to answer the queries online, is to use binary lifting.

First, we will have to examine the given graph. Note it may not be connected. If two nodes are in different components, they will never be reachable. Thus, we may only focus on a single component. If we think of the time an edge is added to the graph as the weight of the edge, we may find a minimum spanning tree of each component. The maximum edge weight on a path between two nodes in the spanning tree corresponds to the first time the nodes will be reachable. The construction of the spanning trees is easily done with Kruskal's algorithm as the edges are already sorted by their weights.

Now, we may use binary lifting to find the maximum weight edge on a path between any two nodes. We will proceed similarly to the binary lifting LCA algorithm, except that for each jump, we will also store the maximum weight edge on the path corresponding to the jump.

```
#include <iostream>
#include <vector>
using namespace std;

struct UnionFind {
    int n;
    // If p[i] < 0, i is a root node with a tree size of -p[i].
    // Otherwise, p[i] represents the parent of i.
    vector<int> p;
    UnionFind(int n) : n(n), p(n, -1) {}
    int root(int s) {
        while (p[s] >= 0) {
            s = p[s];
        }
        return s;
    }
    void merge(int a, int b) {
        a = root(a);
        b = root(b);
        if (a == b) return;
        if (p[a] < p[b]) swap(a, b);
        p[b] += p[a];
        p[a] = b;
    }
};

const int N = 200000;
vector<pair<int, int>> g[N];
int jmp[N][20], jmp_time[N][20];
int depth[N], component[N];
bool visited[N];

void dfs(int node) {
    visited[node] = true;
    for (auto [nxt, t] : g[node]) {
        if (!visited[nxt]) {
            component[nxt] = component[node];
            depth[nxt] = depth[node] + 1;
            jmp[nxt][0] = node;
            jmp_time[nxt][0] = t;
            dfs(nxt);
        }
    }
}

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    UnionFind uf(n);
    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        if (uf.root(a) != uf.root(b)) {
            uf.merge(a, b);
            g[a].emplace_back(b, i);
            g[b].emplace_back(a, i);
        }
    }

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            depth[i] = 0;
            component[i] = i;
            jmp[i][0] = i;
            dfs(i);
        }
    }
```

```
    for (int j = 0; j < 19; ++j) {
        for (int i = 0; i < n; ++i) {
            jmp[i][j + 1] = jmp[jmp[i][j]][j];
            jmp_time[i][j + 1] = max(jmp_time[i][j], jmp_time[jmp[i][j]][j]);
        }
    }

    auto answer = [&](int a, int b) {
        int r = 0;
        if (depth[a] < depth[b]) {
            swap(a, b);
        }
        for (int j = 19; j >= 0; --j) {
            if (depth[jmp[a][j]] >= depth[b]) {
                r = max(r, jmp_time[a][j]);
                a = jmp[a][j];
            }
        }
        if (a == b) {
            return r;
        }
        for (int j = 19; j >= 0; --j) {
            if (jmp[a][j] != jmp[b][j]) {
                r = max(r, jmp_time[a][j]);
                r = max(r, jmp_time[b][j]);
                a = jmp[a][j];
                b = jmp[b][j];
            }
        }
        r = max(r, jmp_time[a][0]);
        r = max(r, jmp_time[b][0]);
        return r;
    };

    for (int i = 0; i < q; ++i) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        if (component[a] != component[b]) {
            cout << -1 << '\n';
        } else {
            cout << answer(a, b) << '\n';
        }
    }
}
```

Solution 3: Union Find Merge

An even simpler offline-approach is to use the union find structure for answering the queries directly.

Let us store a list of query indices in each node. We will store the index of each query in the lists of both of the query nodes. After this, we will add each edge to the structure, one by one. Before each addition, we will check if the lists of the merged trees contain any equal indices. If an index is contained in the lists of both of the disjoint trees, the added edge will be the first one after which the cities of the corresponding query become reachable.

Implementation-wise, the lists will be stored in the root node of each tree, and will be merged using the small-to-large technique. This ensures that each node will be merged at most $O(\log n)$ times and since there are q indices stored in the lists, there will be at most $O(q \log n)$ list element merges. Each merge, in turn, takes $O(\log n)$ time as a single union find query will be used for checking whether the corresponding nodes became reachable. Thus, the resulting algorithm has a time complexity of $O(m \log n + q \log^2 n)$.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 200000;
int qa[N], qb[N], ans[N];

struct UnionFind {
    int n;
    // If p[i] < 0, i is a root node with a tree size of -p[i].
    // Otherwise, p[i] represents the parent of i.
    vector<int> p;
    // Stores the indices of queries at root nodes.
    vector<vector<int>> q;
```

```
    UnionFind(int n) : n(n), p(n, -1), q(n) {}
    int root(int s) {
        while (p[s] >= 0) {
            s = p[s];
        }
        return s;
    }
    void merge(int a, int b, int t) {
        a = root(a);
        b = root(b);
        if (a == b) return;
        if (p[a] < p[b]) {
            swap(a, b);
        }
        p[b] += p[a];
        p[a] = b;
        if (q[b].size() < q[a].size()) {
            swap(q[a], q[b]);
        }
        for (int i : q[a]) {
            if (ans[i] != -1) continue;
            if (root(qa[i]) == root(qb[i])) {
                ans[i] = t;
            } else {
                q[b].push_back(i);
            }
        }
        q[a].clear();
    }
};

int main() {
    int n, m, q;
    cin >> n >> m >> q;

    vector<pair<int, int>> edges(m);
    for (auto &[a, b] : edges) {
        cin >> a >> b;
        a--;
        b--;
    }

    UnionFind uf(n);
    for (int i = 0; i < q; ++i) {
        int &a = qa[i], &b = qb[i];
        cin >> a >> b;
        a--;
        b--;
        if (a == b) {
            ans[i] = 0;
        } else {
            ans[i] = -1;
            uf.q[a].push_back(i);
            uf.q[b].push_back(i);
        }
    }

    for (int i = 0; i < m; ++i) {
        uf.merge(edges[i].first, edges[i].second, i + 1);
    }

    for (int i = 0; i < q; ++i) {
        cout << ans[i] << '\n';
    }
}
```

2088 · Knuth Division

Task · source analysis length 2,386 characters

Problem statement

Given an array of n numbers, your task is to divide it into n subarrays, each of which has a single element.

On each move, you may choose any subarray and split it into two subarrays. The cost of such a move is the sum of values in the chosen subarray.

What is the minimum total cost if you act optimally?

Input

The first input line has an integer n: the array size. The array elements are numbered $1, 2, \ldots, n$.

The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output

Print one integer: the minimum total cost.

Constraints

- $1 \le n \le 5000$
- $1 \le x_i \le 10^9$

We will solve this problem using dynamic programming. If $\mathrm{dp}[l][r]$ represents the cost to divide the subarray $[l, r]$, we will have the following recurrence

$$\mathrm{dp}[i$$

$$j$$

$$= \min_{i \le k < j} \mathrm{dp}$$

$$i$$

$$k$$

$$+\mathrm{dp}$$

$$k + 1$$

$$j$$

$$+\mathrm{cost}i, j,]$$

where $\mathrm{cost}(i, j)$ represents the cost to split the subarray $[i, j]$, in this case that is the sum of the subarray. Filling the dynamic programming table according to this rule would result in an algorithm with a time complexity of $O(n^3)$ which is too slow.

To speed up the solution, we will use a dynamic programming optimization called *Knuth's optimization*. The optimization is applicable to any recurrence similar to the above one if the cost function satisfies these two

inequalities:

$$\text{cost}(b, c) \leq \text{cost}(a, d)$$
$$\text{cost}(a, c) + \text{cost}(b, d) \leq \text{cost}(a, d) + \text{cost}(b, c)$$

for all $a \leq b \leq c \leq d$.

Let $\text{pos}(i, j)$ denote the k that minimizes the original recurrence, for i and j. We know that, if the above inequalities hold,

$$\text{pos}(i, j-1) \leq \text{pos}(i, j) \leq \text{pos}(i+1, j).$$

This allows us to fill the DP table efficiently. Let us process the subarrays in increasing order of their length. Now, when we are processing the subarray $[i, j]$, we have already computed the answer for the arrays $[i, j-1]$ and $[i+1, j]$ and thus we know their pos-values. The pos-values of the previous length are increasing when sorted by the starting index. This means that they divide the search space for the next pos-values into disjoint intervals, which allows us to fill each layer in $O(n)$ time, making the total time complexity of the solution $O(n^2)$.

```
#include <iostream>
using namespace std;
using ll = long long;
const ll INF = 1e18;

const int N = 5000;
int x[N], pos[N][N];
ll s[N + 1], dp[N][N];

int main() {
    int n;
    cin >> n;

    for (int i = 0; i < n; ++i) {
        cin >> x[i];
        s[i + 1] = s[i] + x[i];
        dp[i][i] = 0;
        pos[i][i] = i + 1;
    }

    for (int len = 2; len <= n; ++len) {
        for (int l = 0; l <= n - len; ++l) {
            int r = l + len - 1;
            dp[l][r] = INF;
            for (int p = pos[l][r - 1]; p <= pos[l + 1][r]; ++p) {
                ll off = dp[l][p - 1] + dp[p][r] + (s[r + 1] - s[l]);
                if (off < dp[l][r]) {
                    dp[l][r] = off;
                    pos[l][r] = p;
                }
            }
        }
    }

    cout << dp[0][n - 1] << endl;
}
```

2087 · Houses and Schools

Task · source analysis length 3,654 characters

Problem statement

There are n houses on a street, numbered $1, 2, \ldots, n$. The distance of houses a and b is $|a - b|$. You know the number of children in each house.

Your task is to establish k schools in such a way that each school is in some house. Then, each child goes to the nearest school. What is the minimum total walking distance of the children if you act optimally?

Input

The first input line has two integers n and k: the number of houses and the number of schools. The houses are numbered $1, 2 \ldots, n$.

After this, there are n integers $c_1, c_2, \ldots, c_n$: the number of children in each house.

Output

Print the minimum total distance.

Constraints

- $1 \le k \le n \le 3000$
- $1 \le c_i \le 10^9$

There are a few different approaches to this problem. Most of them will have a dynamic programming table similar to the following one:

$$\mathrm{dp}[j$$

$$i$$

, 1 \le j \le k, 1 \le i \le n]

denotes the minimum possible sum of walking distances for the first i children provided there are j schools with the last of them at index i.

We will use $\mathrm{cost}(i, j)$ to denote the sum of walking distances for the children in houses from i to j, provided that there are schools in houses i and j. Simply filling this table according to the recurrence

$$\mathrm{dp}[j$$

$$i$$

= \min_{p < i} \mathrm{dp}

$$j - 1$$

$$p$$

+ \mathrm{cost}p, i]

would result in an algorithm with a time complexity of $O(n^2k)$. This may be sped up with a DP optimization called *divide and conquer*. It can be applied only if the $\mathbf{cost}$ function satisfies the *quadrangle inequality*

$$\mathbf{cost}(a, c) + \mathbf{cost}(b, d) \leq \mathbf{cost}(a, d) + \mathbf{cost}(b, c)$$

for all $a \leq b \leq c \leq d$. It may be proven that our $\mathbf{cost}$ function $thesumofwalkingdistancesbetweenschools$ satisfies the inequality.

Let $\mathbf{pos}_j(i)$ denote the minimum position p that minimizes the value of

$$\min_{p<i} \mathrm{dp}[j-1$$

$$p$$

+ \mathrm{cost}p, i.]

It may be proven that if the quadrangle inequality holds, then $\mathbf{pos}_j(i)$ forms an increasing sequence for all i, i.e. $\mathbf{pos}_j(i$ \le \mathrm{pos}_j$i+1$). This fact allows us to make a recursive function that fills one layer of the dp table, for a fixed j.

The idea is to start with a search range of $[0, n)$, and at each step, split the range in half and recursively process the two parts. The range $[i, j)$ becomes $[i, m)$ and $[m, r)$, where m denotes the midpoint of the range $m = \lfloor \frac{l+r}{2} \rfloor$. Then, if we are able to find $\mathrm{dp}[j][m]$ and thus, $\mathbf{pos}_j(m)$, we know that $\mathbf{pos}_j(i$ \le \mathrm{pos}_j $m)$, for all $i < m$ and $\mathbf{pos}_j(m$ \le \mathrm{pos}_j$i)$, for all $m < i$. This allows us to reduce the search space for p in the subsequent calls, making the time complexity of filling a dp layer $O(n \log n)$ and the total time complexity $O(nk \log n)$.

There are more advanced solutions to this problem, for example, using a technique called the *aliens trick* combined with the *convex hull trick*.

```
#include <iostream>
using namespace std;
using ll = long long;

const ll INF = 1e18;
const int N = 3000;

int c[N];
ll lcost[N][N];
ll rcost[N][N];

ll left_cost(int l, int r) {
    return l <= r ? lcost[l][r] : 0;
}
ll right_cost(int l, int r) {
    return l <= r ? rcost[l][r] : 0;
}

ll cost(int l, int r) {
    int mid = (l + r) / 2;
    return left_cost(l, mid) + right_cost(mid + 1, r);
}

ll dp[N][N];

void recurse(int lvl, int l, int r, int pos_l, int pos_r) {
    if (l > r) return;

    int mid = (l + r) / 2;
    ll best = INF;
    int pos = l;

    for (int k = pos_l; k <= min(mid, pos_r); k++) {
        ll offer = dp[lvl - 1][k] + cost(k, mid);
        if (offer < best) {
            best = offer;
            pos = k;
        }
    }

    dp[lvl][mid] = best;

    recurse(lvl, l, mid - 1, pos_l, pos);
    recurse(lvl, mid + 1, r, pos, pos_r);
}
```

```
int main() {
    int n, k;
    cin >> n >> k;

    for (int i = 0; i < n; ++i) {
        cin >> c[i];
    }

    for (int i = 0; i < n; ++i) {
        ll s = 0;
        for (int j = i; j < n; ++j) {
            s += ll(j - i) * c[j];
            lcost[i][j] = s;
        }
    }

    for (int i = 0; i < n; ++i) {
        ll s = 0;
        for (int j = i; j >= 0; --j) {
            s += ll(i - j) * c[j];
            rcost[j][i] = s;
        }
    }

    for (int i = 0; i < n; ++i) {
        dp[0][i] = right_cost(0, i);
    }

    for (int i = 1; i < k; ++i) {
        recurse(i, 0, n, 0, n);
    }

    ll ans = INF;
    for (int i = 0; i < n; ++i) {
        ans = min(ans, dp[k - 1][i] + left_cost(i, n - 1));
    }

    cout << ans << '\n';
}
```

2086 · Subarray Squares

Task · source analysis length 3,218 characters

Problem statement

Given an array of n elements, your task is to divide into k subarrays. The cost of each subarray is the square of the sum of the values in the subarray. What is the minimum total cost if you act optimally?

Input

The first input line has two integers n and k: the array elements and the number of subarrays. The array elements are numbered $1, 2, \ldots, n$.

The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output

Print one integer: the minimum total cost.

Constraints

- $1 \le k \le n \le 3000$
- $1 \le x_i \le 10^5$

We will solve this problem using dynamic programming. We will have a dp table, where $\mathrm{dp}[j][i]$ refers to the minimum price to divide the first i elements into j subarrays, the last of which ends at i.

We will fill the table according to the recurrence

$$\mathrm{dp}[j$$

$$i$$

$$= \min_{k< i}\mathrm{dp}$$

$$j-1$$

$$k$$

$$+x_k + x_{k+1} + \cdots + x_{i-1}^2.]$$

If we build a prefix sum array sum, where $\mathrm{sum}[i]$ refers to the sum of the first i elements, the recurrence turns into

$$\mathrm{dp}[j$$

$$i$$

$$= \min_{k < i}\mathrm{dp}$$

$$j-1$$

$$k$$

$$+\mathbf{sum}[i] - \mathbf{sum}[k-1]\text{^2.]}$$

Note that if we expand the expression

$$(\mathbf{sum}[i$$

-\mathrm{sum}

$$k-1$$

)^2= \mathrm{sum}

$$i$$

^2 -2\cdot\mathrm{sum}

$$i$$

\cdot\mathrm{sum}

$$k-1$$

+\mathrm{sum}

$$k-1$$

^2,]

we may rearrange the recurrence into

$$\mathbf{dp}[j$$

$$i$$

=\mathrm{sum}

$$i$$

^2+ \min_{k < i}\mathrm{dp}

$$j-1$$

$$k$$

-2\cdot\mathrm{sum}

$$i$$

\cdot\mathrm{sum}

$$k-1$$

+\mathrm{sum}

$$k-1$$

^2.]

Now, each previous position $k < i$ can be thought of as a line

$$y_k = x \cdot (-2 \cdot \mathbf{sum}[k-1$$

) +\mathrm{dp}

$$j-1$$

k

+\mathrm{sum}

$k-1$

^2,]

and we are asked to find the minimum height of a line at position $x = \mathrm{sum}[i]$.

A technique known as the *convex hull trick* does exactly this. As the sequence of slopes ($-2 \cdot \mathrm{sum}[k-1]$) is decreasing, we only need to maintain a stack of lines. Furthermore, as the sequence of query points ($\mathrm{sum}[i]$) is increasing, we can maintain a pointer to the array and find the answer to each query with an amortized cost of $O(1)$.

Note that, at all times, we are using only the two latest layers of the dp table. For this reason, it suffices to store only two arrays, representing the last two layers.

This approach fills each layer in linear time and thus has a time complexity of $O(nk)$. It is possible to solve the problem even quicker, using a techinique known as the *Aliens trick* $\mathit{linkedbelow}$.

References

- CP-Algorithms, *Convex Hull Trick*
- serbanology.com, *The Trick From Aliens*

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const ll INF = 1e18;

struct Line {
    ll a, b;  // Represents the line y = ax + b
    // Returns the height of the line at x.
    ll operator()(ll x) { return a * x + b; }
};

int main() {
    int n, k;
    cin >> n >> k;

    vector<int> x(n);
    vector<ll> sum(n + 1);
    for (int i = 0; i < n; ++i) {
        cin >> x[i];
        sum[i + 1] = sum[i] + x[i];
    }

    vector<ll> dp(n + 1, INF);
    dp[0] = 0;

    for (int i = 0; i < k; ++i) {
        vector<Line> lines;
        int pos = 0;  // Stores the index of the current lowest line.

        // Finds the height of the lowest line at position x.
        // Assumes that the sequence of x's is increasing.
        auto get_best = [&](ll x) {
            while (pos + 1 < size(lines) && lines[pos + 1](x) < lines[pos](x)) {
                pos++;
            }
            return lines[pos](x);
        };

        vector<ll> ndp(n + 1);
        for (int j = i; j <= n; ++j) {
            if (j != i) {
                ndp[j] = sum[j] * sum[j] + get_best(sum[j]);
            }
            Line l = {-2 * sum[j], dp[j] + sum[j] * sum[j]};
            while (lines.size() > 1) {
                auto a = lines[lines.size() - 2];
                auto b = lines[lines.size() - 1];
                if ((l.b - a.b) * (a.a - b.a) <= (b.b - a.b) * (a.a - l.a)) {
                    lines.pop_back();
                } else {
```

```
                    break;
                }
            }
            lines.push_back(l);
        }
        swap(dp, ndp);
    }
    cout << dp[n] << '\n';
}
```

2085 · Monster Game II

Task · source analysis length 10,201 characters

Problem statement

You are playing a game that consists of n levels. Each level has a monster. On levels $1, 2, \ldots, n-1$, you can either kill or escape the monster. However, on level n you must kill the final monster to win the game.

Killing a monster takes sf time where s is the monster's strength and f is your skill factor. After killing a monster, you get a new skill factor $lowerskill factorisbetter$. What is the minimum total time in which you can win the game?

Input

The first input line has two integers n and x: the number of levels and your initial skill factor.

The second line has n integers $s_1, s_2, \ldots, s_n$: each monster's strength.

The third line has n integers $f_1, f_2, \ldots, f_n$: your new skill factor after killing a monster.

Output

Print one integer: the minimum total time to win the game.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le x \le 10^6$
- $1 \le s_i, f_i \le 10^6$

Three separate approaches to this problem are presented below. We are trying to solve, for each i, the minimum time it takes to kill the ith monster and possibly some others before it. The key idea in all of the solutions is that these minimum times $t_1, t_2, \ldots t_n$ form offers of the form (t_i, f_i), where t_i is the minimum time it takes to kill the ith monster and f_i is the player's strength after killing the monster. These offers remain the same for all $j > i$.

With this observation, the task turns into one where we have to find a data structure for storing the offers and querying the lowest total time for a monster of strength s_i. The total time for killing monster i by utilizing offer j is $t_j + f_j \cdot s_i$. The offers can be thought of geometrically as lines and the different approaches to this problem are known collectively as the *convex hull trick*.

Solution 1: Ordinary Convex Hull Trick & Bentley-Saxe Transformation

The simplest possible data structure for storing the convex hull is an array. However, an array does not allow efficient insertions to the middle of it. Thus, by itself, a simple array would be too slow. Fortunately, there is a way to speed up this solution. It is known as the *Bentley-Saxe transformation*.

The Bentley-Saxe transformation allows us to make insertions to static data structures, i.e. ones that do not support insertions, turning them into dynamic ones. The key idea is to maintain $O(\log N)$ static data structures and to merge two of them when they are of equal size. Merging two hulls means taking their lines and constructing a completely new hull from the ground up. The process goes like this:

- Maintain a stack of static hulls.
- When adding a new line, make a hull of only the new line, and push it onto the stack.
- Whenever the two latest data structures in the back of the stack are of equal size, merge them into a new hull of double the size.

This process ensures that, at all times, we have at most $O(\log N)$ data structures to query from and that each line is merged at most $O(\log N)$ times. The total time complexity of the construction is $O(N \log N)$ as each line contributes $O(1)$ work to the construction of each of the hulls it is part of. Querying takes $O(\log^2 N)$ time as there are $O(\log N)$ hulls and binary search needs to be done in each of them to find the best line.

References

- CP-Algorithms, *Convex Hull Trick*
- J. L. Bentley, J. B. Saxe, Decomposable searching problems I. Static-to-dynamic transformation, Journal of Algorithms, Volume 1, Issue 4, 1980


```
#include <algorithm>
#include <iostream>
#include <limits>
#include <tuple>
#include <vector>
using namespace std;
using ll = long long;
const ll INF = numeric_limits<ll>::max();

struct Line {
    ll a, b;  // Represents the line y = ax + b
    // Returns the height of the line at x.
    ll operator()(ll x) const { return a * x + b; }
    bool operator<(Line oth) const {
        return a != oth.a ? a > oth.a : b < oth.b;
    }
    ll intersect(Line oth) {
        ll num = oth.b - b, dem = a - oth.a;
        return num / dem - ((num ^ dem) < 0 && num % dem);
    }
};

struct Hull {
    // Stores all lines regardless of whether they are part of the hull or not.
    vector<Line> all;
    // Stores the convex hull lines and the intersection points between
    // subsequent lines.
    vector<pair<ll, Line>> hull;
    Hull(Line x) : all{x}, hull{{-INF, x}} {}
    // Constructs a new hull out of hulls a and b.
    Hull(const Hull &a, const Hull &b) {
        all.resize(a.size() + b.size());
        // Note that because the lines of a and b are already sorted, we are
        // able to merge them in O(N) time without needing to sort the array.
        merge(a.all.begin(), a.all.end(), b.all.begin(), b.all.end(),
              all.begin());
        hull.emplace_back(-INF, all[0]);
        for (int i = 1; i < (int)all.size(); ++i) {
            auto l = all[i];
            if (hull.back().second.a == l.a) continue;
            while (hull.size() >= 2) {
                auto al = hull[hull.size() - 2].second;
                auto bl = hull[hull.size() - 1].second;
                if (bl.intersect(l) <= al.intersect(bl)) {
                    hull.pop_back();
                } else {
                    break;
                }
            }
            hull.emplace_back(hull.back().second.intersect(l), l);
        }
    }
    // Returns the lowest line at point x.
    ll query(ll x) const {
        auto p = pair{x, Line{INF, INF}};
        auto it = prev(lower_bound(hull.begin(), hull.end(), p));
        return it->second(x);
    }
    size_t size() const { return all.size(); }
};

int main() {
```

```
    int n, x;
    cin >> n >> x;

    vector<int> s(n), f(n);
    for (int i = 0; i < n; ++i) cin >> s[i];
    for (int i = 0; i < n; ++i) cin >> f[i];

    vector<Hull> hulls;
    hulls.emplace_back(Line{x, 0});

    auto best_line = [&](ll x) {
        ll r = INF;
        for (auto &h : hulls) r = min(r, h.query(x));
        return r;
    };

    for (int i = 0; i < n - 1; ++i) {
        ll b = best_line(s[i]);
        hulls.emplace_back(Line{f[i], b});
        while (hulls.size() > 1) {
            auto &a = hulls[hulls.size() - 2];
            auto &b = hulls[hulls.size() - 1];
            if (a.size() == b.size()) {
                a = Hull(a, b);
                hulls.pop_back();
            } else {
                break;
            }
        }
    }

    cout << best_line(s[n - 1]) << '\n';
}
```

Solution 2: Set Structure

Another approach is to store the lines in a set structure. The set maintains the order of the lines and if we store the intersection points of subsequent lines, we are again able to find the minimum line at a point using binary search.

Inserting a line has an amortized cost of $O(\log N)$ as there may be multiple deletions following an insertion but each line can be deleted only once. Querying the minimum line at a point takes $O(\log N)$ time. The downside of this approach is that the set structure is cumbersome to implement as the hull and intersection points have to be maintained and updated with each insertion.

References

- KACTL, ICPC Team Reference Document, LineContainer.h
- CP-Algorithms, *Convex hull trick*


```
#include <iostream>
#include <limits>
#include <set>
#include <vector>
using namespace std;
using ll = long long;
const ll INF = numeric_limits<ll>::max();

struct Line {
    ll a, b;  // Represents the line y = ax + b
    ll p;  // The x-coordinate at which this line intersects the previous one.
    // Returns the height of the line at x.
    ll operator()(ll x) const { return a * x + b; }
    bool operator<(Line oth) const { return a > oth.a; }
    bool operator<(ll x) const { return p < x; }
    ll intersect(Line oth) const {
        ll num = oth.b - b, dem = a - oth.a;
        return num / dem - ((num ^ dem) < 0 && num % dem);
    }
};

struct Hull {
    set<Line, less<>> hull;
    using Iter = set<Line, less<>>::iterator;
    // Recalculates the intersection point of the line at `it`.
    void fix(Iter it) {
```

```
        Line l = *it;
        it = hull.erase(it);
        if (it != end(hull)) {
            l.p = l.intersect(*it);
        } else {
            l.p = INF;
        }
        hull.insert(it, l);
    }
    // Check whether the line at `it` is useless.
    // A line is said to be useless if it never has the minimum value.
    bool check(Iter it) {
        if (it != begin(hull) && next(it) != end(hull)) {
            auto a = *prev(it);
            auto b = *it;
            auto c = *next(it);
            return __int128(a.b - c.b) * (b.a - a.a) <=
                   __int128(c.a - a.a) * (a.b - b.b);
        }
        return false;
    }
    // Insert line l into the hull.
    void insert(Line l) {
        auto it = hull.lower_bound(l);
        if (it != end(hull) && it->a == l.a) {
            if (l.b >= it->b) return;
            it = hull.erase(it);
        }
        it = hull.insert(it, l);
        if (check(it)) {
            hull.erase(it);
        } else {
            while (it != begin(hull) && check(prev(it))) {
                hull.erase(prev(it));
            }
            while (next(it) != end(hull) && check(next(it))) {
                hull.erase(next(it));
            }
            if (it != begin(hull)) fix(prev(it));
            if (next(it) != end(hull)) fix(next(it));
            fix(it);
        }
    }
    // Returns the lowest line at point x.
    ll query(ll x) const {
        // Note that we have implemented two separate comparator functions,
        // one for comparing lines with coordinates and another for comparing
        // with other lines.
        // When we compare a line with a coordinate, we are actually comparing
        // the start of the line's active range with the coordinate.
        Line l = *hull.lower_bound(x);
        return l(x);
    }
};

int main() {
    int n, x;
    cin >> n >> x;

    vector<int> s(n), f(n);
    for (int i = 0; i < n; ++i) cin >> s[i];
    for (int i = 0; i < n; ++i) cin >> f[i];

    Hull h;
    h.insert({x, 0});

    for (int i = 0; i < n - 1; ++i) {
        ll b = h.query(s[i]);
        h.insert({f[i], b});
    }

    cout << h.query(s[n - 1]) << '\n';
}
```

Solution 3: Li Chao Tree

The last approach presented here is known as the *Li Chao tree*. It is a variation on segment trees which allows us to store lines and query the minimum line at a certain point. The only downside of the Li Chao tree approach is that it requires us to know the query points beforehand.

References

- CP-Algorithms, *Li Chao Tree*

```
#include <algorithm>
#include <iostream>
#include <limits>
#include <vector>
using namespace std;
using ll = long long;
const ll INF = numeric_limits<ll>::max();

struct Line {
    ll a, b;  // Represents the line y = ax + b
    // Returns the height of the line at x.
    ll operator()(ll x) const { return a * x + b; }
};

struct LiChao {
    int n;
    vector<Line> p;
    vector<ll> xs;

    // Constructs a Li Chao tree which supports the insertion of any line and
    // queries in all positions included in the array x.
    LiChao(vector<int> x) {
        n = 1;
        while (n <= x.size()) n *= 2;
        p.assign(n * 2, Line{0, INF});
        sort(x.begin(), x.end());
        while (x.size() < n) x.push_back(x.back());
        xs.assign(n * 2, 0);
        for (int i = 0; i < n; ++i) xs[i + n] = x[i];
        for (int i = n - 1; i > 0; --i) {
            xs[i] = xs[i * 2];
        }
    }

    // Inserts line x into the tree.
    void insert(Line x) {
        int s = 1;
        while (s < n) {
            ll p1 = xs[s * 2], p2 = xs[s * 2 + 1];
            bool c1 = x(p1) < p[s](p1);
            bool c2 = x(p2) < p[s](p2);
            if (c2) swap(x, p[s]);
            s = s * 2 + (c1 == c2);
        }
        if (x(xs[s]) < p[s](xs[s])) {
            p[s] = x;
        }
    }
    // Returns the lowest line at point x.
    // Note that x has to be included in the vector of points provided earlier.
    ll query(ll x) {
        int i = lower_bound(xs.begin() + n, xs.end(), x) - xs.begin();
        ll r = INF;
        while (i > 0) {
            r = min(r, p[i](x));
            i /= 2;
        }
        return r;
    }
};

int main() {
    int n, x;
    cin >> n >> x;

    vector<int> s(n), f(n);
    for (int i = 0; i < n; ++i) cin >> s[i];
    for (int i = 0; i < n; ++i) cin >> f[i];

    LiChao l(s);
    l.insert({x, 0});

    for (int i = 0; i < n - 1; ++i) {
        ll b = l.query(s[i]);
        l.insert({f[i], b});
    }
```

```
    cout << l.query(s[n - 1]) << '\n';
}
```

2084 · Monster Game I

Task · source analysis length 4,210 characters

Problem statement

You are playing a game that consists of n levels. Each level has a monster. On levels $1, 2, \ldots, n-1$, you can either kill or escape the monster. However, on level n you must kill the final monster to win the game.

Killing a monster takes sf time where s is the monster's strength and f is your skill factor $lowerskill factorisbetter$. After killing a monster, you get a new skill factor. What is the minimum total time in which you can win the game?

Input

The first input line has two integers n and x: the number of levels and your initial skill factor.

The second line has n integers $s_1, s_2, \ldots, s_n$: each monster's strength.

The third line has n integers $f_1, f_2, \ldots, f_n$: your new skill factor after killing a monster.

Output

Print one integer: the minimum total time to win the game.

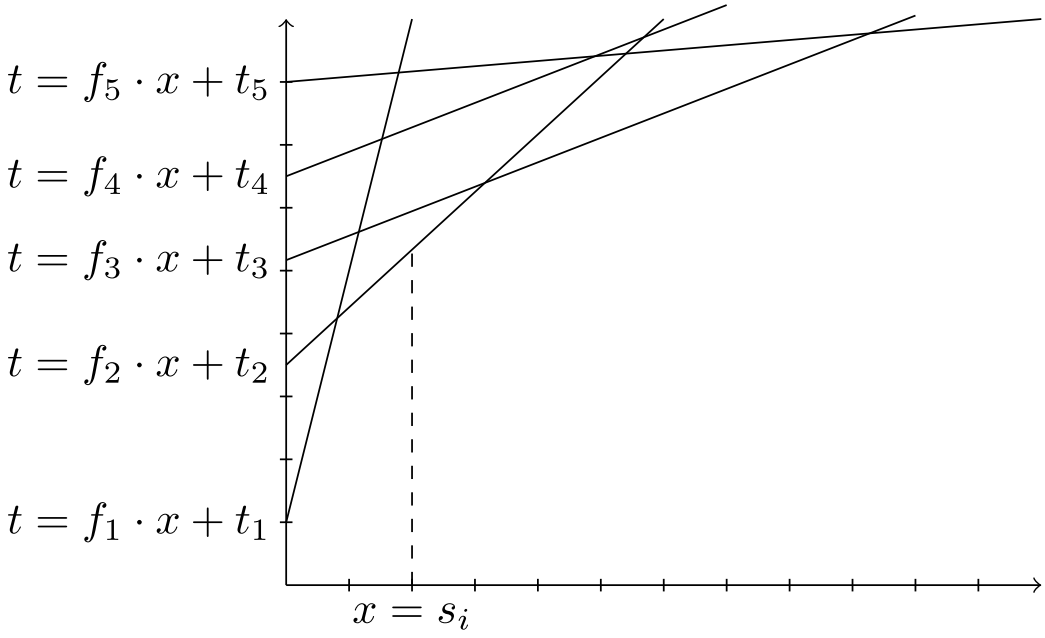
Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le x \le 10^6$
- $1 \le s_1 \le s_2 \le \cdots \le s_n \le 10^6$
- $x \ge f_1 \ge f_2 \ge \cdots \ge f_n \ge 1$

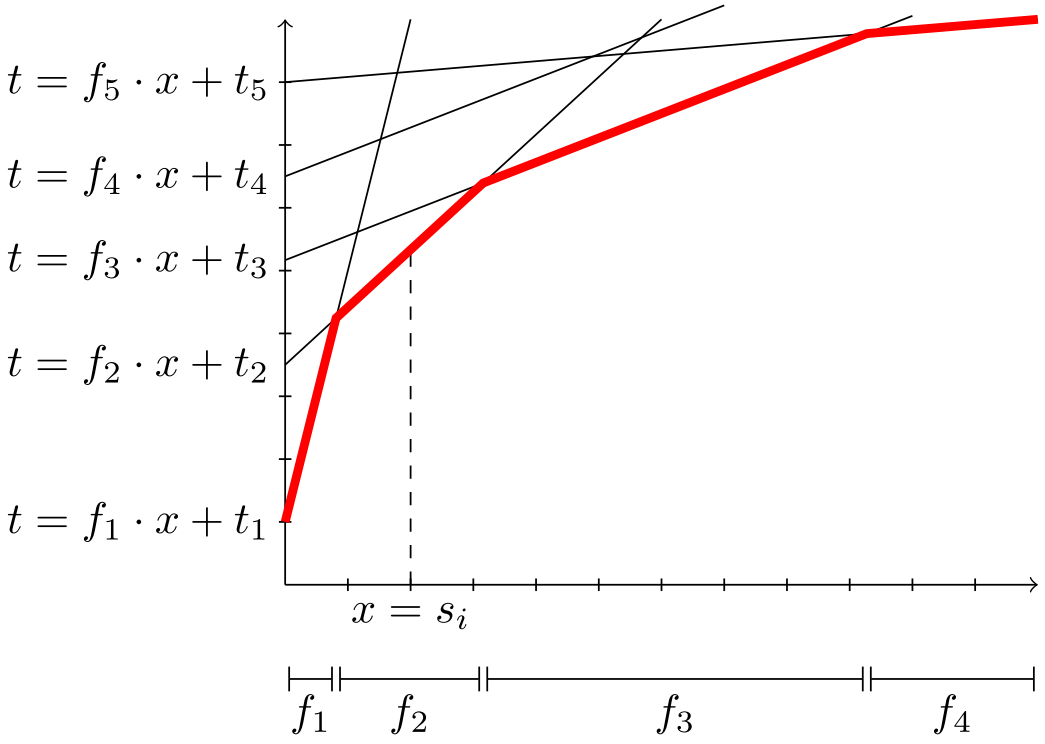
For each i, we will find the minimum possible time to kill the ith monster and possibly some other monsters before it. Let us focus on the ith monster. Assuming we have solved the minimum time for each previous monster, we have $i-1$ offers of the form (t_j, f_j), where t_j is the minimum possible time it takes to kill the j th monster and f_j is the player's skill factor after killing the jth monster.

For any offer $j < i$, it takes $s_i \cdot f_j$ more time to kill the ith monster and thus $t = t_j + s_i \cdot f_j$ time in total. Notice that the offers can be thought of geometrically as lines $y = ax + b$ where the height of a line at point x corresponds to the total time it takes to kill a monster of strength x.

We are interested in finding out the minimum possible time it takes to kill the ith monster. In other words, the minimum height of a line at point $x = s_i$.



Existing offers do not change depending on our position in the array, that is, they do not depend on i. This means that if we are able to find an efficient data structure for inserting and storing the lines, it may be used for calculating the best time for each i.



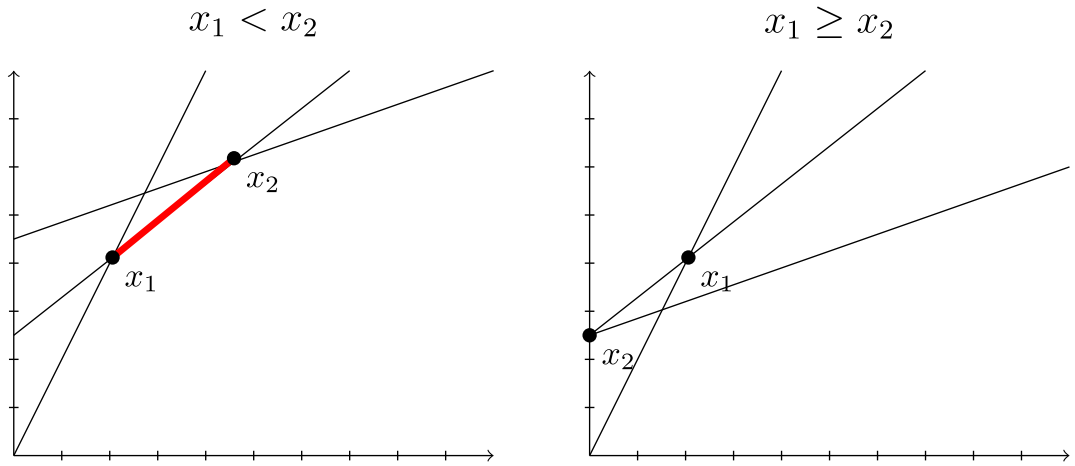
Note the minimum line changes depending on x. In fact, the lines divide the x-axis into ranges where a certain line is the lowest. As can be seen from the picture, each line corresponds to at most one range, which means the x-axis is divided into at most n segments.

Let us analyze the situation a bit further. Recall from the task statement that the slopes of the lines (f_i's) are decreasing and the points at which we are querying the best line (s_i's) are increasing. The slopes of the minimum lines are also in decreasing order. This means that each new offer will be the new last minimum line. This allows us to maintain a simple array of lines and push each new offer to the back of it.

Note that some lines may never have the minimum value. This is the case for the fourth line in the picture. When adding a new line to the back of the array, we need to check if it makes the previous line redundant. If

that is the case, we can simply pop the previous line from the array before adding the new one.

There is a simple way for checking whether the new line will make the previous one redundant. It suffices to compare two intersection points. Let x_1 be the intersection of the last line and the one before that and x_2 the intersection point of the last line and the new one. The last line will become redundant if $x_1 \geq x_2$. This is because the last line has the minimum value on the range $[x_1, x_2]$.



To find the minimum line at position x, we may use binary search. However, as the sequence of query points is increasing, it is also possible to just maintain a pointer to the array of lines, which makes the amortized cost of querying $O(1)$. In total, the solution has a time complexity of $O(n)$. The key idea of maintaining the convex hull of linear functions is known as the *convex hull trick*.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

struct Line {
    ll a, b;  // Represents the line y = ax + b
    // Returns the height of the line at x.
    ll operator()(ll x) { return a * x + b; }
};

int main() {
    int n, x;
    cin >> n >> x;

    vector<int> s(n), f(n);
    for (int i = 0; i < n; ++i) cin >> s[i];
    for (int i = 0; i < n; ++i) cin >> f[i];

    vector<Line> lines;
    lines.push_back({x, 0});

    int pos = 0;  // Stores the index of the current lowest line.

    // Finds the height of the lowest line at position x.
    // Assumes that the sequence of x's is increasing.
    auto get_best = [&](ll x) {
        while (pos + 1 < lines.size() && lines[pos + 1](x) < lines[pos](x)) {
            pos++;
        }
        return lines[pos](x);
    };

    for (int i = 0; i < n - 1; ++i) {
        ll b = get_best(s[i]);
        Line l = {f[i], b};
        while (lines.size() > 1) {
            auto a = lines[lines.size() - 2];
            auto b = lines[lines.size() - 1];
            // Compare x1 and x2.
            //
            //      b.b - a.b          l.b - a.b
            // x1 = ---------     x2 = ---------
            //      a.a - b.a          a.a - l.a
            if ((l.b - a.b) * (a.a - b.a) <= (b.b - a.b) * (a.a - l.a)) {
                lines.pop_back();
            } else {
```

```
                break;
            }
        }
        lines.push_back(l);
    }

    cout << get_best(s[n - 1]) << '\n';
}
```

References

- CP-Algorithms, *Convex hull trick*

2081 · Fixed-Length Paths II

Task · source analysis length 6,505 characters

Problem statement

Given a tree of n nodes, your task is to count the number of distinct paths that have at least k_1 and at most k_2 edges.

Input

The first input line contains three integers n, k_1 and k_2: the number of nodes and the path lengths. The nodes are numbered $1, 2, \ldots, n$.

Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: the number of paths.

Constraints

- $1 \le k_1 \le k_2 \le n \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Solution 1: Centroid Decomposition

This problem can be solved with centroid decomposition.

The idea is to recursively split the tree into disjoint trees. We find a centroid node of each tree and calculate the number of paths with a length between k_1 and k_2 which go through the centroid. After this, all unaccounted paths are completely contained in the subtrees of the node. Thus, we may remove the centroid from the tree and repeat the same procedure for the resulting new trees.

We use a Fenwick tree to count the number of nodes at specific distances from the centroid.

At each step, the sizes of the resulting trees will be at most half of the original size. This means that each node will be part of at most $O(\log n)$ trees. As processing a node corresponds to a query in the Fenwick tree and every node of every tree is processed exactly twice, the total time complexity of the solution is $O(n \log^2 n)$.

```
#include <array>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

struct FenwickTree {
    static const int N = 200000;
    array<int, N + 1> tree;
    void add(int k, int d) {
        for (int i = k + 1; i <= N; i += i & -i) {
            tree[i] += d;
        }
    }
    int sum(int k) {
        int s = 0;
        for (int i = k; i > 0; i -= i & -i) {
```

```
            s += tree[i];
        }
        return s;
    }
    int sum(int l, int r) { return sum(r + 1) - sum(l); }
} tree;

const int N = 200001;
bool deleted[N];
int n, k1, k2, sizes[N];
vector<int> g[N];
ll answer;

void dfs_sizes(int node, int parent) {
    sizes[node] = 1;
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_sizes(child, node);
        sizes[node] += sizes[child];
    }
}

int find_centroid(int node, int parent, int n) {
    for (int child : g[node]) {
        if (child != parent && !deleted[child] && sizes[child] > n / 2) {
            return find_centroid(child, node, n);
        }
    }
    return node;
}

void dfs_add(int node, int parent, int depth, int change) {
    tree.add(depth, change);
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_add(child, node, depth + 1, change);
    }
}

void dfs_query(int node, int parent, int depth) {
    answer += tree.sum(k1 - depth, k2 - depth);
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_query(child, node, depth + 1);
    }
}

void recurse(int node) {
    dfs_sizes(node, -1);
    int r = find_centroid(node, -1, sizes[node]);
    tree.add(0, 1);
    for (int child : g[r]) {
        if (!deleted[child]) {
            dfs_query(child, r, 1);
            dfs_add(child, r, 1, 1);
        }
    }
    dfs_add(r, -1, 0, -1);
    deleted[r] = true;
    for (int child : g[r]) {
        if (!deleted[child]) {
            recurse(child);
        }
    }
}

int main() {
    cin >> n >> k1 >> k2;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    recurse(1);
    cout << answer << '\n';
}
```

Solution 2: Small-To-Large Merging

There is another, simpler solution to this problem. It uses a technique known as *small-to-large merging* combined with *policy-based* data structures.

Policy-based data structures are not part of the C++ standard, rather they are built-in to the GNU compiler. The policy-based balanced binary tree allows us to query the number of elements in a range using the function `order_of_key`.

After rooting the tree, we make a depth-first traversal of it. During the traversal, we calculate the number of paths with a length between k_1 and k_2 whose highest node is the current one.

With small-to-large merging, it can be proven that each element will be moved at most $O(\log n)$ times making the total time complexity of the solution $O(n \log^2 n)$.

Note that a policy-based tree only supports unique values. We can get around this by storing the depths in `pairs` with the node index as the second element.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using indexed_set =
    tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

int query_range(indexed_set<pair<int, int>> &tree, int l, int r) {
    return tree.order_of_key({r + 1, 0}) - tree.order_of_key({l, 0});
}

const int N = 200001;
int n, k1, k2;
vector<int> g[N];
ll answer;

indexed_set<pair<int, int>> dfs(int node, int parent, int depth) {
    indexed_set<pair<int, int>> dist;
    dist.insert({depth, node});
    for (int child : g[node]) {
        if (child == parent) continue;
        auto child_dist = dfs(child, node, depth + 1);
        if (child_dist.size() > dist.size()) {
            dist.swap(child_dist);
        }
        for (auto [d, s] : child_dist) {
            answer += query_range(dist, k1 - d + 2 * depth, k2 - d + 2 * depth);
        }
        for (auto p : child_dist) dist.insert(p);
    }
    return dist;
}

int main() {
    cin >> n >> k1 >> k2;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1, 0);
    cout << answer << '\n';
}
```

Solution 3: Improved merge

Instead of using a policy-based tree to store information about the depths of the nodes, we may use a deque. If each position of the deque corresponds to a depth, we are only able to query the amount of nodes at a specific depth. To query the number of nodes with a depth between k_1 and k_2, we may store suffix sums to the deque. That way, each position corresponds to the amount of nodes with a depth larger or equal to the index of the position.

We use small-to-large merging but as the sizes of merged data structures correspond to the depth of the subtrees, rather than to the sizes, the solution runs in $O(n)$ time.

```
#include <deque>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
int n, k1, k2;
vector<int> g[N];
ll answer;

void merge(deque<int> &a, deque<int> &b) {
    if (b.size() > a.size()) swap(a, b);
    auto get = [&](int i) {
        if (i < 0) return a.front();
        if (i >= a.size()) return 0;
        return a[i];
    };
    int bs = b.size();
    b.push_back(0);
    for (int i = 0; i < bs; ++i) {
        ll cur_b = b[i] - b[i + 1];
        answer += cur_b * (get(k1 - i) - get(k2 - i + 1));
    }
    for (int i = 0; i < bs; ++i) {
        a[i] += b[i];
    }
}

deque<int> dfs(int node, int parent) {
    deque<int> ret{1};
    for (int child : g[node]) {
        if (child == parent) continue;
        auto child_depths = dfs(child, node);
        child_depths.push_front(child_depths.front());
        merge(ret, child_depths);
    }
    return ret;
}

int main() {
    cin >> n >> k1 >> k2;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);
    cout << answer << '\n';
}
```

2080 · Fixed-Length Paths I

Task · source analysis length 7,847 characters

Problem statement

Given a tree of n nodes, your task is to count the number of distinct paths that consist of exactly k edges.

Input

The first input line contains two integers n and k: the number of nodes and the path length. The nodes are numbered $1, 2, \ldots, n$.

Then there are $n - 1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: the number of paths.

Constraints

- $1 \le k \le n \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Solution 1: Centroid Decomposition

This problem can be solved with a technique known as centroid decomposition.

The idea is to recursively split the tree into disjoint trees. We start by selecting a node from the tree. Then, we calculate the number of paths of length k which go through the selected node. After this, all unaccounted paths are completely contained in the subtrees of the node. Thus, we may remove the selected node and repeat the same procedure for the resulting new trees.

The solution would be too slow if we each time selected the node at random. The key idea is to pick the node so that it is a centroid of the tree, i.e., a node such that when it is appointed the root of the tree, each subtree has at most $\lfloor n/2 \rfloor$ nodes.

If we choose a centroid node, the sizes of the resulting trees will be at most half of the original size. This means that each node will be part of at most $O(\log n)$ trees. As every node of every tree is processed once, the total time complexity of the solution is $O(n \log n)$.

At each step, a centroid node can be found with a greedy algorithm, which works as follows: root the tree, traverse recursively from the root, each time choosing a child with a subtree size of at least $\lfloor n/2 \rfloor$. If there is no such child, the traversal has arrived at a centroid node.

When we have found a centroid, we can start counting the paths. For this purpose, we use an array `dist`. Each position i of `dist` stores the amount of nodes at distance i from the centroid. To ensure that the counted paths are not contained solely in a single subtree and that each path is only counted once, we count the number of paths ending in a subtree before updating the array `dist` with the distances of the nodes in the subtree.

The `dist` array is reused for all of the individually processed trees. So, we have to reset it between the recursive calls.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
bool deleted[N];
int n, k, sizes[N];
vector<int> g[N];
ll answer;

void dfs_sizes(int node, int parent) {
    sizes[node] = 1;
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_sizes(child, node);
        sizes[node] += sizes[child];
    }
}

int find_centroid(int node, int parent, int n) {
    for (int child : g[node]) {
        if (child != parent && !deleted[child] && sizes[child] > n / 2) {
            return find_centroid(child, node, n);
        }
    }
    return node;
}

int dist[N];

void dfs_add(int node, int parent, int depth, int change) {
    dist[depth] += change;
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_add(child, node, depth + 1, change);
    }
}

void dfs_query(int node, int parent, int depth) {
    if (k - depth >= 0) answer += dist[k - depth];
    for (int child : g[node]) {
        if (child == parent || deleted[child]) continue;
        dfs_query(child, node, depth + 1);
    }
}

void recurse(int node) {
    dfs_sizes(node, -1);
    int r = find_centroid(node, -1, sizes[node]);
    dist[0] = 1;
    for (int child : g[r]) {
        if (!deleted[child]) {
            dfs_query(child, r, 1);
            dfs_add(child, r, 1, 1);
        }
    }
    dfs_add(r, -1, 0, -1);
    deleted[r] = true;
    for (int child : g[r]) {
        if (!deleted[child]) {
            recurse(child);
        }
    }
}

int main() {
    cin >> n >> k;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    recurse(1);
    cout << answer << '\n';
}
```

Solution 2: Small-To-Large Merging

There is another, simpler solution to this problem. It uses a technique sometimes known as *small-to-large merging*. The implementation has only one tree traversal.

After rooting the tree, we make a depth-first traversal of it. During the traversal, we calculate the number of paths through each node. Any path in a tree has a highest node: a node that is the closest to the root. We will calculate each path only once, at its highest node.

To make sure we only count paths of length k, we use the fact that the distance between tree nodes a and b equals

$$\text{depth}(a) + \text{depth}(b) - 2 \cdot \text{depth}(c),$$

where c is the highest node on the path between a and b, also known as the lowest common ancestor of a and b. With this, finding the number of paths of length k through node x corresponds to finding the number of ways to pick two nodes a and b from different subtrees of x with the following condition:

$$\text{depth}(a) + \text{depth}(b) - 2 \cdot \text{depth}(x) = k.$$

We create a `map` for each node, which stores the depths of the nodes in the subtree of the node. We use the technique of *small-to-large* merging for merging a node's `map` to its parent's `map` efficiently: if a map of a child is larger than the map of its parent, we swap the data structures before merging. This means that instead of moving elements from the map of the child to the map of its parent, we move them from the smaller map to the larger map.

It can be proven that each element will be moved at most $O(\log n)$ times making the total time complexity of the solution $O(n \log^2 n)$.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
int n, k;
vector<int> g[N];
ll answer;

map<int, int> dfs(int node, int parent, int depth) {
    map<int, int> dist;
    dist[depth] = 1;
    for (int child : g[node]) {
        if (child == parent) continue;
        auto child_dist = dfs(child, node, depth + 1);
        if (child_dist.size() > dist.size()) {
            swap(dist, child_dist);
        }
        for (auto [d, amt] : child_dist) {
            answer += 1ll * dist[k - d + 2 * depth] * amt;
        }
        for (auto [d, amt] : child_dist) {
            dist[d] += amt;
        }
    }
    return dist;
}

int main() {
    cin >> n >> k;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1, 0);
    cout << answer << '\n';
}
```

Solution 3: Improved merge

Instead of using a `map` to store information about the depths of the nodes in each subtree, we use a `deque`. Other than that, this solution works in a way that is very similar to the previous solution.

Note that, in contrast to the previous solution, the `deque` indexes represent depths relative to the current node, as opposed to relative to the root node. Also note that the size of the merged data structure is proportional to the depth of the node, instead of the subtree size of the node.

It may come as a surprise that this solution runs in $O(n)$ time.

This proof was first presented by Codeforces user neal.

The time complexity of constructing the deques is linear because each insertion works in $O(1)$ time and each node corresponds to at most two insertions. Thus, we need to focus only on the `merge` function.

A `merge` operation results in a `deque` with a size equal to the size of the larger of the merged `deques`, as opposed to the sum of the two sizes. Thus, the smaller `deque` is "consumed" and its elements will not amount to any more work. As there are only $O(n)$ `deque` elements in total and each of them is consumed at most once, the total time complexity is linear.

```
#include <deque>
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int N = 200001;
int n, k;
vector<int> g[N];
ll answer;

void merge(deque<int> &a, deque<int> &b) {
    if (b.size() > a.size()) swap(a, b);
    int bs = b.size();
    for (int i = 0; i < bs; ++i) {
        if (0 <= k - i && k - i < (int)a.size()) {
            answer += b[i] * a[k - i];
        }
    }
    for (int i = 0; i < bs; ++i) {
        a[i] += b[i];
    }
}

deque<int> dfs(int node, int parent) {
    deque<int> ret{1};
    for (int child : g[node]) {
        if (child == parent) continue;
        auto child_depths = dfs(child, node);
        child_depths.push_front(0);
        merge(ret, child_depths);
    }
    return ret;
}

int main() {
    cin >> n >> k;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);
    cout << answer << '\n';
}
```

2079 · Finding a Centroid

Task · source analysis length 4,813 characters

Problem statement

Given a tree of n nodes, your task is to find a *centroid*, i.e., a node such that when it is appointed the root of the tree, each subtree has at most $\lfloor n/2 \rfloor$ nodes.

Input

The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$.

Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: a centroid node. If there are several possibilities, you can choose any of them.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le a, b \le n$

First approach

The problem becomes easier to solve when we root the tree arbitrarily. After that, we calculate the size of the subtree of each node and the maximum subtree size among the children of each node.

The arrays `sz` and `max_child` store the size of the subtree of each node and the maximum subtree size among the children of each node, respectively.

When we remove a node from the tree, a number of separate components will remain. Throughout this analysis we will refer to the component corresponding to the parent of a node as the *parent's component* of the node.

Note that the parent's component can be the largest of the remaining components. The array `max_child` does not take this case into account and thus, it needs to be checked separately. The size of the parent's component is equal to $n - \texttt{sz}[i]$ for any node i as each of the n nodes of the tree is part of either the subtree of i or the parent's component.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 200001;
vector<int> g[N];
int sz[N], max_child[N];

void dfs(int node, int parent) {
    sz[node] = 1;
    for (int child : g[node]) {
        if (child == parent) continue;
        dfs(child, node);
        sz[node] += sz[child];
        max_child[node] = max(max_child[node], sz[child]);
    }
```

```
}

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);

    for (int i = 1; i <= n; ++i) {
        if (max(max_child[i], n - sz[i]) <= n / 2) {
            cout << i << '\n';
            break;
        }
    }
}
```

Second approach

There exists a recursive approach for this problem where we keep track of the size of the parent's component as we traverse the tree.

This approach is useful in the case that we do not know the exact set of nodes the tree consists of as is sometimes the case when constructing a centroid decomposition of the tree.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 200001;
vector<int> g[N];
int sz[N], max_child[N];
int n;

void dfs(int node, int parent) {
    sz[node] = 1;
    for (int child : g[node]) {
        if (child == parent) continue;
        dfs(child, node);
        sz[node] += sz[child];
        max_child[node] = max(max_child[node], sz[child]);
    }
}

int centroid(int node, int parent, int above) {
    if (max(above, max_child[node]) <= n / 2) {
        return node;
    }
    for (int child : g[node]) {
        if (child == parent) continue;
        int ans = centroid(child, node, above + sz[node] - sz[child]);
        if (ans != -1) return ans;
    }
    return -1;
}

int main() {
    cin >> n;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);

    cout << centroid(1, -1, 0) << '\n';
}
```

Third approach

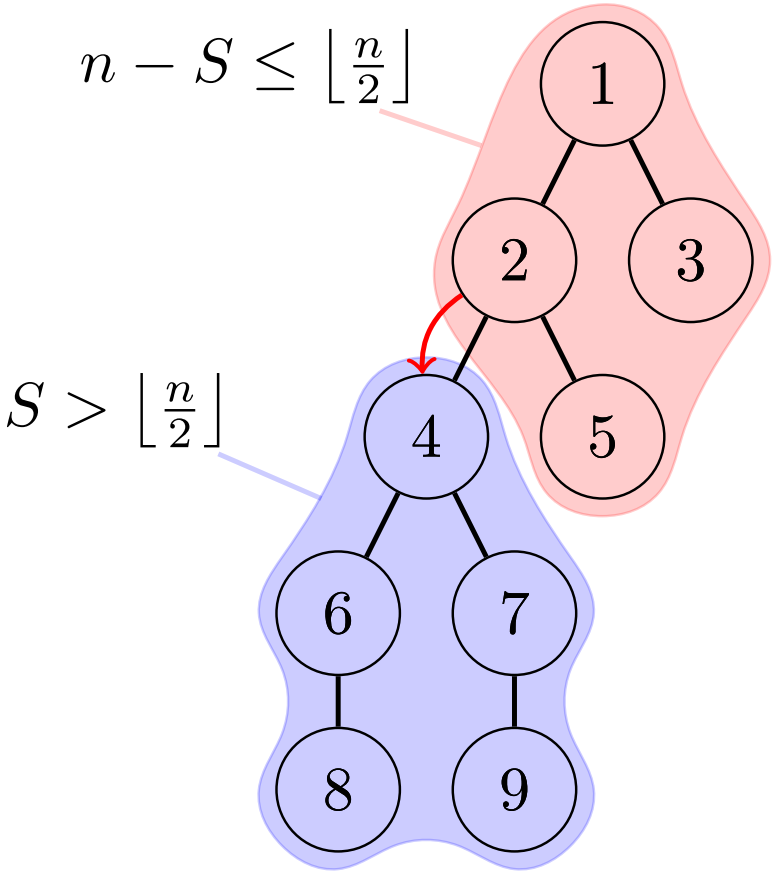
The previous solution can be improved even further. Turns out a greedy approach works.

First, we root the tree. Then, starting from the root, we recursively move to a child with a subtree size larger than $\left\lfloor \frac{n}{2} \right\rfloor$. When we arrive to a node which has no such children, the node is a centroid of the tree.

Proof for the algorithm

As the size of the current subtree decreases with each move, the traversal will eventually arrive to a node with no child with a subtree size larger than $\left\lfloor \frac{n}{2} \right\rfloor$. We just have to prove that the found node is, in fact, a centroid of the tree.

We know that no child of the chosen node has a child with a subtree size too large, which means that the only case in which the chosen node is not a centroid is when the size of the parent's component is too large.



If the traversal consists of only one node, then the root is a centroid node as it does not have a parent. Otherwise, let us analyze the very last step of the search, moving from the previous node to the last node (in the figure from 2 to 4).

Let the subtree size of the last node (4 in the figure) be S. Then, the size of the parent's component has a size of $n - S$. By definition, we know that $S > \left\lfloor \frac{n}{2} \right\rfloor$. From these pieces of information, we can work out that

$$S > \left\lfloor \frac{n}{2} \right\rfloor \iff n - S < n - \left\lfloor \frac{n}{2} \right\rfloor \implies n - S \leq \left\lfloor \frac{n}{2} \right\rfloor. \square$$

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 200001;
vector<int> g[N];
int sz[N];
int n;

void dfs(int node, int parent) {
    sz[node] = 1;
    for (int child : g[node]) {
        if (child == parent) continue;
        dfs(child, node);
        sz[node] += sz[child];
    }
}
```

```
int centroid(int node, int parent) {
    for (int child : g[node]) {
        if (child != parent && sz[child] > n / 2) {
            return centroid(child, node);
        }
    }
    return node;
}

int main() {
    cin >> n;

    for (int i = 1; i <= n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, -1);

    cout << centroid(1, -1) << '\n';
}
```

2077 · Necessary Cities

Task · source analysis length 2,663 characters

Problem statement

There are n cities and m roads between them. There is a route between any two cities.

A city is called *necessary* if there is no route between some other two cities after removing that city $and adjacent roads$. Your task is to find all necessary cities.

Input

The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$.

After this, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b. There is at most one road between two cities, and every road connects two distinct cities.

Output

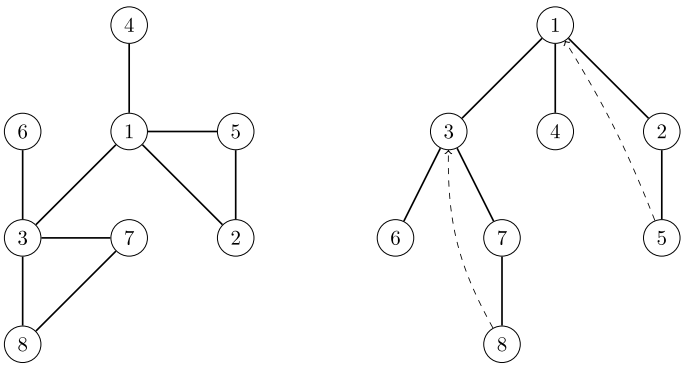
First print an integer k: the number of necessary cities. After that, print a list of k cities. You may print the cities in any order.

Constraints

- $2 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Necessary cities are more commonly referred to as the *articulation points* of a graph. It is useful to think of the graph in terms of its DFS trees. A DFS tree represents the order in which we visit the nodes when we do a depth-first search. The root of the tree is the first node of the traversal and subsequent nodes are placed as children of the nodes from which they are visited.

Here is an example of a graph and one of its DFS trees where the traversal was started at node 1.



From the DFS tree, it can be seen that the nodes 1 and 3 are the articulation points of the graph.

The edges of the DFS tree form a subset of the edges of the original graph. Let us call the edges that are part of the tree *tree edges* and the rest of them *back-edges*. There is a simple condition for checking whether a

node is an articulation point which works for all nodes expect the root of the DFS tree: The node x is an articulation point of the graph if it has a child from whose subtree there are no *back-edges* to any of the x's strict ancestors. In the example graph, 3 is an articulation point as neither of its children's subtrees contain *back-edges* to 3's ancestors. Lastly, the root of the DFS tree is an articulation point if the has more than one child.

The implementation details of the algorithm are similar to Tarjan's algorithm for finding the strongly connected components of a directed graph. We will start by assigning each node a depth value which tells us its distance from the root node in the DFS tree. We will then find, for each node, the minimum depth of a node that is reachable by a back-edge from the subtree of the node. The algorithm can be implemented in linear time.

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 1e5 + 1;
vector<int> g[N];
vector<int> answer;
bool visited[N];
int depth[N], min_depth[N];

void dfs(int node, int prev) {
    bool is_root = prev == 0;
    bool found = false;
    int children = 0;
    visited[node] = true;
    min_depth[node] = depth[node];
    for (int nxt : g[node]) {
        if (nxt == prev) continue;
        if (visited[nxt]) {
            min_depth[node] = min(min_depth[node], depth[nxt]);
        } else {
            depth[nxt] = depth[node] + 1;
            dfs(nxt, node);
            min_depth[node] = min(min_depth[node], min_depth[nxt]);
            children++;
            if (min_depth[nxt] >= depth[node]) found = true;
        }
    }
    if ((is_root && children > 1) || (!is_root && found)) {
        answer.push_back(node);
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, 0);

    cout << answer.size() << '\n';
    for (int node : answer) {
        cout << node << ' ';
    }
    cout << '\n';
}
```

2076 · Necessary Roads

Task · source analysis length 2,777 characters

Problem statement

There are n cities and m roads between them. There is a route between any two cities.

A road is called *necessary* if there is no route between some two cities after removing that road. Your task is to find all necessary roads.

Input

The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$.

After this, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b. There is at most one road between two cities, and every road connects two distinct cities.

Output

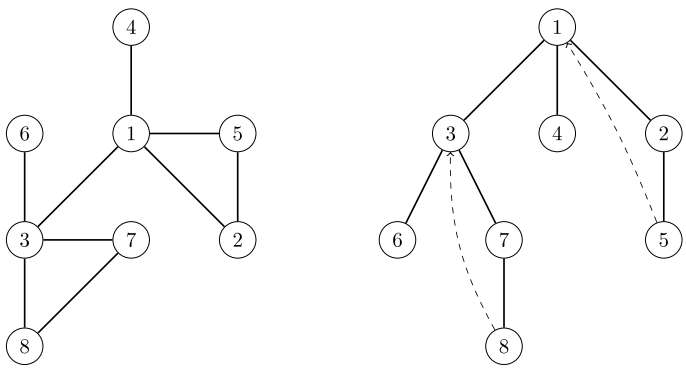
First print an integer k: the number of necessary roads. After that, print k lines that describe the roads. You may print the roads in any order.

Constraints

- $2 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

It is worth acknowledging that the *necessary roads* are more commonly known as bridges. To find the bridges of the given graph, think of the graph in terms of its DFS trees. A DFS tree represents the order we visit the nodes when we do a depth-first search. The root of the tree is the first node of the traversal and subsequent nodes are placed as children of the nodes from which they are visited.

Here is an example of a graph and one of its DFS trees where the traversal was started at node 1.



The bridges of the graph are $(3, 6, 1, 3)$ and $(1, 4)$.

Note that there may be more than one way to visit the nodes and thus, more than one DFS tree. However, the properties we are interested in apply to all DFS trees. The edges of the DFS tree form a subset of the edges of

the original graph. The edges that are not part of the tree are called *back-edges*. It can be proven that back-edges always go from a node to its ancestor.

Notice that bridge edges are always *tree edges*, that is, they are always part of the DFS tree as otherwise, if a back-edge was a bridge, removing the edge would not break the graph into two separate components. Bridges also have another important property. A tree edge $a \rightarrow b$ is bridge if there are no back-edges from b's subtree to a or any a's ancestors. In the example, the edge $(1, 3)$ is a bridge as there are no back-edges from the subtree of 3 to 1.

The implementation details of the algorithm are similar to Tarjan's algorithm for finding the strongly connected components of a directed graph. We will start by assigning each node a depth value which tells us its distance from the root node in the DFS tree. We will then find, for each node, the minimum depth of a node that is reachable by a back-edge from the subtree of the node. The minimum reachable depth determines which edges are bridges. The implementation has a time complexity of $O(n)$

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 1e5 + 1;
vector<int> g[N];
vector<pair<int, int>> bridges;
bool visited[N];
int depth[N], min_depth[N];

void dfs(int node, int prev) {
    visited[node] = true;
    min_depth[node] = depth[node];
    for (int nxt : g[node]) {
        if (nxt == prev) continue;
        if (visited[nxt]) {
            min_depth[node] = min(min_depth[node], depth[nxt]);
        } else {
            depth[nxt] = depth[node] + 1;
            dfs(nxt, node);
            min_depth[node] = min(min_depth[node], min_depth[nxt]);
            if (min_depth[nxt] > depth[node]) {
                bridges.emplace_back(node, nxt);
            }
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 1; i <= m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    dfs(1, 0);

    cout << bridges.size() << '\n';
    for (auto [a, b] : bridges) {
        cout << a << ' ' << b << '\n';
    }
}
```

References

- Bridge *Wikipedia*

2072 · Cut and Paste

Task · source analysis length 2,824 characters

Problem statement

Given a string, your task is to process operations where you cut a substring and paste it to the end of the string. What is the final string after all the operations?

Input

The first input line has two integers n and m: the length of the string and the number of operations. The characters of the string are numbered $1, 2, \ldots, n$.

The next line has a string of length n that consists of characters A–Z.

Finally, there are m lines that describe the operations. Each line has two integers a and b: you cut a substring from position a to position b.

Output

Print the final string after all the operations.

Constraints

- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le a \le b \le n$

A treap is a binary search search tree which may be used to simulate the cut/paste operations efficiently.

In a treap, each node is assigned a random weight, and the structure of the tree is determined by the heap property, that is, the weight of a parent is always greater or equal to the weights of its children. This property guarantees that the height of the tree is very likely proportional to the logarithm of the number of nodes, which ensures that split/merge operations are efficient. Merge and split operations work, on average, in $O(\log n)$ time, where n is the size of the tree.

The time complexity of our solution is $O((n + m \log n))$ as it takes $O(n \log n)$ time to build the tree and $O(m \log n)$ time to process the queries. In the implementation, we reserve an array of treap nodes beforehand. This way, we do not have to worry about freeing memory, and additionally it is faster than allocating each node individually.

```
#include <chrono>
#include <iostream>
#include <random>
#include <tuple>
using namespace std;

mt19937 rng(chrono::high_resolution_clock::now().time_since_epoch().count());

struct Treap {
    Treap *left = nullptr, *right = nullptr;
    int v, weight, sz = 1;
    Treap() {}
    Treap(int v) : v(v), weight(rng()) {}

    static void update(Treap *node) {
        node->sz = 1;
        if (node->left) node->sz += node->left->sz;
        if (node->right) node->sz += node->right->sz;
    }
```

```
    // Merges the treaps a and b.
    static Treap *merge(Treap *a, Treap *b) {
        if (!a) return b;
        if (!b) return a;
        Treap *r;
        if (a->weight < b->weight) {
            a->right = merge(a->right, b);
            r = a;
        } else {
            b->left = merge(a, b->left);
            r = b;
        }
        update(r);
        return r;
    }

    // Splits the treap into two treaps.
    // The first of the returned treaps will contain the k first nodes of a.
    static pair<Treap *, Treap *> split(Treap *a, int k) {
        if (!a) return {nullptr, nullptr};
        int al = a->left ? a->left->sz : 0;
        Treap *r;
        if (al >= k) {
            tie(r, a->left) = split(a->left, k);
            update(a);
            return {r, a};
        }
        tie(a->right, r) = split(a->right, k - al - 1);
        update(a);
        return {a, r};
    }

    static void print(Treap *a, char endl = '\n') {
        if (a->left) print(a->left, '\0');
        cout << (char)a->v;
        if (a->right) print(a->right, '\0');
        if (endl) cout << endl;
    }
};

Treap *new_treap(int v) {
    const int pool_sz = 2e5;
    static Treap pool[pool_sz];
    static Treap *pool_ptr = pool;

    *pool_ptr = Treap(v);
    return pool_ptr++;
}

int main() {
    Treap *root = nullptr;

    int n, m;
    cin >> n >> m;

    string s;
    cin >> s;

    for (char c : s) {
        root = Treap::merge(root, new_treap(c));
    }

    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        auto [r0, r1] = Treap::split(root, a - 1);
        auto [r2, r3] = Treap::split(r1, b - a + 1);
        root = Treap::merge(r0, r3);
        root = Treap::merge(root, r2);
    }

    Treap::print(root);
}
```

References

- Treap *Wikipedia*

2064 · Bracket Sequences I

Task · source analysis length 1,730 characters

Problem statement

Your task is to calculate the number of valid bracket sequences of length n. For example, when $n = 6$, there are 5 sequences:

- ( ) ( ) ( )
- ( ) (( ))
- (( )) ( )
- ((( )))
- (( ) ( ))

Input

The only input line has an integer n.

Output

Print the number of sequences modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10^6$

If n is odd, we can't create any valid bracket sequence, so we assume that n is even. We focus on bracket sequences that consist of $\frac{n}{2}$ left brackets and $\frac{n}{2}$ right brackets. The total number of such sequences is

$$\binom{n}{\frac{n}{2}}.$$

Then, let us calculate the number of invalid bracket sequences that consist of $\frac{n}{2}$ left brackets and $\frac{n}{2}$ right brackets. A sequence is invalid if some prefix of it contains more right brackets than left brackets.

If we reverse each bracket in the minimal invalid prefix, we get another bracket sequence that consists of $n+1$ left brackets and $n-1$ right brackets. Moreover, there is a one-to-one mapping between invalid bracket sequences and bracket sequences that consist of $n+1$ left brackets and $n-1$ right brackets. Thus, the number of invalid bracket sequences is

$$\binom{n}{\frac{n}{2}+1}.$$

We can now calculate the answer using the formula

$$\binom{n}{\frac{n}{2}} - \binom{n}{\frac{n}{2}+1} = \frac{2}{n+2}\binom{n}{\frac{n}{2}}$$

where we subtract the number of invalid bracket sequences from the number of all bracket sequences.

```
#include <iostream>
using namespace std;
```

```
using ll = long long;
const int M = 1000000007;
const int N = 1000000;

int power(int a, int b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

int inv(int x) {
    return power(x, M - 2);
}

ll fac[N + 1];

int ncr(int a, int b) {
    return fac[a] * inv(fac[b] * fac[a - b] % M) % M;
}

int main() {
    fac[0] = 1;
    for (int i = 1; i <= N; i++) {
        fac[i] = fac[i - 1] * i % M;
    }

    int n;
    cin >> n;
    if (n % 2 == 0) {
        ll answer = 2LL * ncr(n, n / 2) * inv(n + 2) % M;
        cout << answer << "\n";
    } else {
        cout << "0\n";
    }
}
```

1755 · Palindrome Reorder

Task · source analysis length 1,034 characters

Problem statement

Given a string, your task is to reorder its letters in such a way that it becomes a palindrome $i.e., it reads the same forwards and backwards$.

Input

The only input line has a string of length n consisting of characters A–Z.

Output

Print a palindrome consisting of the characters of the original string. You may print any valid solution. If there are no solutions, print "NO SOLUTION".

Constraints

- $1 \le n \le 10^6$

Let $\text{count}(c)$ denote the number of times letter c appears in the string. We can create a palindrome if $\text{count}(c)$ is odd for at most one letter c.

To create the palindrome, we first go from A to Z and add $\lfloor \text{count}(c/2 \rfloor$ copies of each letter c to the string. Then, if some letter appears an odd number of times, we add it as the center letter. Finally, we go from Z to A and add $\lfloor \text{count}(c/2 \rfloor$ copies of each letter c to the string.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main() {
    string s;
    cin >> s;

    map<char, int> count;
    for (auto c : s) {
        count[c]++;
    }

    int odds = 0;
    char center;
    for (char c = 'A'; c <= 'Z'; c++) {
        if (count[c] % 2 == 1) {
            odds++;
            center = c;
        }
    }

    if (odds > 1) {
        cout << "NO SOLUTION\n";
    } else {
        for (char c = 'A'; c <= 'Z'; c++) {
            cout << string(count[c] / 2, c);
        }
        if (odds) {
            cout << center;
        }
        for (char c = 'Z'; c >= 'A'; c--) {
            cout << string(count[c] / 2, c);
```

```
        }
        cout << "\n";
    }
}
```

1753 · String Matching

Task · source analysis length 2,943 characters

Problem statement

Given a string and a pattern, your task is to count the number of positions where the pattern occurs in the string.

Input

The first input line has a string of length n, and the second input line has a pattern of length m. Both of them consist of characters a–z.

Output

Print one integer: the number of occurrences.

Constraints

- $1 \le n, m \le 10^6$

We are given a string s of length n and a pattern p of length m, and we want to count the number of times p appears in s.

We discuss two ways to solve the problem in $O(n+m)$ time.

Solution 1 $Z-algorithm$

One way to solve the problem is to use the Z-algorithm. Given a string, the algorithm constructs a Z-array that contains, for each suffix of the string, the maximum length of a prefix that is also a prefix of the entire string. The Z-algorithm works in linear time.

We create a new string $p\$s$ and construct its Z-array using the Z-algorithm. After that, we go through the array and find the positions that have value m. Each such position corresponds to an occurrence of p in s.

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> z_algo(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        z[i] = max(0, min(z[i - l], r - i));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

int main() {
    string s, p;
    cin >> s;
    int n = s.size();
    cin >> p;
```

```
    int m = p.size();

    string t = p + "$" + s;
    auto z = z_algo(t);
    int count = 0;
    for (int i = 0; i < n + m + 1; i++) {
        if (z[i] == m) count++;
    }
    cout << count << "\n";
}
```

Solution 2 *hashing*

Another way to solve the problem is to use hashing. After preprocessing a string, we can calculate the hash value of any substring on $O(1)$ time. Thus, we can check all positions where the pattern can appear and compare the hash values.

Warning: a code that uses hashing may fail due to collisions $two strings are different but have the same hash value$.

It is possible to deliberately find an input which causes a hashing solution to break. Some care has been taken to protect the solution from that. The coefficient used in the polynomial rolling hash algorithm is selected at random and the random number generator uses the amount of *ticks* since epoch as its seed.

```
#include <chrono>
#include <iostream>
#include <random>
#include <vector>
using namespace std;
using ll = long long;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
const ll M = 991831889;
const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9 * M)(rng);

struct HashString {
    int n;
    vector<ll> pows, sums;

    HashString(string s) : n(s.size()), pows(n + 1, 1), sums(n + 1) {
        for (int i = 1; i <= n; i++) {
            pows[i] = pows[i - 1] * C % M;
            sums[i] = (sums[i - 1] * C + s[i - 1]) % M;
        }
    }

    // Returns the hash of the substring [l, r)
    ll hash(int l, int r) {
        ll h = sums[r] - sums[l] * pows[r - l];
        return (h % M + M) % M;
    }
};

int main() {
    string s, p;
    cin >> s;
    int n = s.size();
    cin >> p;
    int m = p.size();

    HashString h(s);
    ll target = HashString(p).hash(0, m);

    int count = 0;
    for (int i = 0; i < n - m + 1; i++) {
        if (h.hash(i, i + m) == target) {
            count++;
        }
    }
    cout << count << "\n";
}
```

References

- Wikipedia, Polynomial rolling hash

1751 · Planets Cycles

Task · source analysis length 2,035 characters

Problem statement

You are playing a game consisting of n planets. Each planet has a teleporter to another planet $ortheplanetitself$.

You start on a planet and then travel through teleporters until you reach a planet that you have already visited before.

Your task is to calculate for each planet the number of teleportations there would be if you started on that planet.

Input

The first input line has an integer n: the number of planets. The planets are numbered $1, 2, \ldots, n$.

The second line has n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. It is possible that $t_i = i$.

Output

Print n integers according to the problem statement.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le t_i \le n$

We start by noticing that the planets and teleporters form a graph where every connected component is formed of a cycle and trees $possiblyconsistingoffonlyoneplanet$ starting from each planet in the cycle. The answer for planet x is then the length of the cycle plus the number of teleports it takes to reach the cycle.

We will process every component separately. We start processing a component by finding the cycle $inthissolution, usingFloyd'stortoiseandharealgorithm$. We will then run a depth-first search DFS from a cycle planet in a graph where we have reversed all teleports. We additionally give the DFS function the parameter $\mathbf{parent_answer}$, the answer for the parent planet. For the first DFS function call this is just the number of planets in the cycle. If the current planet is part of the cycle, the answer for the current planet is just $\mathbf{parent_answer}$. If the current planet is not part of the cycle, the answer is $\mathbf{parent_answer} + 1$.

The time complexity of the solution is $O(n)$.

```
#include <iostream>
#include <vector>

using namespace std;
const int N = 200001;

int teleporter[N];
vector<int> rev_graph[N];
bool cycle[N];
bool visited[N];
int answer[N];

// returns a pair (cycle node, length of the cycle)
std::pair<int, int> find_cycle(int x) {
    int a = x;
```

```
    int b = teleporter[x];
    while (a != b) {
        a = teleporter[a];
        b = teleporter[teleporter[b]];
    }
    int size = 0;
    do {
        ++size;
        cycle[a] = true;
        a = teleporter[a];
    } while (a != b);
    return {a, size};
}

void dfs(int x, int parent_answer) {
    if (visited[x]) return;
    visited[x] = true;
    answer[x] = parent_answer;
    if (!cycle[x]) ++answer[x];
    for (int y : rev_graph[x]) {
        dfs(y, answer[x]);
    }
}

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; ++i) {
        cin >> teleporter[i];
        rev_graph[teleporter[i]].push_back(i);
    }

    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            auto [x, size] = find_cycle(i);
            dfs(x, size);
        }
    }

    for (int i = 1; i <= n; ++i) {
        cout << answer[i] << ' ';
    }
    cout << '\n';
}
```

1750 · Planets Queries I

Task · source analysis length 1,583 characters

Problem statement

You are playing a game consisting of n planets. Each planet has a teleporter to another planet $or the planet itself$.

Your task is to process q queries of the form: when you begin on planet x and travel through k teleporters, which planet will you reach?

Input

The first input line has two integers n and q: the number of planets and queries. The planets are numbered $1, 2, \ldots, n$.

The second line has n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. It is possible that $t_i = i$.

Finally, there are q lines describing the queries. Each line has two integers x and k: you start on planet x and travel through k teleporters.

Output

Print the answer to each query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le t_i \le n$
- $1 \le x \le n$
- $0 \le k \le 10^9$

We start solving the problem by precomputing the destinations for every planet when the number of teleports is a power of two. We will store the destination reached from planet x after traveling through 2^l teleports in $\mathrm{succ}[l][x]$. We fill the array layer by layer in the order of increasing l. For $l = 0$ we simply set $\mathrm{succ}[0][x]$ to the planet reached from x by going through one teleport. For $l > 0$ we set $\mathrm{succ}[l][x] = \mathrm{succ}[l-1][\mathrm{succ}[l-1][x]]$.

To solve the case where the number of teleports k is not a power of two, we will use the binary representation of k to find powers of two $s_1, \ldots, s_m$ such that $\sum_i s_i = k$. We can then find the final destination by using the succ array to take s_1 steps, s_2 steps, and so on.

Let's denote the maximum value of k by K ($K = 10^9$ in the task statement). Building the succ array takes $O(n \log K)$ time and answering the queries takes $O(q \log K)$. Therefore the total time complexity of the solution is $O((n + q$ \log K)).

```
#include <iostream>
using namespace std;

const int N = 200001;
const int M = 30;

int succ[M][N];

int main() {
```

```
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) {
        cin >> succ[0][i];
    }

    for (int i = 1; i < M; ++i) {
        for (int j = 1; j <= n; ++j) {
            succ[i][j] = succ[i - 1][succ[i - 1][j]];
        }
    }

    for (int i = 0; i < q; ++i) {
        int x, k;
        cin >> x >> k;
        for (int j = 0; j < M; ++j) {
            if (k & (1 << j)) {
                x = succ[j][x];
            }
        }
        std::cout << x << '\n';
    }
}
```

References

- CPHB *Competitive Programmer's Handbook*, Chapter 16.3

1749 · List Removals

Task · source analysis length 3,181 characters

Problem statement

You are given a list consisting of n integers. Your task is to remove elements from the list at given positions, and report the removed elements.

Input

The first input line has an integer n: the initial size of the list. During the process, the elements are numbered $1, 2, \ldots, k$ where k is the current size of the list.

The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list.

The last line has n integers $p_1, p_2, \ldots, p_n$: the positions of the elements to be removed.

Output

Print the elements in the order they are removed.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le p_i \le n - i + 1$

First approach: Segment tree

We can use a segment tree to solve this problem. In the tree, active $notyetremoved$ elements are represented by a one and inactive $removed$ by a zero.

To find the kth active value, we will do a binary search in the tree.

```
#include <iostream>
using namespace std;

const int TREE_SIZE = 1 << 18;
int tree[TREE_SIZE * 2];

// Changes the value in the position i to x.
void change(int i, int x) {
    i += TREE_SIZE;
    tree[i] = x;
    while (i > 1) {
        i /= 2;
        tree[i] = tree[i * 2] + tree[i * 2 + 1];
    }
}

// Returns the index of the kth 'active' position.
int query(int k) {
    int s = 1;
    while (s < TREE_SIZE) {
        // The index of the left child is s * 2.
        // The index of the right child is s * 2 + 1.
        if (tree[s * 2] >= k) {
            s = s * 2;
        } else {
```

```
            k -= tree[s * 2];
            s = s * 2 + 1;
        }
    }
    return s - TREE_SIZE;
}

const int N = 200001;
int x[N];

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; ++i) {
        cin >> x[i];
        change(i, 1);
    }

    for (int i = 1; i <= n; ++i) {
        int p;
        cin >> p;
        int pos = query(p);
        cout << x[pos] << ' ';
        change(pos, 0);
    }
    cout << '\n';
}
```

References for segment trees

- CPHB *Competitive Programmer's Handbook*, Chapter 9
- CP-Algorithms, Segment Tree

Second approach: Policy-based data structures

The g++ compiler – the one used on CSES – has some built-in data structures that are not part of the C++ standard library. One of these data structures is a binary tree, similar to `std::set`. This binary search tree provides some useful additional functionality, including a function for finding the i-th element in the set.

With this function, we may solve this problem without using a segment tree.

```
#include <iostream>
using namespace std;

// These lines are needed for the policy-based data structures.
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using indexed_set =
    tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;

const int N = 200001;
int x[N];

int main() {
    int n;
    cin >> n;

    indexed_set<int> s;

    for (int i = 1; i <= n; ++i) {
        cin >> x[i];
        s.insert(i);
    }

    for (int i = 1; i <= n; ++i) {
        int p;
        cin >> p;
        // Note, that the function `find_by_order` expects
        // a 0-indexed position.
        auto it = s.find_by_order(p - 1);
        int pos = *it;
        cout << x[pos] << ' ';
        s.erase(it);
    }
    cout << '\n';
}
```

Note that these unorthodox data structures are seldom used in the indented solution to a competitive programming problem. That said they are worth knowing as they sometimes come in handy and can be used to achieve better time complexities and much shorter implementations.

Lastly, the policy-based binary search tree has a notable pitfall: there exists no policy-based alternative for `std::multiset`. To get around this problem `std::pairs` can be stored in the tree with the second element of each pair being a unique value.

References for policy-based data structures

- CPHB $\mathit{Competitive Programmer's Handbook}$, Chapter 4
- Codeforces, *C++ STL: Policy based data structures* by adamant
- Codeforces, *C++ STL: Policy based data structures. Part 2* by adamant
- The GNU C++ Library Manual, Chapter 21. Policy-Based Data Structures

1748 · Increasing Subsequence II

Task · source analysis length 1,708 characters

Problem statement

Given an array of n integers, your task is to calculate the number of increasing subsequences it contains. If two subsequences have the same values but in different positions in the array, they are counted separately.

Input

The first input line has an integer n: the size of the array.

The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output

Print one integer: the number of increasing subsequences modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

We go through the array from left to right and calculate the number of increasing subsequences ending at each array position.

Consider an array position i that contains the element x. In this situation, we count the number of increasing subsequences that end before position i and whose last element is less than x. We then add one to that number to obtain the number of increasing subsequences ending at position i.

We use a segment tree to implement the algorithm efficiently. Since the array values can be large, we apply index compression with the segment tree.

The resulting algorithm works in $O(n \log n)$ time.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

const int M = 1000000007;
const int N = 1 << 18;
int tree[2 * N];

void inc_val(int pos, int value) {
    pos += N;
    tree[pos] = (tree[pos] + value) % M;
    for (pos /= 2; pos >= 1; pos /= 2) {
        tree[pos] = (tree[pos * 2] + tree[pos * 2 + 1]) % M;
    }
}

int find_sum(int a, int b) {
    a += N;
    b += N;
    int sum = 0;
    while (a <= b) {
        if (a % 2 == 1) sum = (sum + tree[a++]) % M;
        if (b % 2 == 0) sum = (sum + tree[b--]) % M;
        a /= 2;
```

```
        b /= 2;
    }
    return sum;
}

int main() {
    int n;
    cin >> n;

    vector<pair<int, int>> pairs(n);
    for (int i = 0; i < n; i++) {
        cin >> pairs[i].first;
        pairs[i].second = i;
    }
    sort(pairs.begin(), pairs.end());

    vector<int> values(n);
    int new_value = 1;
    for (int i = 0; i < n; i++) {
        if (i != 0 && pairs[i - 1].first != pairs[i].first) {
            new_value++;
        }
        values[pairs[i].second] = new_value;
    }

    int total = 0;
    for (int i = 0; i < n; i++) {
        int result = find_sum(0, values[i] - 1) + 1;
        inc_val(values[i], result);
        total = (total + result) % M;
    }
    cout << total << "\n";
}
```

1746 · Array Description

Task · source analysis length 1,710 characters

Problem statement

You know that an array has n integers between 1 and m, and the absolute difference between two adjacent values is at most 1.

Given a description of the array where some values may be unknown, your task is to count the number of arrays that match the description.

Input

The first input line has two integers n and m: the array size and the upper bound for each value.

The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Value 0 denotes an unknown value.

Output

Print one integer: the number of arrays modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 100$
- $0 \le x_i \le m$

This problem can be solved with dynamic programming. Let $\mathbf{count}(i, j)$ count the valid combinations for values in the array at indices $1, \ldots, i$, such that the last of them, at index i, is equal to j.

Let us define $\mathbf{count}(1, j)$ as a base case: If $x_1 = 0$, then all values $1 \le j \le m$ are possible. Otherwise only $j = x_1$ is possible.

$$\mathrm{count}(1, j) = \begin{cases} 1 & \text{if } x_1 = 0 \text{ or } x_1 = j \\ 0 & \text{otherwise} \end{cases}$$

Assuming that the array is valid up to the index $i - 1$, it is enough to ensure that the values at indices $i - 1$ and i have absolute difference at most one. The amount of combinations is set to zero if j does not match the description.

$$\mathrm{count}(i, j) = \begin{cases} \sum_{k=-1}^{1} \mathrm{count}(i-1, j+k) & \text{if } x_i = 0 \text{ or } x_i = j \\ 0 & \text{otherwise} \end{cases}$$

This makes for a time complexity of $O(nm)$.

In the implementation below, the dynamic programming array is padded with zeros at `count[i][0]` and `count[i][m + 1]` to avoid going out of bounds.

```
#include <iostream>
#include <vector>
using namespace std;
const int M = 1000000007;
const int N = 100001;
```

```
int count[N][102];

int main() {
    int n, m;
    cin >> n >> m;

    vector<int> x(n + 1);
    for (int i = 1; i <= n; ++i) cin >> x[i];

    for (int j = 1; j <= m; ++j) {
        if (x[1] == 0 || x[1] == j) {
            count[1][j] = 1;
        }
    }
    for (int i = 2; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (x[i] == 0 || x[i] == j) {
                for (int k = -1; k <= 1; ++k) {
                    count[i][j] = (count[i][j] + count[i - 1][j + k]) % M;
                }
            }
        }
    }

    int result = 0;
    for (int j = 1; j <= m; ++j) {
        result = (result + count[n][j]) % M;
    }
    cout << result << "\n";
}
```

1745 · Money Sums

Task · source analysis length 1,501 characters

Problem statement

You have n coins with certain values. Your task is to find all money sums you can create using these coins.

Input

The first input line has an integer n: the number of coins.

The next line has n integers $x_1, x_2, \ldots, x_n$: the values of the coins.

Output

First print an integer k: the number of distinct money sums. After this, print all possible sums in increasing order.

Constraints

- $1 \le n \le 100$
- $1 \le x_i \le 1000$

To solve the problem using dynamic programming, define the function $\text{possible}(i, s)$ to describe whether it is possible to form the money sum s using some of the first i coins.

When considering the next coin, we can either choose it and add its value to the sum, or not and use only previous coins.

$$\text{possible}(i, s) = \text{possible}(i-1, s) \vee \text{possible}(i-1, s-x_i)$$

The symbol $\vee$ denotes the boolean OR operation.

When no coins have yet been considered, the only possible sum is 0. This shouldn't be printed in this problem though.

$$\text{possible}(0, s) = \begin{cases} \text{true} & \text{if } s = 0 \\ \text{false} & \text{otherwise} \end{cases}$$

Since the recurrence only references computed values of the previous coin, it is enough to share a one-dimensional array as long as it is iterated in a suitable order.

The time complexity of this solution is $O(n^2 x)$, where x is the maximum value of each coin, as any sum of coins $s \le nx$.

```
#include <iostream>
using namespace std;
const int X = 100000;

bool possible[X + 1];

int main() {
    int n;
    cin >> n;

    possible[0] = true;
    for (int i = 0; i < n; ++i) {
```

```
        int x;
        cin >> x;
        for (int s = X; s >= x; --s) {
            possible[s] |= possible[s - x];
        }
    }

    int k = 0;
    for (int s = 1; s <= X; ++s) {
        k += possible[s];
    }
    cout << k << "\n";
    for (int s = 1; s <= X; ++s) {
        if (possible[s]) cout << s << " ";
    }
    cout << "\n";
}
```

References

- CPHB $Competitive Programmer's Handbook$, Chapter 7, Knapsack problems

1744 · Rectangle Cutting

Task · source analysis length 1,349 characters

Problem statement

Given an $a \times b$ rectangle, your task is to cut it into squares. On each move you can select a rectangle and cut it into two rectangles in such a way that all side lengths remain integers. What is the minimum possible number of moves?

Input

The only input line has two integers a and b.

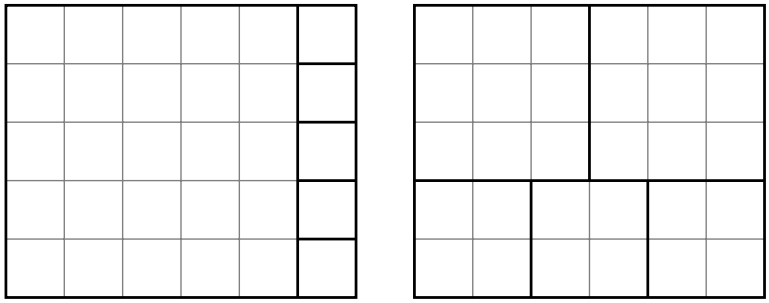
Output

Print one integer: the minimum number of moves.

Constraints

- $1 \le a, b \le 500$

A greedy solution to this problem could always cut off a square from the rectangle, by choosing the shorter side of the rectangle. However, this is not necessarily optimal:



5 moves 4 moves

Instead, we use dynamic programming to exhaustively check every possible move and choose the optimal one.

The function $\gdef\mmoves\m(i,j)$ represents the minimum number of moves to cut an $i \times j$ rectangle into squares.

If the rectangle is already a square, then $i = j$ and $\m(i,j$=0).

Otherwise, find the optimum among all horizontal and vertical cuts:

Could not find closing ']' for argument to \\

\min_{y=1}^{j-1} $\m(i, y$ + \m$i, j - y$ + 1) \end{cases}]

Considering the maximum of the two sides, $n = \max(a, b)$, the time complexity of this solution is $O(n^3)$.

```
#include <iostream>
#include <vector>
```

```
using namespace std;
const int INF = 1e9;

int main() {
    int a, b;
    cin >> a >> b;

    vector<vector<int>> moves(a + 1, vector<int>(b + 1));
    for (int i = 1; i <= a; ++i) {
        for (int j = 1; j <= b; ++j) {
            if (i == j) {
                moves[i][j] = 0;
            } else {
                moves[i][j] = INF;
                for (int x = 1; x < i; ++x) {
                    moves[i][j] =
                        min(moves[i][j], moves[x][j] + moves[i - x][j] + 1);
                }
                for (int y = 1; y < j; ++y) {
                    moves[i][j] =
                        min(moves[i][j], moves[i][y] + moves[i][j - y] + 1);
                }
            }
        }
    }

    cout << moves[a][b] << "\n";
}
```

1743 · String Reorder

Task · source analysis length 1,351 characters

Problem statement

Your task is to reorder the characters of a string so that no two adjacent characters are the same. What is the lexicographically minimal such string?

Input

The only line has a string of length n consisting of characters A–Z.

Output

Print the lexicographically minimal reordered string where no two adjacent characters are the same. If it is not possible to create such a string, print -1.

Constraints

- $1 \le n \le 10^6$

Let us first analyze when is it possible construct a string such that no two adjacent characters are the same. The maximum amount of occurrences for a single character is reached when it appears in every other position, including the first and the last. This leads to the inequality $2k \le n + 1$, where k is the number of occurrences of the most common character and n is the length of the string. The construction is possible if and only if the above inequality holds.

Now, we may construct the answer, one character at a time. To get the lexicographically smallest string, we will, at each step, choose the smallest character greedily. The above condition is used to check if the string can be completed by choosing a specific character.

```
#include <iostream>
using namespace std;

int count[128];

int main() {
    string s;
    cin >> s;

    int n = s.size();

    for (auto c : s) {
        count[(int)c]++;
    }

    auto check = [&](int length) {
        for (int i = 'A'; i <= 'Z'; ++i) {
            if (count[i] * 2 > length + 1) {
                return false;
            }
        }
        return true;
    };

    if (!check(n)) {
        cout << -1 << '\n';
        return 0;
    }
```

```
    char prev = '$';
    for (int i = 1; i <= n; ++i) {
        for (int nxt = 'A'; nxt <= 'Z'; ++nxt) {
            if (!count[nxt]) continue;
            if (nxt == prev) continue;
            count[nxt]--;
            if (check(n - i)) {
                cout << (char)nxt;
                prev = nxt;
                break;
            }
            count[nxt]++;
        }
    }
    cout << '\n';
}
```

1740 · Intersection Points

Task · source analysis length 2,208 characters

Problem statement

Given n horizontal and vertical line segments, your task is to calculate the number of their intersection points.

You can assume that no parallel line segments intersect, and no endpoint of a line segment is an intersection point.

Input

The first line has an integer n: the number of line segments.

Then there are n lines describing the line segments. Each line has four integers: x_1, y_1, x_2 and y_2: a line segment begins at point (x_1, y_1) and ends at point (x_2, y_2).

Output

Print the number of intersection points.

Constraints

- $1 \leq n \leq 10^5$
- $-10^6 \leq x_1 \leq x_2 \leq 10^6$
- $-10^6 \leq y_1 \leq y_2 \leq 10^6$
- $(x_1, y_1$ \neq $x_2, y_2)$

This problem can be solved with a sweep line algorithm. We will sweep through the x-coordinates in increasing order, and at the same time, maintain active horizontal lines in a segment tree. The segment tree allows us to query the amount of intersection points of a vertical line.

We will compress the y-coordinates to the range $[0, 2n)$. This allows us to make a segment tree of size $O(n)$. The algorithm has a time complexity $O(n \log n)$ as each of the $O(n)$ events corresponds to a $O(\log n)$ time segment tree query.

```
#include <algorithm>
#include <iostream>
#include <tuple>
#include <vector>
using namespace std;
using ll = long long;

// Segment tree data structure
//  - Supports changing single values and querying range sums.
struct Tree {
    int n;
    vector<int> tree;

    Tree(int k) {
        n = 1;
        while (n < k) n *= 2;
        tree.assign(n * 2, 0);
    }

    // Add x to position k.
    void add(int k, int x) {
        k += n;
```

```
        tree[k] += x;
        while (k > 1) {
            k /= 2;
            tree[k] = tree[k * 2] + tree[k * 2 + 1];
        }
    }

    // Returns the sum of values on range [l, r].
    int query(int l, int r) {
        int sum = 0;
        for (l += n, r += n; l <= r; l /= 2, r /= 2) {
            if (l % 2 == 1) sum += tree[l++];
            if (r % 2 == 0) sum += tree[r--];
        }
        return sum;
    }
};

// Event types
enum { QUERY, ADD, REMOVE };

int main() {
    int n;
    cin >> n;

    vector<int> y_values;
    vector<tuple<int, int, int, int>> events;

    for (int i = 0; i < n; ++i) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;

        y_values.push_back(y1);
        y_values.push_back(y2);

        if (y1 == y2) {
            events.emplace_back(x1, ADD, y1, y2);
            events.emplace_back(x2, REMOVE, y1, y2);
        } else {
            events.emplace_back(x1, QUERY, y1, y2);
        }
    }

    sort(events.begin(), events.end());
    sort(y_values.begin(), y_values.end());
    y_values.erase(unique(y_values.begin(), y_values.end()), y_values.end());

    auto index = [&](int y) {
        auto it = lower_bound(y_values.begin(), y_values.end(), y);
        return distance(y_values.begin(), it);
    };

    Tree tree(y_values.size());
    ll ans = 0;

    for (auto [x, type, y1, y2] : events) {
        y1 = index(y1);
        y2 = index(y2);
        if (type == ADD) {
            tree.add(y1, 1);
        } else if (type == REMOVE) {
            tree.add(y1, -1);
        } else {
            ans += tree.query(y1, y2);
        }
    }

    cout << ans << '\n';
}
```

1739 · Forest Queries II

Task · source analysis length 1,996 characters

Problem statement

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or has a tree. Your task is to process q queries of the following types:

1. Change the state $empty/tree$ of a square.
2. How many trees are inside a rectangle in the forest?

Input

The first input line has two integers n and q: the size of the forest and the number of queries.

Then, there are n lines describing the forest. Each line has n characters: `.` is an empty square and * is a tree.

Finally, there are q lines describing the queries. The format of each line is either "$1\ y\ x$" or "$2\ y_1\ x_1\ y_2\ x_2$".

Output

Print the answer to each query of the second type.

Constraints

- $1 \le n \le 1000$
- $1 \le q \le 2 \cdot 10^5$
- $1 \le y, x \le n$
- $1 \le y_1 \le y_2 \le n$
- $1 \le x_1 \le x_2 \le n$

This problem can be solved with a *2D segment tree*. In a *2D segment tree*, each node contains a regular segment tree.

We query $O(\log n)$ nodes in the top level tree and answering each of these queries in a second level tree takes $O(\log n)$ time. Thus, answering a single query takes $O(\log^2 n)$ time.

The total time complexity is $O((n + q$\log^2n)).

The tree consumes $O(n^2)$ memory in total as each of the $O(n)$ nodes in the top level tree consume $O(n)$ memory.

```
#include <iostream>
using namespace std;

const int TREE_SIZE = 1 << 10;
int trees[TREE_SIZE * 2][TREE_SIZE * 2];

// Adds d to the position x in the segment tree `tree`.
void add(int *tree, int x, int d) {
    x += TREE_SIZE;
    while (x > 0) {
        tree[x] += d;
        x /= 2;
    }
}

// Adds d to the position (i, j).
```

```
void add(int y, int x, int d) {
    y += TREE_SIZE;
    while (y > 0) {
        add(trees[y], x, d);
        y /= 2;
    }
}

// Returns the sum of value in range [x1, x2] in `tree`.
int query(int *tree, int x1, int x2) {
    x1 += TREE_SIZE;
    x2 += TREE_SIZE;
    int sum = 0;
    while (x1 <= x2) {
        if (x1 % 2 == 1) sum += tree[x1++];
        if (x2 % 2 == 0) sum += tree[x2--];
        x1 /= 2;
        x2 /= 2;
    }
    return sum;
}

// Returns the sum of value in the rectangle [y1, y2] x [x1, x2]
int query(int y1, int y2, int x1, int x2) {
    y1 += TREE_SIZE;
    y2 += TREE_SIZE;
    int sum = 0;
    while (y1 <= y2) {
        if (y1 % 2 == 1) sum += query(trees[y1++], x1, x2);
        if (y2 % 2 == 0) sum += query(trees[y2--], x1, x2);
        y1 /= 2;
        y2 /= 2;
    }
    return sum;
}

const int N = 1001;
int forest[N][N];

int main() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            char c;
            cin >> c;
            if (c == '*') {
                forest[i][j] = 1;
                add(i, j, 1);
            }
        }
    }

    for (int qi = 0; qi < q; ++qi) {
        int t;
        cin >> t;
        if (t == 1) {
            int i, j;
            cin >> i >> j;
            if (forest[i][j]) {
                add(i, j, -1);
                forest[i][j] = 0;
            } else {
                add(i, j, 1);
                forest[i][j] = 1;
            }
        } else {
            int y1, x1, y2, x2;
            cin >> y1 >> x1 >> y2 >> x2;
            cout << query(y1, y2, x1, x2) << '\n';
        }
    }
}
```

1737 · Range Queries and Copies

Task · source analysis length 2,201 characters

Problem statement

Your task is to maintain a list of arrays which initially has a single array. You have to process the following types of queries:

1. Set the value a in array k to x.
2. Calculate the sum of values in range $[a, b]$ in array k.
3. Create a copy of array k and add it to the end of the list.

Input

The first input line has two integers n and q: the array size and the number of queries.

The next line has n integers $t_1, t_2, \ldots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is one of the following: "1 $k\ a\ x$", "2 $k\ a\ b$" or "3 k".

Output

Print the answer to each sum query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le t_i, x \le 10^9$
- $1 \le a \le b \le n$

This problem can be solved with a *persistent segment tree*. Persistent segment trees allow us to query and modify historical versions of the tree.

We will make a new segment tree after each operation. The key observation here is that only $O(\log n)$ nodes are changed per operation and the rest of the nodes can be reused.

Each segment tree operation takes $O(\log n)$ time making the total time complexity of the solution $O((n + q$ \log n)).

As each operation creates $O(\log n)$ nodes, the solution uses $O(q \log n)$ memory.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

struct Node {
    Node *left = nullptr, *right = nullptr;
    ll sum;
};

// Builds a segment tree from vector v.
// Returns a pointer to the constructed tree.
Node *build(int l, int r, vector<int> &v) {
    Node *p = new Node();
    if (r - l == 1) {
        p->sum = v[l];
    } else {
        int m = (l + r) / 2;
```

```
        p->left = build(l, m, v);
        p->right = build(m, r, v);
        p->sum = p->left->sum + p->right->sum;
    }
    return p;
}

// Changes value in position k to x in tree p.
// Returns a pointer to the new tree.
Node *change(Node *p, int l, int r, int k, int x) {
    Node *n = new Node(*p);
    if (r - l == 1) {
        n->sum = x;
    } else {
        int m = (l + r) / 2;
        if (k < m) {
            n->left = change(p->left, l, m, k, x);
        } else {
            n->right = change(p->right, m, r, k, x);
        }
        n->sum = n->left->sum + n->right->sum;
    }
    return n;
}

// Returns the sum of values in range [ql, qr).
ll query(Node *p, int l, int r, int ql, int qr) {
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return p->sum;
    int m = (l + r) / 2;
    return query(p->left, l, m, ql, qr) + query(p->right, m, r, ql, qr);
}

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
    }

    vector<Node *> trees{build(0, n, v)};

    for (int qi = 0; qi < q; ++qi) {
        int t, k;
        cin >> t >> k;
        k--;
        if (t == 1) {
            int a, x;
            cin >> a >> x;
            trees[k] = change(trees[k], 0, n, a - 1, x);
        } else if (t == 2) {
            int l, r;
            cin >> l >> r;
            cout << query(trees[k], 0, n, l - 1, r) << '\n';
        } else {
            trees.push_back(trees[k]);
        }
    }
}
```

References

- CPHB *Competitive Programmer's Handbook*, Chapter 28
- CP-Algorithms, Segment Tree

1736 · Polynomial Queries

Task · source analysis length 3,572 characters

Problem statement

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range $[a, b]$ by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range $[a, b]$.

Input

The first input line has two integers n and q: the size of the array and the number of queries.

The next line has n values $t_1, t_2, \ldots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output

Print the answer to each sum query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le t_i \le 10^6$
- $1 \le a \le b \le n$

This problem can be solved with a *lazy segment tree*.

Let us focus on a more general version of the problem, one where any arithmetic progression may be added.

Any arithmetic progression $a, a+d, a+2d, \ldots$ can be identified with two numbers a and d, the first element of the sequence and the difference between subsequent elements.

How can one calculate the sum of the first n elements in an arithmetic progression? Firstly, notice that a is repeated in each term. Thus we may take the sum $n \cdot a$ and focus on the sequence $0, d, 2d, \ldots, (n-1)$d).

Recall the well-known sum formula

$$0 + 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

From the above formula we can derive

$$0 + d + 2d + \cdots + d(n-1) = d\frac{n(n-1)}{2}.$$

In conclusion, the sum of the first n elements in any arithmetic progression $a, a+d, a+2d, \ldots$ is equal to

$$na + d\frac{n(n-1)}{2}.$$

This can be implemented using three arrays for the segment tree:

1. `sum` stores the sum of the values in the range of each node

2. `lazy_a` stores the starting value of the sequence that needs to be added to the elements in the range of the node
3. `lazy_d` stores the difference between subsequent elements in the sequence that needs to be added to the elements in the range of the node

In other words, the values a and d, of `lazy_a` and `lazy_d` respectively, correspond to the sequence

$$a, a + d, a + 2d, \ldots$$

The *lazy* values only apply to the children of a node – the change is already accounted for in the sum of the corresponding node.

When we push an arithmetic progression into the two children of a node in the segment tree, the sequence of the right child will start from the value $a + d \cdot \mathrm{length}$, where length is the size of the range corresponding to the left child.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

// This lazy segment tree implementation supports the following operations:
//  1. add(l, r, x): in range [l, r), add x + 0 to the first element,
//                                    x + 1 to the second,
//                                    x + 2 to the third and so on
//  3. query(l, r): find the sum of values in range [l, r)
struct Tree {
    int n;
    vector<ll> sum;
    vector<ll> lazy_a;
    vector<ll> lazy_d;

    Tree(int n) : n(n), sum(n * 4), lazy_a(n * 4), lazy_d(n * 4) {}

    void add(int s, int l, int r, int ql, int qr, int x) {
        if (r <= ql || qr <= l) return;
        if (ql <= l && r <= qr) {
            apply(s, r - l, x + (l - ql), 1);
            return;
        }
        push(s, r - l);
        int m = (l + r) / 2;
        add(s * 2 + 0, l, m, ql, qr, x);
        add(s * 2 + 1, m, r, ql, qr, x);
        sum[s] = sum[s * 2] + sum[s * 2 + 1];
    }

    ll query(int s, int l, int r, int ql, int qr) {
        if (r <= ql || qr <= l) return 0;
        if (ql <= l && r <= qr) return sum[s];
        push(s, r - l);
        int m = (l + r) / 2;
        return query(s * 2, l, m, ql, qr) + query(s * 2 + 1, m, r, ql, qr);
    }

    void push(int s, int len) {
        apply(s * 2 + 0, len / 2, lazy_a[s], lazy_d[s]);
        apply(s * 2 + 1, (len + 1) / 2, lazy_a[s] + len / 2 * lazy_d[s],
              lazy_d[s]);
        lazy_a[s] = 0;
        lazy_d[s] = 0;
    }

    void apply(int s, int len, ll a, ll d) {
        sum[s] += a * len + d * ll(len - 1) * len / 2;
        lazy_a[s] += a;
        lazy_d[s] += d;
    }

    void add(int l, int r, int x) { add(1, 0, n, l, r, x); }
    ll query(int l, int r) { return query(1, 0, n, l, r); }
};

int main() {
    int n, q;
    cin >> n >> q;

    Tree tree(n);

    for (int i = 0; i < n; ++i) {
        int t;
```

```
        cin >> t;
        tree.add(i, i + 1, t);
    }

    for (int qi = 0; qi < q; ++qi) {
        int t, l, r;
        cin >> t >> l >> r;
        l--;
        if (t == 1) {
            tree.add(l, r, 1);
        } else {
            cout << tree.query(l, r) << '\n';
        }
    }
}
```

References

- CPHB $\mathcal{C}ompetitive\mathcal{P}rogrammer's\mathcal{H}andbook$, Chapter 28
- CP-Algorithms, Segment Tree

1735 · Range Updates and Sums

Task · source analysis length 3,093 characters

Problem statement

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase each value in range $[a, b]$ by x.
2. Set each value in range $[a, b]$ to x.
3. Calculate the sum of values in range $[a, b]$.

Input

The first input line has two integers n and q: the array size and the number of queries.

The next line has n values $t_1, t_2, \ldots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is one of the following: "1 $a\ b\ x$", "2 $a\ b\ x$", or "3 $a\ b$".

Output

Print the answer to each sum query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le t_i, x \le 10^6$
- $1 \le a \le b \le n$

This problem can be solved with a *lazy segment tree*.

We will use three arrays for the segment tree:

1. `sum` stores the sum of the values in the range of each node
2. `lazy_add` stores the sum that needs to be added to each value in the range of the node
3. `lazy_set` stores the value to which each element in the range of the node needs to be changed

The *lazy* values only apply to the children of a node – the change is already accounted for in the sum of the corresponding node.

When we push the lazy updates downwards in the tree, we have to be careful to merge them correctly. For example, a set operation overwrites all previous add operations.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

// This lazy segment tree implementation supports the following operations:
//  1. add(l, r, x): add x to every element in range [l, r)
//  2. set(l, r, x): set every element in range [l, r) to x
//  3. query(l, r): find the sum of values in range [l, r)
struct Tree {
    int n;
    vector<ll> sum;
    vector<ll> lazy_add;
    vector<ll> lazy_set;

    Tree(int n) : n(n), sum(n * 4), lazy_add(n * 4), lazy_set(n * 4) {}
```

```
    void add(int s, int l, int r, int ql, int qr, int x) {
        if (r <= ql || qr <= l) return;
        if (ql <= l && r <= qr) {
            apply_add(s, r - l, x);
            return;
        }
        push(s, r - l);
        int m = (l + r) / 2;
        add(s * 2 + 0, l, m, ql, qr, x);
        add(s * 2 + 1, m, r, ql, qr, x);
        sum[s] = sum[s * 2] + sum[s * 2 + 1];
    }

    void set(int s, int l, int r, int ql, int qr, int x) {
        if (r <= ql || qr <= l) return;
        if (ql <= l && r <= qr) {
            apply_set(s, r - l, x);
            return;
        }
        push(s, r - l);
        int m = (l + r) / 2;
        set(s * 2 + 0, l, m, ql, qr, x);
        set(s * 2 + 1, m, r, ql, qr, x);
        sum[s] = sum[s * 2] + sum[s * 2 + 1];
    }

    ll query(int s, int l, int r, int ql, int qr) {
        if (r <= ql || qr <= l) return 0;
        if (ql <= l && r <= qr) return sum[s];
        push(s, r - l);
        int m = (l + r) / 2;
        return query(s * 2, l, m, ql, qr) + query(s * 2 + 1, m, r, ql, qr);
    }

    void push(int s, int len) {
        if (lazy_set[s]) {
            apply_set(s * 2, len / 2, lazy_set[s]);
            apply_set(s * 2 + 1, (len + 1) / 2, lazy_set[s]);
            lazy_set[s] = 0;
        }
        if (lazy_add[s]) {
            apply_add(s * 2, len / 2, lazy_add[s]);
            apply_add(s * 2 + 1, (len + 1) / 2, lazy_add[s]);
            lazy_add[s] = 0;
        }
    }

    void apply_add(int s, int len, ll x) {
        sum[s] += x * len;
        lazy_add[s] += x;
    }

    void apply_set(int s, int len, ll x) {
        sum[s] = x * len;
        lazy_add[s] = 0;
        lazy_set[s] = x;
    }

    void add(int l, int r, int x) { add(1, 0, n, l, r, x); }
    void set(int l, int r, int x) { set(1, 0, n, l, r, x); }
    ll query(int l, int r) { return query(1, 0, n, l, r); }
};

int main() {
    int n, q;
    cin >> n >> q;

    Tree tree(n);

    for (int i = 0; i < n; ++i) {
        int t;
        cin >> t;
        tree.set(i, i + 1, t);
    }

    for (int qi = 0; qi < q; ++qi) {
        int t, l, r;
        cin >> t >> l >> r;
        l--;
        if (t == 1) {
            int x;
            cin >> x;
            tree.add(l, r, x);
        } else if (t == 2) {
            int x;
```

```
            cin >> x;
            tree.set(l, r, x);
        } else {
            cout << tree.query(l, r) << '\n';
        }
    }
}
```

References

- CPHB $Competitive Programmer's Handbook$, Chapter 28
- CP-Algorithms, Segment Tree

1734 · Distinct Values Queries

Task · source analysis length 6,423 characters

Problem statement

You are given an array of n integers and q queries of the form: how many distinct values are there in a range $[a, b]$?

Input

The first input line has two integers n and q: the array size and number of queries.

The next line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has two integers a and b.

Output

For each query, print the number of distinct values in the range.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le a \le b \le n$

First approach: Segment tree

This problem can be solved using a segment tree. Note that we can answer the queries offline, which means that we can change the order in which we answer them.

We process the queries in increasing order of their starting points. At each point, we answer all queries that begin at that point.

We always want to have only one occurrence of each distinct value in the segment tree, and we store, for each value, the next occurrence to the right of our pointer.

It is guaranteed that if any occurrence of a value is in the query range then the leftmost occurrence is also in the query range. Thus, each distinct value in the query range is counted exactly once which is precisely what we want.

The time complexity of this implementation is $O(n \log n)$.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <vector>
using namespace std;

const int TREE_SIZE = 1 << 18;
int tree[TREE_SIZE * 2];

void change(int k, int x) {
    k += TREE_SIZE;
    tree[k] = x;
    while (k > 1) {
        k /= 2;
        tree[k] = tree[k * 2] + tree[k * 2 + 1];
```

```
    }
}

// Returns the sum of values in range [l, r].
int query(int l, int r) {
    int ans = 0;
    l += TREE_SIZE;
    r += TREE_SIZE;
    while (l <= r) {
        if (l % 2 == 1) ans += tree[l++];
        if (r % 2 == 0) ans += tree[r--];
        l /= 2;
        r /= 2;
    }
    return ans;
}

struct Query {
    int l, r, i;
};

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
    }

    // Stores occurrences of each value — in decreasing order.
    // Done this way, we can, at any time, efficiently
    // remove the smallest occurrence of any value
    // using the function `pop_back`.
    map<int, vector<int>> occurrences;
    for (int i = n - 1; i >= 0; --i) {
        occurrences[v[i]].push_back(i);
    }

    for (auto &[v, l] : occurrences) {
        change(l.back(), 1);
        l.pop_back();
    }

    vector<Query> queries(q);
    for (int i = 0; i < q; ++i) {
        cin >> queries[i].l >> queries[i].r;
        queries[i].l--;
        queries[i].r--;
        queries[i].i = i;
    }

    sort(queries.begin(), queries.end(),
         [&](auto a, auto b) { return a.l < b.l; });

    vector<int> answer(q);

    for (int i = 0, j = 0; i < n; ++i) {
        while (j < q && queries[j].l == i) {
            answer[queries[j].i] = query(i, queries[j].r);
            ++j;
        }
        if (!occurrences[v[i]].empty()) {
            change(occurrences[v[i]].back(), 1);
            occurrences[v[i]].pop_back();
        }
    }

    for (int a : answer) {
        cout << a << ' ';
    }
    cout << '\n';
}
```

Second approach: Mo's algorithm

Mo's algorithm is an offline algorithm for answering range queries. The algorithm is based on a special ordering where queries are split in blocks of size $B = \sqrt{n}$. The query $[l_0, r_0]$ comes before $[l_1, r_1]$ if either

- $\left\lfloor \frac{l_0}{B} \right\rfloor < \left\lfloor \frac{l_1}{B} \right\rfloor$

- $\left\lfloor \frac{l_0}{B} \right\rfloor = \left\lfloor \frac{l_1}{B} \right\rfloor$ and $r_0 < r_1$.

We maintain an active range with two pointers. These pointers are moved, one step at a time, from one query to the next.

The left pointer moves $O(n)$ times $O(\sqrt{n})$ steps and the right pointer moves $O(\sqrt{n})$ times $O(n)$ steps.

Thus, the total time complexity of this algorithm is

$$O(n \cdot f(n)\sqrt{n})$$

where $f(n)$ represents the cost of moving a pointer.

An implementation of Mo's algorithm can be found below. Unfortunately, this solution is too slow.

As moving a pointer changes a value inside a `map`, it takes $O(\log n)$ time and the total time complexity of the implementation is $O(n\sqrt{n}\log n)$ if we assume that there are n queries. This translates to roughly $1.57 \cdot 10^9$ operations with a large constant factor.

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <map>
#include <vector>
using namespace std;

// Set block size equal to sqrt(N).
const int B = (int)sqrt(2e5);

struct Query {
    int l, r, i;
};

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
    }

    vector<Query> queries(q);
    for (int i = 0; i < q; ++i) {
        cin >> queries[i].l >> queries[i].r;
        queries[i].l--;
        queries[i].r--;
        queries[i].i = i;
    }

    sort(queries.begin(), queries.end(), [&](auto a, auto b) {
        if (a.l / B != b.l / B) {
            return a.l / B < b.l / B;
        }
        return a.r < b.r;
    });

    vector<int> answer(q);

    map<int, int> s;

    auto add = [&](int i) {
        s[v[i]]++;
    };

    auto remove = [&](int i) {
        auto it = s.find(v[i]);
        it->second--;
        if (it->second == 0) {
            s.erase(it);
        }
    };

    int al = 0, ar = 0;
    add(0);

    for (auto [l, r, i] : queries) {
        while (ar < r) add(++ar);
        while (al > l) add(--al);
        while (al < l) remove(al++);
        while (ar > r) remove(ar--);
```

```
        answer[i] = (int)s.size();
    }

    for (int a : answer) {
        cout << a << ' ';
    }
    cout << '\n';
}
```

Fortunately, there is an easy fix: coordinate compression. When we translate the numbers from the range $[1, 10^9]$ to the range $[0, n-1]$, we do not need a `map` anymore. Instead, we can use an array to count the number of each value. This new implementation has a time complexity of $O(n\sqrt{n})$ which easily fits in the time limit.

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <map>
#include <vector>
using namespace std;

// Set block size equal to sqrt(N).
const int B = (int)sqrt(2e5);

struct Query {
    int l, r, i;
};

int main() {
    int n, q;
    cin >> n >> q;

    vector<int> v(n);
    for (int i = 0; i < n; ++i) {
        cin >> v[i];
    }

    auto values = v;
    sort(values.begin(), values.end());

    for (int &x : v) {
        x = lower_bound(values.begin(), values.end(), x) - values.begin();
    }

    vector<Query> queries(q);
    for (int i = 0; i < q; ++i) {
        cin >> queries[i].l >> queries[i].r;
        queries[i].l--;
        queries[i].r--;
        queries[i].i = i;
    }

    sort(queries.begin(), queries.end(), [&](auto a, auto b) {
        if (a.l / B != b.l / B) {
            return a.l / B < b.l / B;
        }
        return a.r < b.r;
    });

    int distinct = 0;
    vector<int> occurrences(n);
    vector<int> answer(q);

    auto add = [&](int i) {
        if (occurrences[v[i]]++ == 0) {
            distinct++;
        }
    };

    auto remove = [&](int i) {
        if (--occurrences[v[i]] == 0) {
            distinct--;
        }
    };

    int al = 0, ar = 0;
    add(0);

    for (auto [l, r, i] : queries) {
        while (ar < r) add(++ar);
        while (al > l) add(--al);
        while (al < l) remove(al++);
        while (ar > r) remove(ar--);
```

```
        answer[i] = distinct;
    }

    for (int a : answer) {
        cout << a << ' ';
    }
    cout << '\n';
}
```

References for Mo's algorithm

- CP-Algorithms, Sqrt Decomposition, Mo's algorithm
- Codeforces, *Mo's Algorithm* by kAc
- CPHB $\mathit{Competitive Programmer's Handbook}$, Chapter 27, Mo's algorithm

1733 · Finding Periods

Task · source analysis length 1,919 characters

Problem statement

A *period* of a string is a prefix that can be used to generate the whole string by repeating the prefix. The last repetition may be partial. For example, the periods of abcabca are abc, abcabc and abcabca.

Your task is to find all period lengths of a string.

Input

The only input line has a string of length n consisting of characters a–z.

Output

Print all period lengths in increasing order.

Constraints

- $1 \le n \le 10^6$

We discuss two ways to solve the problem in $O(n)$ time.

Solution 1 $Z - algorithm$

Let us construct the Z-array for the string using the Z-algorithm. After that, we go through all positions of the string. If the suffix at position i is also a prefix of the string, we can conclude that i is a period length of the string. In addition, n is always a period length of the string.

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> z_algo(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        z[i] = max(0, min(z[i - l], r - i));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

int main() {
    string s;
    cin >> s;
    int n = s.size();

    auto z = z_algo(s);
    for (int i = 1; i < n; i++) {
        if (z[i] == n - i) {
            cout << i << " ";
        }
    }
```

```
    cout << n << "\n";
}
```

Solution 2 *hashing*

We can also solve the problem using hashing. Since we can calculate the hash value of any substring in $O(1)$ time, we can go through the string and check the suffixes like in Solution 1.

Warning: a code that uses hashing may fail due to collisions $two strings are different but have the same hash value$.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

struct HashString {
    const ll A = 31;
    const ll B = 991831889;

    vector<ll> pows, sums;

    HashString(string s) {
        int n = s.size();
        pows.resize(n + 1);
        pows[0] = 1;
        sums.resize(n + 1);
        sums[0] = 0;
        for (int i = 1; i <= n; i++) {
            pows[i] = pows[i - 1] * A % B;
            sums[i] = (sums[i - 1] * A + s[i - 1]) % B;
        }
    }

    ll hash(int a, int b) {
        ll h = sums[b + 1] - sums[a] * pows[b - a + 1];
        return (h % B + B) % B;
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();

    HashString h(s);
    for (int i = 1; i < n; i++) {
        if (h.hash(i, n - 1) == h.hash(0, n - i - 1)) {
            cout << i << " ";
        }
    }
    cout << n << "\n";
}
```

1732 · Finding Borders

Task · source analysis length 1,947 characters

Problem statement

A *border* of a string is a prefix that is also a suffix of the string but not the whole string. For example, the borders of abcababcab are ab and abcab.

Your task is to find all border lengths of a given string.

Input

The only input line has a string of length n consisting of characters a–z.

Output

Print all border lengths of the string in increasing order.

Constraints

- $1 \le n \le 10^6$

We discuss two ways to solve the problem in $O(n)$ time.

Solution 1 $Z - algorithm$

Let us construct the Z-array for the string using the Z-algorithm. After that, we go through all suffixes of the string. If a suffix is also a prefix of the string, we have found a border. We can check that using the Z-array: a suffix at position i is a border exactly when the array value is $n - i$.

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> z_algo(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        z[i] = max(0, min(z[i - l], r - i));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

int main() {
    string s;
    cin >> s;
    int n = s.size();

    auto z = z_algo(s);
    for (int i = n - 1; i > 0; i--) {
        if (z[i] == n - i) {
            cout << n - i << " ";
        }
    }
```

```
    cout << "\n";
}
```

Solution 2 *hashing*

We can also solve the problem using hashing. Since we can calculate the hash value of any substring in $O(1)$ time, we can just go through every prefix and check if it equals the suffix of the same length.

Warning: a code that uses hashing may fail due to collisions $two strings are different but have the same hash value$.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

struct HashString {
    const ll A = 31;
    const ll B = 991831889;

    vector<ll> pows, sums;

    HashString(string s) {
        int n = s.size();
        pows.resize(n + 1);
        pows[0] = 1;
        sums.resize(n + 1);
        sums[0] = 0;
        for (int i = 1; i <= n; i++) {
            pows[i] = pows[i - 1] * A % B;
            sums[i] = (sums[i - 1] * A + s[i - 1]) % B;
        }
    }

    ll hash(int a, int b) {
        ll h = sums[b + 1] - sums[a] * pows[b - a + 1];
        return (h % B + B) % B;
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();

    HashString h(s);
    for (int i = 0; i < n - 1; i++) {
        if (h.hash(0, i) == h.hash(n - i - 1, n - 1)) {
            cout << i + 1 << " ";
        }
    }
    cout << "\n";
}
```

1731 · Word Combinations

Task · source analysis length 3,800 characters

Problem statement

You are given a string of length n and a dictionary containing k words. In how many ways can you create the string using the words?

Input

The first input line has a string containing n characters between a–z.

The second line has an integer k: the number of words in the dictionary.

Finally there are k lines describing the words. Each word is unique and consists of characters a–z.

Output

Print the number of ways modulo $10^9 + 7$.

Constraints

- $1 \le n \le 5000$
- $1 \le k \le 10^5$
- the total length of the words is at most 10^6

Solution 1 *trie*

We create a trie that contains all words in the dictionary. The root node of the trie has index 0, and $\texttt{trie}[i][x]$ points to a node we will move from node i after reading letter x. In addition, $\texttt{final}[i]$ is true if some word ends at node i.

We use dynamic programming to count the combinations. We create an array ways that stores, for each suffix of the string, the number of ways to create the suffix using the dictionary. We go through each prefix of the suffix and follow the links in the trie to find the prefixes that are in the dictionary.

The algorithm works in $O(n^2 + N)$ time where N is the total length of the words in the dictionary.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

const int N = 1000000;
const int M = 1000000007;

int trie[N + 1][26];
int trie_pos;
bool final[N + 1];

int main() {
    string s;
    cin >> s;
    int n = s.size();

    int k;
    cin >> k;
    while (k--) {
```

```
        string w;
        cin >> w;
        int node = 0;
        for (auto c : w) {
            int idx = c - 'a';
            if (trie[node][idx] == 0) {
                trie[node][idx] = ++trie_pos;
            }
            node = trie[node][idx];
        }
        final[node] = true;
    }

    vector<ll> ways(n + 1);
    ways[n] = 1;
    for (int i = n - 1; i >= 0; i--) {
        ways[i] = 0;
        int node = 0;
        for (int j = i; j < n; j++) {
            node = trie[node][s[j] - 'a'];
            if (node == 0) break;
            if (final[node]) {
                ways[i] += ways[j + 1];
                ways[i] %= M;
            }
        }
    }

    cout << ways[0] << "\n";
}
```

Solution 2 *hashing*

We create a set `hashes` that contains all hashes of the words in the dictionary. Then, we can use dynamic programming to calculate the answer like in the previous solution and use the set to check if a substring belongs to the dictionary.

Since we compare each substring with all words, we need 64-bit hash values to make sure there will be no collisions, due to the birthday paradox.

To make the solution more efficient, we use an additional trick: we create a vector `lengths` which is used to check if the dictionary contains at least one word with a given length. Using this vector, we don't have to check if a substring appears in the dictionary if we know that there are no words of that length in the dictionary.

Since the total length of the words is at most 10^6, there will be only about $\sqrt{2 \cdot 10^6} \approx 1414$ distinct word lengths, compared to 5000 possible substring lengths.

The solution works in $O((n^2 + k$ C+N)) time where C denotes the time needed for set operations and N is the total length of the words.

```
#include <chrono>
#include <iostream>
#include <random>
#include <unordered_set>
#include <vector>
using namespace std;
using ll = long long;

const int M = 1000000007;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
const ll Z = 1111111111111111111;  // Z is a prime.
const ll C = uniform_int_distribution<ll>(Z / 10, Z / 10 * 9)(rng);

struct HashString {
    ll mul(ll a, ll b) { return __int128(a) * b % Z; }

    int n;
    vector<ll> pows, sums;

    HashString(string s) : n(s.size()), pows(n + 1, 1), sums(n + 1) {
        for (int i = 1; i <= n; i++) {
            pows[i] = mul(pows[i - 1], C);
            sums[i] = (mul(sums[i - 1], C) + s[i - 1]) % Z;
        }
    }

    // Returns the hash of the substring [l, r)
```

```
    ll hash(int l, int r) {
        ll h = sums[r] - mul(sums[l], pows[r - l]);
        return (h % Z + Z) % Z;
    }
};

int main() {
    string s;
    cin >> s;
    int n = s.size();
    HashString hs(s);
    int k;
    cin >> k;

    unordered_set<ll> hashes;
    vector<bool> lengths(n + 1);
    for (int i = 1; i <= k; i++) {
        string w;
        cin >> w;
        HashString hw(w);
        hashes.insert(hw.hash(0, w.size()));
        lengths[w.size()] = true;
    }

    vector<ll> ways(n + 1);
    ways[n] = 1;
    for (int i = n - 1; i >= 0; i--) {
        for (int j = i + 1; j <= n; j++) {
            if (lengths[j - i] && hashes.count(hs.hash(i, j))) {
                ways[i] += ways[j];
                ways[i] %= M;
            }
        }
    }

    cout << ways[0] << "\n";
}
```

1728 · Inversion Probability

Task · source analysis length 1,784 characters

Problem statement

An array has n integers $x_1, x_2, \ldots, x_n$, and each of them has been randomly chosen between 1 and r_i. An inversion is a pair (a, b) where $a < b$ and $x_a > x_b$.

What is the expected number of inversions in the array?

Input

The first input line contains an integer n: the size of the array.

The second line contains n integers $r_1, r_2, \ldots, r_n$: the range of possible values for each array position.

Output

Print the expected number of inversions rounded to six decimal places $roundinghalftoeven$.

Constraints

- $1 \le n \le 100$
- $1 \le r_i \le 100$

We consider each index pair (a, b) where $a < b$ and calculate the probability that $x_a > x_b$ which corresponds to an inversion.

Using linearity of expectation, the sum of the probabilities equals the expected number of inversions.

C++ solution

Here is a C++ solution for the problem:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> r(n);
    for (int i = 0; i < n; i++) {
        cin >> r[i];
    }

    double answer = 0;
    for (int a = 0; a < n; a++) {
        for (int b = a + 1; b < n; b++) {
            for (int x_a = 1; x_a <= r[a]; x_a++) {
                for (int x_b = 1; x_b <= min(x_a - 1, r[b]); x_b++) {
                    answer += 1.0 / (r[a] * r[b]);
                }
            }
        }
    }
    cout.precision(6);
```

```
    cout << fixed << answer << "\n";
}
```

Unfortunately, there are test cases where this solution fails due to precision issues. In general, it is difficult to create a correct solution in C++.

Python solution

In Python, we can use the `Fraction` class to avoid precision issues:

```
from fractions import Fraction

n = int(input())
r = [int(x) for x in input().split(" ")]

answer = Fraction(0, 1)
for a in range(n):
    for b in range(a + 1, n):
        for x_a in range(1, r[a] + 1):
            for x_b in range(1, min(x_a, r[b] + 1)):
                answer += Fraction(1, r[a] * r[b])

answer = float(round(answer, 6))
print(f"{answer:06f}")
```

The above solution works correctly but it is too slow. However, we can optimize the solution as follows:

```
from fractions import Fraction

n = int(input())
r = [int(x) for x in input().split(" ")]

answer = Fraction(0, 1)
for a in range(n):
    for b in range(a + 1, n):
        count = 0
        for x_a in range(1, r[a] + 1):
            count += min(x_a - 1, r[b])
        answer += Fraction(count, r[a] * r[b])

answer = float(round(answer, 6))
print(f"{answer:06f}")
```

This solution works both correctly and efficiently enough.

1726 · Moving Robots

Task · source analysis length 2,067 characters

Problem statement

Each square of an 8×8 chessboard has a robot. Each robot independently moves k steps, and there can be many robots on the same square.

On each turn, a robot moves one step left, right, up or down, but not outside the board. It randomly chooses a direction among those where it can move.

Your task is to calculate the expected number of *empty* squares after k turns.

Input

The only input line has an integer k.

Output

Print the expected number of empty squares rounded to six decimal places $roundinghalftoeven$.

Constraints

- $1 \le k \le 100$

We assume that the rows and columns are numbered from 0 to $N-1$ and the robots are numbered from 0 to N^2-1. The robot that starts at square (i,j) has number $i \cdot N + j$. In this problem $N = 8$ in each test.

We solve the problem using dynamic programming: $p[m][r][i][j]$ denotes the probability that robot r is in square (i,j) after m turns.

As base cases, $p[0][i \cdot N + j][i][j] = 1$ for each square (i,j). Then, when we know the value of $p[m][r][i][j]$, we find the squares where robot r can move from square (i,j). For each such square (i',j'), we increase the value of $p[m+1][r][i'][j']$ by $p[m][r][i][j]/c$ where c is the total number of squares where the robot can move.

After that, we go through the grid and calculate, for each square (i,j), the probability that the square is empty after k turns. We calculate the probability using the formula

$$\prod_{r=0}^{N^2-1} (1 - p[k][r][i][j])$$

which requires that no robot is in the square after k turns. Due to linearity of expectation, the sum of all such probabilities equals the expected number of empty squares after k turns.

The solution works in $O(kN^4)$ time.

```cpp
#include <iostream>
#include <vector>
using namespace std;

const int N = 8;
const int K = 100;
double p[K + 1][N * N][N][N];

int main() {
    int k;
    cin >> k;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            p[0][i * N + j][i][j] = 1;
        }
    }

    for (int m = 0; m < k; m++) {
        for (int r = 0; r < N * N; r++) {
            for (int i = 0; i < N; i++) {
                for (int j = 0; j < N; j++) {
                    vector<pair<int, int>> moves;
                    if (i > 0) moves.emplace_back(i - 1, j);
                    if (j > 0) moves.emplace_back(i, j - 1);
                    if (i + 1 < N) moves.emplace_back(i + 1, j);
                    if (j + 1 < N) moves.emplace_back(i, j + 1);
                    double add = p[m][r][i][j] / moves.size();
                    for (auto d : moves) {
                        p[m + 1][r][d.first][d.second] += add;
                    }
                }
            }
        }
    }

    double answer = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double e = 1;
            for (int r = 0; r < N * N; r++) {
                e *= (1 - p[k][r][i][j]);
            }
            answer += e;
        }
    }
    cout.precision(6);
    cout << fixed << answer << "\n";
}
```

1725 · Dice Probability

Task · source analysis length 923 characters

Problem statement

You throw a dice n times, and every throw produces an outcome between 1 and 6. What is the probability that the sum of outcomes is between a and b?

Input

The only input line contains three integers n, a and b.

Output

Print the probability rounded to six decimal places $roundinghalftoeven$.

Constraints

- $1 \le n \le 100$
- $1 \le a \le b \le 6n$

We solve the problem using dynamic programming: $p[i][j]$ denotes the probability of having sum j after throwing a dice i times.

As a base case $p[0][0] = 1$, because the sum is certainly 0 if there are no throws. Then, when we know a value of $p[i][j]$, we go through the possible throws whose outcomes are $k = 1, 2, \ldots, 6$. For each such throw, we increase $p[i+1][j+k]$ by $p[i][j]/6$. Finally, we calculate the sum of probabilities $p[n][i]$ where i is between a and b.

The solution works in $O(n^2)$ time.

```
#include <iostream>
using namespace std;

const int N = 100;
double p[N + 1][6 * N + 1];

int main() {
    int n, a, b;
    cin >> n >> a >> b;

    p[0][0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = i; j <= 6 * i; j++) {
            for (int k = 1; k <= 6; k++) {
                p[i + 1][j + k] += p[i][j] / 6;
            }
        }
    }

    double answer = 0;
    for (int i = a; i <= b; i++) {
        answer += p[n][i];
    }
    cout.precision(6);
    cout << fixed << answer << "\n";
}
```

1724 · Graph Paths II

Task · source analysis length 1,684 characters

Problem statement

Consider a directed weighted graph having n nodes and m edges. Your task is to calculate the minimum path length from node 1 to node n with exactly k edges.

Input

The first input line contains three integers n, m and k: the number of nodes and edges, and the length of the path. The nodes are numbered $1, 2, \ldots, n$.

Then, there are m lines describing the edges. Each line contains three integers a, b and c: there is an edge from node a to node b with weight c.

Output

Print the minimum path length. If there are no such paths, print -1.

Constraints

- $1 \le n \le 100$
- $1 \le m \le n(n-1)$
- $1 \le k \le 10^9$
- $1 \le a, b \le n$
- $1 \le c \le 10^9$

We create a custom matrix implementation where the product $C = AB$ is defined as follows:

$$C[i$$

$$j$$

= \min_{k=1}^N A

$$i$$

$$k$$

+ B

$$k$$

$$j$$

]

In addition, we use a special identity matrix where the main diagonal values are zeros and all other values are infinity values.

Then, given the distance matrix G of a graph, the shortest path length from node a to node b using exactly k edges equals $G^k[a][b]$. We construct G^k using fast exponentiation and then read the path length from the matrix.

Note that there can be duplicate edges, so we maintain the minimum single-edge path length when reading the input.

The solution works in $O(N^3 \log k)$ time where N is the upper bound for the number of nodes and k is the number of edges on the path.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const ll INF = 2e18;

template <int N>
struct Matrix {
    ll m[N][N];

    Matrix<N>(ll x = 0) { fill((ll*)m, (ll*)m + N * N, x); }

    static Matrix<N> ident() {
        Matrix<N> r(INF);
        for (int i = 0; i < N; i++) r.m[i][i] = 0;
        return r;
    }

    Matrix<N> operator*(Matrix<N> b) {
        Matrix<N> r(INF);
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                for (int k = 0; k < N; ++k) {
                    r.m[i][j] = min(r.m[i][j], m[i][k] + b.m[k][j]);
                }
            }
        }
        return r;
    }

    Matrix<N> power(ll b) {
        if (b == 0) return ident();
        auto a = *this;
        return b & 1 ? a.power(b - 1) * a : (a * a).power(b / 2);
    }
};

int main() {
    int n, m, k;
    cin >> n >> m >> k;

    Matrix<101> g(INF);
    for (int i = 1; i <= m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        g.m[a][b] = min(g.m[a][b], (ll)c);
    }

    auto p = g.power(k);
    ll answer = p.m[1][n];
    if (answer == INF) answer = -1;
    cout << answer << "\n";
}
```

1723 · Graph Paths I

Task · source analysis length 2,005 characters

Problem statement

Consider a directed graph that has n nodes and m edges. Your task is to count the number of paths from node 1 to node n with exactly k edges.

Input

The first input line contains three integers n, m and k: the number of nodes and edges, and the length of the path. The nodes are numbered $1, 2, \ldots, n$.

Then, there are m lines describing the edges. Each line contains two integers a and b: there is an edge from node a to node b.

Output

Print the number of paths modulo $10^9 + 7$.

Constraints

- $1 \le n \le 100$
- $1 \le m \le n(n-1)$
- $1 \le k \le 10^9$
- $1 \le a, b \le n$

Given the adjacency matrix G of a graph, the number of paths from node a to node b using exactly k edges equals $G^k[a][b]$. We can prove this property using induction:

Case $k = 1$: The property holds because $G[a][b]$ denotes the number of paths from node a to node b using exactly one edge.

Case $k = n$: Assume that the property holds for G^{n-1}, i.e., $G^{n-1}[a][b]$ denotes the number of paths from node a to node b using exactly $n-1$ edges. Then the property also holds for $G^n = G^{n-1}G$ because we can think that the matrix multiplication formula

$$G^n[a$$

$$b$$

= \sum_{i=1}^N $G^{n-1}[a][i] \cdot G[i][b]$]

goes through all possible ways to choose node i which is the last node before the final node b. In each case, there is first a path of length $n-1$ from node a to node i and then a path of length 1 from node i to node b.

To efficiently solve the problem, we construct G^k using fast exponentiation and then read the number of paths from the matrix.

Note that there can be duplicate edges, so we increase the number of paths by one after reading each edge.

The solution works in $O(N^3 \log k)$ time where N is the upper bound for the number of nodes and k is the number of edges on the path.

```cpp
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

template <int N>
struct Matrix {
    ll m[N][N];

    static Matrix<N> ident() {
        Matrix<N> r{};
        for (int i = 0; i < N; i++) r.m[i][i] = 1;
        return r;
    }

    Matrix<N> operator*(Matrix<N> b) {
        Matrix<N> r{};
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                for (int k = 0; k < N; ++k) {
                    r.m[i][j] += m[i][k] * b.m[k][j];
                    r.m[i][j] %= M;
                }
            }
        }
        return r;
    }

    Matrix<N> power(ll b) {
        if (b == 0) return ident();
        auto a = *this;
        return b & 1 ? a.power(b - 1) * a : (a * a).power(b / 2);
    }
};

int main() {
    int n, m, k;
    cin >> n >> m >> k;

    Matrix<101> g{};
    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        g.m[a][b]++;
    }

    auto p = g.power(k);
    cout << p.m[1][n] << "\n";
}
```

1722 · Fibonacci Numbers

Task · source analysis length 2,028 characters

Problem statement

The Fibonacci numbers can be defined as follows:

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-2} + F_{n-1}$

Your task is to calculate the value of F_n for a given n.

Input

The only input line has an integer n.

Output

Print the value of F_n modulo $10^9 + 7$.

Constraints

- $0 \le n \le 10^{18}$

Solution 1

We create a matrix

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

that has the following property:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_i \\ F_{i+1} \end{bmatrix} = \begin{bmatrix} F_{i+1} \\ F_{i+2} \end{bmatrix}$$

Then, we can calculate any value of F_n using matrix exponentiation:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{bmatrix} F_0 \\ F_1 \end{bmatrix} = \begin{bmatrix} F_n \\ F_{n+1} \end{bmatrix}$$

The resulting algorithm works in $O(\log n)$ time.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

template <int N>
struct Matrix {
    ll m[N][N];
```

```
    static Matrix<N> ident() {
        Matrix<N> r{};
        for (int i = 0; i < N; i++) r.m[i][i] = 1;
        return r;
    }

    Matrix<N> operator*(Matrix<N> b) {
        Matrix<N> r{};
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                for (int k = 0; k < N; ++k) {
                    r.m[i][j] += m[i][k] * b.m[k][j];
                    r.m[i][j] %= M;
                }
            }
        }
        return r;
    }

    Matrix<N> power(ll b) {
        if (b == 0) return ident();
        auto a = *this;
        return b & 1 ? a.power(b - 1) * a : (a * a).power(b / 2);
    }
};

int main() {
    ll n;
    cin >> n;

    Matrix<2> m{{{0, 1}, {1, 1}}};
    auto p = m.power(n);
    cout << p.m[0][1] << "\n";
}
```

Solution 2

Here is a shorter ad hoc solution that doesn't need matrix calculations. The solution is based on following identities:

- $F_{2n} = (2F_{n+1} - F_n$F_n)
- $F_{2n+1} = F_n^2 + F_{n+1}^2$

To make sure that the solution is efficient, we store each value in a map after calculating it. The solution works in $O(\log n)$ time.

```
#include <iostream>
#include <map>
using namespace std;
using ll = long long;
const int M = 1000000007;

map<ll, ll> dp;

ll f(ll n) {
    if (n <= 2) return n >= 1;
    if (dp.count(n)) return dp[n];
    auto a = f(n / 2);
    auto b = f(n / 2 + 1);
    if (n % 2 == 0) {
        dp[n] = (2 * b - a) * a;
    } else {
        dp[n] = a * a + b * b;
    }
    dp[n] = (dp[n] % M + M) % M;
    return dp[n];
}

int main() {
    ll n;
    cin >> n;
    cout << f(n) << "\n";
}
```

References

- Fibonacci number $Wikipedia$

1717 · Christmas Party

Task · source analysis length 2,852 characters

Problem statement

There are n children at a Christmas party, and each of them has brought a gift. The idea is that everybody will get a gift brought by someone else.

In how many ways can the gifts be distributed?

Input

The only input line has an integer n: the number of children.

Output

Print the number of ways modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10^6$

In this problem our task is to calculate the number of derangements of n elements, i.e., the number of permutations of n elements where no element has its original position.

Solution 1

Let us first calculate the number of permutations that are *not* derangements. We can do that using the inclusion-exclusion technique. After that, we subtract the result from the number of all permutations.

We calculate a value $f(n, i)$ for $i = 1, 2, \ldots, n$: the number of permutations of n elements where i elements have their original positions and other elements have arbitrary positions. We use the formula

$$f(n,i) = \binom{n}{i}(n-i)!$$

where $\binom{n}{i}$ counts the number of ways to choose the elements that have their original positions and $(n-i!)$ counts the number of ways to order the other elements. If i is odd, we add $f(n, i)$ to the result, and if i is even, we subtract $f(n, i)$ from the result.

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int N = 1000000;

int power(int a, int b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

int inv(int x) {
    return power(x, M - 2);
}
```

```
ll fac[N + 1];

int ncr(int a, int b) {
    return fac[a] * inv(fac[b] * fac[a - b] % M) % M;
}

int main() {
    fac[0] = 1;
    for (int i = 1; i <= N; i++) {
        fac[i] = fac[i - 1] * i % M;
    }

    int n;
    cin >> n;
    ll count = 0;
    for (int i = 1; i <= n; i++) {
        ll add = ncr(n, i) * fac[n - i] % M;
        if (i % 2 == 0) add = -add;
        count += add;
    }
    ll answer = ((fac[n] - count) % M + M) % M;
    cout << answer << "\n";
}
```

Solution 2

We can also count the number of derangements recursively as follows:

$$d(n) = (n-1)(d(n-2) + d(n-1))$$

The base cases are $d(1$=0) and $d(2$=1).

We can derive the formula as follows: Suppose there are n children and there is a fixed child x. We know that child x gives a gift to some child y and there are $n-1$ ways to choose child y. Then there are two cases:

- Case 1: Child y also gives a gift to child x. In this case we can remove children x and y and there are $d(n-2)$ ways to distribute gifts to the remaining $n-2$ children.
- Case 2: Child y gives a gift to some other child. Now let us look at any gift distribution to $n-1$ children without child x. There are $d(n-1)$ such distributions, and there is a single way to modify each distribution so that child x is included: child x gives a gift to child y and then the child who originally gave a gift to child y gives it to child x.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;
const int M = 1000000007;

int main() {
    int n;
    cin >> n;

    vector<ll> count(n + 1);
    count[1] = 0;
    count[2] = 1;
    for (int i = 3; i <= n; i++) {
        count[i] = (i - 1) * (count[i - 2] + count[i - 1]) % M;
    }
    cout << count[n] << "\n";
}
```

References

- Derangement $Wikipedia$

1716 · Distributing Apples

Task · source analysis length 1,144 characters

Problem statement

There are n children and m apples that will be distributed to them. Your task is to count the number of ways this can be done.

For example, if $n = 3$ and $m = 2$, there are 6 ways: $[0, 0, 2]$, $[0, 1, 1]$, $[0, 2, 0]$, $[1, 0, 1]$, $[1, 1, 0]$ and $[2, 0, 0]$.

Input

The only input line has two integers n and m.

Output

Print the number of ways modulo $10^9 + 7$.

Constraints

- $1 \le n, m \le 10^6$

We can calculate the answer using the formula

$$\binom{n+m-1}{m}$$

where n is the number of children and m is the number of apples.

The idea is that there is a one-to-one mapping between apple distributions and strings of length $n+m-1$ that have m times character X and $n-1$ times character $\rightarrow$. The interpretation of each string is as follows: We begin at the first child and read the string from left to right. If the character is X, we give an apple to the current child, and if the character is $\rightarrow$, we move to the next child.

```
#include <iostream>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int N = 2000000;

int power(int a, int b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

int inv(int x) {
    return power(x, M - 2);
}

ll fac[N + 1];

int ncr(int a, int b) {
    return fac[a] * inv(fac[b] * fac[a - b] % M) % M;
}

int main() {
```

```
    fac[0] = 1;
    for (int i = 1; i <= N; i++) {
        fac[i] = fac[i - 1] * i % M;
    }

    int n, m;
    cin >> n >> m;
    cout << ncr(n + m - 1, m) << "\n";
}
```

References

- Stars and bars *combinatorics Wikipedia*

1715 · Creating Strings II

Task · source analysis length 2,183 characters

Problem statement

Given a string, your task is to calculate the number of different strings that can be created using its characters.

Input

The only input line has a string of length n. Each character is between a–z.

Output

Print the number of different strings modulo $10^9 + 7$.

Constraints

- $1 \le n \le 10^6$

Solution 1

We first count the number of times each character appears in the string. After that, we go through the characters and calculate the answer using binomial coefficients.

Suppose the string has n characters and the first character appears k times in the string. This means that there are $\binom{n}{k}$ ways to choose the positions for the first character. After that, there are $n - k$ positions left and we continue in the same way.

```
#include <iostream>
#include <map>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int N = 1000000;

int power(int a, int b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

int inv(int x) {
    return power(x, M - 2);
}

ll fac[N + 1];

int ncr(int a, int b) {
    return fac[a] * inv(fac[b] * fac[a - b] % M) % M;
}

int main() {
    fac[0] = 1;
    for (int i = 1; i <= N; i++) {
        fac[i] = fac[i - 1] * i % M;
    }
```

```
    string s;
    cin >> s;
    int n = s.size();

    map<char, int> count;
    for (auto c : s) count[c]++;

    ll answer = 1;
    for (auto [c, k] : count) {
        answer = answer * ncr(n, k) % M;
        n -= k;
    }
    cout << answer << "\n";
}
```

Solution 2

We can also calculate a multinomial coefficient

$$\binom{n}{k_1, k_2, \ldots, k_m} = \frac{n!}{k_1! k_2! \ldots k_m!}$$

which corresponds to the number of ways we can divide n elements into m sets whose sizes are $k_1, k_2, \ldots, k_m$.

In this problem m equals the number of characters and k_i denotes the number of times the ith character appears in the string.

```
#include <iostream>
#include <map>
using namespace std;
using ll = long long;
const int M = 1000000007;
const int N = 1000000;

int power(int a, int b) {
    if (b == 0) return 1;
    ll u = power(a, b / 2);
    u = u * u % M;
    if (b % 2 == 1) u = u * a % M;
    return u;
}

int inv(int x) {
    return power(x, M - 2);
}

ll fac[N + 1];

int main() {
    fac[0] = 1;
    for (int i = 1; i <= N; i++) {
        fac[i] = fac[i - 1] * i % M;
    }

    string s;
    cin >> s;
    int n = s.size();

    map<char, int> count;
    for (auto c : s) count[c]++;

    ll answer = fac[n];
    for (auto [c, k] : count) {
        answer = answer * inv(fac[k]) % M;
    }
    cout << answer << "\n";
}
```

References

- Wikipedia $\mathit{Multinomialtheorem}$

1713 · Counting Divisors

Task · source analysis length 1,327 characters

Problem statement

Given n integers, your task is to report for each integer the number of its divisors.

For example, if $x = 18$, the correct answer is 6 because its divisors are $1, 2, 3, 6, 9, 18$.

Input

The first input line has an integer n: the number of integers.

After this, there are n lines, each containing an integer x.

Output

For each integer, print the number of its divisors.

Constraints

- $1 \le n \le 10^5$
- $1 \le x \le 10^6$

Solution 1

The following function `count` returns the number of divisors for x. It checks every integer k between 1 and $\sqrt{x}$. If x is divisible by k, we have found one divisor. In addition, if $k \neq x/k$, we have found another divisor x/k.

The solution works in $O(n\sqrt{N})$ time where N is the upper bound for x.

```
#include <iostream>
#include <vector>
using namespace std;

int count(int x) {
    int c = 0;
    for (int k = 1; k * k <= x; k++) {
        if (x % k) continue;
        c++;
        if (k != x / k) c++;
    }
    return c;
}

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        cout << count(x) << "\n";
    }
}
```

Solution 2

This solution works in a similar manner to the sieve of Eratosthenes. It creates a vector `count` that will contain the number of divisors for each number x between 1 and N.

Creating the vector takes $O(N \log N)$ time because the total number of steps is bounded by the harmonic sum

$$N + N/2 + N/3 + \cdots = O(N \log N).$$

Thus, the solution works in $O(N \log N + n)$ time.

```
#include <iostream>
#include <vector>
using namespace std;
const int N = 1000000;

int main() {
    vector<int> count(N + 1);
    for (int i = 1; i <= N; i++) {
        for (int j = i; j <= N; j += i) {
            count[j]++;
        }
    }

    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        cout << count[x] << "\n";
    }
}
```

1711 · Distinct Routes

Task · source analysis length 2,100 characters

Problem statement

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n.

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Input

The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered $1, 2, \ldots, n$.

After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b.

There are no two teleporters whose starting and ending room are the same.

Output

First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Constraints

- $2 \le n \le 500$
- $1 \le m \le 1000$
- $1 \le a, b \le n$

The maximum number of distinct paths equals the maximum flow in the graph. The following solution uses the scaling algorithm to find the maximum flow.

We also need to identify the distinct paths. We can find each path using a greedy algorithm that finds a path from the source to the sink where every edge has flow. After finding a path, we remove the flow from the path edges.

```
#include <iostream>
#include <vector>
using namespace std;
using ll = long long;

struct MaxFlow {
    static const ll INF = 1e18;

    struct Edge {
        int from;
        int to;
        ll w;
        bool real;
    };

    int n, source, sink;
    vector<vector<int>> g;
    vector<Edge> edges;
    vector<bool> seen;
    ll flow = 0;
```

```
    MaxFlow(int n, int source, int sink)
        : n(n), source(source), sink(sink), g(n) {}

    int add_edge(int from, int to, ll forward, ll backward = 0) {
        const int id = (int)edges.size();
        g[from].emplace_back(id);
        edges.push_back({from, to, forward, true});
        g[to].emplace_back(id + 1);
        edges.push_back({to, from, backward, false});
        return id;
    }

    bool dfs(int node, ll lim) {
        if (node == sink) return true;
        if (seen[node]) return false;
        seen[node] = true;
        for (int i : g[node]) {
            auto &e = edges[i];
            auto &back = edges[i ^ 1];
            if (e.w >= lim) {
                if (dfs(e.to, lim)) {
                    e.w -= lim;
                    back.w += lim;
                    return true;
                }
            }
        }
        return false;
    }

    ll max_flow() {
        for (ll bit = 1ll << 62; bit > 0; bit /= 2) {
            bool found = false;
            do {
                seen.assign(n, false);
                found = dfs(source, bit);
                flow += bit * found;
            } while (found);
        }
        return flow;
    }
};

int main() {
    int n, m;
    cin >> n >> m;

    MaxFlow flow(n + 1, 1, n);
    for (int i = 1; i <= m; i++) {
        int a, b;
        cin >> a >> b;
        flow.add_edge(a, b, 1);
    }

    int k = flow.max_flow();
    cout << k << "\n";

    for (int i = 1; i <= k; i++) {
        int node = 1;
        vector<int> path;
        path.push_back(1);

        do {
            for (auto id : flow.g[node]) {
                auto edge = flow.edges[id];
                if (edge.w == 0 && edge.real) {
                    node = edge.to;
                    path.push_back(node);
                    flow.edges[id].w = 1;
                    break;
                }
            }
        } while (node != n);

        cout << path.size() << "\n";
        for (auto node : path) {
            cout << node << " ";
        }
        cout << "\n";
    }
}
```